

Online Hospital Management System for HealthCare Plus Hospital.

Prepared by: Edirisinghe EMMGDB

E2240189

Bachelor of Information Technology

Center for Open and Distance Learning

University of Moratuwa

Sri Lanka

25.06.2026

Declaration.

We declare that this report is my own work and has not been submitted in any form for another degree or diploma at any university or other institution of tertiary education. Information derived from the published or unpublished work of others has been acknowledged in the text and a list of references is given.

Name of Student: Edirisinghe EMMGDB

Signature of Student: 

Date: 24.06.2026

Supervised by,

Name of the Supervisor: Mrs. Thumashi De Silva

Signature of Supervisor(s): 

Date: 24.06.2026

Abstract.

This is because the existing process of managing a hospital, which involves a lot of documentation and is done manually, gives rise to scattered patient information, redundant information, inefficient appointment handling, delay in billing, and a lack of coordination between hospital personnel. This inefficiency has a negative impact on patient satisfaction.

The proposed project aims to demonstrate the design and development of an Online Hospital Management System to overcome these problems. The proposed system has been designed with multiple modules like patient registration, appointment scheduling, management of doctors and staff, billing, and reporting. The system will also be facilitated with role-based security. The patient engagement features in this system will be further enhanced with an online symptom checker facility.

The system was designed with the Agile development process, which facilitated improvisation. The front-end technology used HTML, CSS, and JavaScript, with the server-side technology PHP and MySQL for secure data storage. The system's architecture takes a three-tier approach for scalability. This system meets the requirements as indicated by the system specification.

System analysis and evaluation of the system were done by unit testing, integration testing, and user-functional testing. The results showed the correctness of data processing, conflict prevention during appointment, successful billing processing, and reduced response time compared to the existing system.

From the results of this project, it is observed that the system has a significant impact on efficiency, accuracy, and decision-making at the HealthCare Plus Hospital. The reason behind this is that this system replaces traditional hospital management practices with an efficient online solution.

Acknowledgement.

This project named Online Hospital Management System for HealthCare Plus Hospital will be developed by Edirisinghe EMMGDB who is a student from University of Moratuwa in BIT 24S2 Batch. The developer gratefully thanked the people who helped and gave valuable ideas and advice to successfully complete this Final Report document. Especially Mrs. Thumashi De Silva as the supervisor who contributed her maximum effort to improve our knowledge and gave the clear and suitable ideas and advice to finish this assignment in a limited time.

Thank you,

Edirisinghe EMMGDB.

Table of Content.

Declaration.....	i
Abstract.....	ii
Acknowledgement.....	iii
Table of Content.....	iv
List of Figures.....	vi
List of Tables.....	vii
List of Abbreviations.....	viii
1. Chapter I - Introduction.....	1
1.1. Introduction.....	1
1.2. Motivation and Significance of the Project.....	2
1.3. Aim and Objectives.....	3
1.4. The intended audience.....	4
1.5. Scope of the Project.....	4
1.6. Approach.....	6
1.7. Assumptions.....	7
1.8. Summary.....	8
2. Chapter II – Background.....	9
2.1. Introduction.....	9
2.2. Problem.....	9
2.3. Stakeholders.....	10
2.4. Existing Solutions.....	11
2.5. Research Gap.....	20
2.6. Review Objectives.....	21
2.7. Methods and Tools.....	21
2.8. Summary.....	29
3. Chapter III – Specification and Design.....	30
3.1. Introduction.....	30
3.2. User Interface.....	30
3.3. Dynamic Behavior of the System.....	41
3.4. System Architecture.....	43
3.5. Diagrams.....	44
3.6. Data Structures and Algorithms Implemented.....	52
3.7. Constraints and Design Justification.....	53
3.8. Summary.....	53
4. Chapter IV – Implementation.....	54
4.1. Introduction.....	54
4.2. Problems during Implementation.....	54
4.3. Coding Segment.....	55
4.4. Implementation Code.....	59
4.5. Summary.....	64
5. Chapter V – Result and Evaluation.....	65
5.1. Introduction.....	65
5.2. Plan of Evolution.....	65
5.3. Test Plan and Test Cases.....	66
6. Chapter VI – Future Work.....	75
6.1. Introduction.....	75
6.2. Future Works.....	75

6.3. Summary.	76
7. Chapter VII – Conclusions.....	77
7.1. Introduction.	77
7.2. Conclusion.....	77
7.3. Summary.	77
8. Chapter VIII – Reflection.....	78
8.1. Introduction.	78
8.2. Lessons Learned.....	78
8.3. Summary.	78
References.....	79
Appendices.	80
Appendix A: Use Case Diagrams.	80
Appendix B: Activity Diagrams.	82
Appendix C: Source Code for the System.	85
Appendix D: Software Requirement Specification (SRS).....	86

List of Figures.

Figure 1: SimplyBook.me.....	12
Figure 2: CS Channeling.....	14
Figure 3: Practo.....	15
Figure 4: OpenMRS.....	17
Figure 5: Bahmni.	19
Figure 6: Comparison Table.	19
Figure 7: Review of Existing Systems.....	20
Figure 8: Agile Methodology.....	22
Figure 9: XAMPP Server.....	28
Figure 10: Notepad++.....	28
Figure 11: Visual Studio Code.....	29
Figure 12: Login Page.....	30
Figure 13: Registration Page.....	31
Figure 14: Sign Up Page.	31
Figure 15: Home Page.	32
Figure 16: Admin Dashboard.....	33
Figure 17: Doctor Management.	33
Figure 18: Appointment Management Page.	34
Figure 19: Schedule Management.	34
Figure 20: Patient Management Page.	35
Figure 21: Feedback Management.....	35
Figure 22: Report Generation.	36
Figure 23: System Statistics.....	36
Figure 24: Patient Dashboard.....	37
Figure 25: Book Appointment.	37
Figure 26: Payment.....	38
Figure 27: Booking History.	38
Figure 28: AI-Powered Symptom Checker.....	39
Figure 29: Doctor Dashboard.....	39
Figure 30: Doctor Schedules.....	40
Figure 31: Patient List.....	40
Figure 32: System Architecture.	43
Figure 33: Use Case Diagram.....	44
Figure 34: Enhanced Entity Relationship Diagram.	45
Figure 35: Database for the System.	55
Figure 36: Admin Table.....	56
Figure 37: Allergies Table.	56
Figure 38: Appointment Table.....	56
Figure 39: Chatbot Conversation.	57
Figure 40: Doctor Table.....	57
Figure 41: Patient Table.....	57
Figure 42: Feedback Table.....	58
Figure 43: Reports Table.	58
Figure 44: Schedule Table.	58
Figure 45: Code for Authentication.	59
Figure 46: Code for Registration Page.....	59
Figure 47: Code for Home Page.	60

Figure 48: Code for Admin Dashboard.	60
Figure 49: Code for Patient Dashboard.	61
Figure 50: Code for Doctor Dashboard.	61
Figure 51: Code for Report Generation.	62
Figure 52: Code for Payment Page.	62
Figure 53: Code for Email.	63
Figure 54: Code for Chatbot.	63
Figure 55: Code for Feedback.....	64
Figure 56: Test Case 01 Output.	69
Figure 57: Test Case 02 Output.	70
Figure 58: Test Case 03 Output.	71
Figure 59: Test Case 04 Output.	72
Figure 60: Test Case 05 Output.	73
Figure 61: Test Case 06 Output.	74
Figure 62: Use Case for Patient Registration.....	80
Figure 63: Use Case for Booking Appointment.	80
Figure 64: Use Case for Payment.	81
Figure 65: Use Case for Feedback and Notification.....	81
Figure 66: Activity Diagram for Checking Doctor Availability.....	82
Figure 67: Activity Diagram for Payment.	83
Figure 68: Activity Diagram for Booking Appointment.	84

List of Tables.

Table 1: Software Components.....	25
Table 2: Hardware Components.	25
Table 3: Relationships and Cardinalities.	52
Table 4: Algorithms Overview.	52
Table 5: Justification of Technology Choices.	53
Table 6: Test Case 01.....	69
Table 7: Test Case 02.....	70
Table 8: Test Case 03.....	71
Table 9: Test Case 04.....	72
Table 10: Test Case 05.....	73
Table 11: Teset Case 06.	74

List of Abbreviations.

SRS – Software Requirements Specification.

HTML – Hyper Text Markup Language.

CSS – Cascading Style Sheets.

PHP – Hypertext Preprocessor.

EER – Enhanced Entity Relationship Diagram.

HIS – Hospital Information System.

EMR – Electronic Medical Record.

HMS – Hospital Management System.

1. Chapter I - Introduction.

1.1. Introduction.

The healthcare sector is undergoing a transformation and there is a growing need for faster, accurate and personalized healthcare services. Despite this, a significant number of healthcare institutions, especially the smaller ones, adopt a manual system of healthcare service management that is carried out through paper documentation for registration, appointments and basic file management. HealthCare Plus Hospital is one such healthcare facility. They incorporate a manual system for handling critical tasks such as patient registration, doctor appointments, staff meetings, reporting and billing. This manual system creates numerous problems, such as lost documentation, delayed access to information, lengthy waiting periods, information duplication and errors that result from basic administrative processes.

In order to overcome such drawbacks, this project suggests the development of an Online Hospital Management System that is web-based and tailored for HealthCare Plus Hospital use only. Devised primarily for automating certain inefficient manual processes performed at HealthCare Plus hospital, this proposed system aims at elimination of certain inefficient manual processes and instead uses an automatic digital system. By carrying out different tasks such as patient registration, appointment, doctor and staff administration, billing and generation of reports within the single prescribed system at HealthCare Plus Hospital, increased efficiency and timeliness of decision-making can be ensured.

In addition, it proposes additional functionality like SMS notification and an artificial intelligence-based symptom evaluator for improving patient engagement and accessibility. By automating daily tasks and eliminating the need for unwanted paperwork, the proposed online hospital management system would not only minimize costs and maximize savings in terms of precious time, it would also ensure that HealthCare Plus hospital provides more credible and pleasant medical facilities to the patients. Ultimately, this proposed project would play its role in the modernization of HealthCare Plus Hospital, helping it achieve an efficient and patient-centered operational model.

1.2. Motivation and Significance of the Project.

The motivation for the development of the Online Hospital Management system for HealthCare Plus hospital stems from the necessity to automate hospital processes and increase patient satisfaction. A time when healthcare delivery is becoming patient-centric, effective delivery of services and immediate access to healthcare are a necessity. The shortcomings of the existing manual system (slower administrative procedures, greater likelihood of errors and no immediate access to data) render it inevitable that an effective, technology-based system be adopted.

This project is significant for several reasons:

- **Operational Efficiency:** Streamlining repetitive administrative work will minimize paperwork, save time and enable staff to deal with more issues like patient care instead of handling paper records.
- **Data Accuracy and Reliability:** Storing centrally in digital form will prevent duplication, provide consistent management of data and minimize the chances of human error in handling records.
- **Enhanced Patient Engagement:** Patients may use online appointment booking, medical histories, billings and a feedback system, introducing increased transparency and convenience.
- **Informed Decision Making:** The module for integrated reporting will provide timely, real-time accurate data to allow administrators to plan strategically and monitor performance.
- **Regulatory Compliance and Security:** Role-based access control, encrypted storage and secure authentication will be used to provide assurance that the system complies with healthcare data protection standards.

1.3. Aim and Objectives.

1.3.1. Aim.

To design and develop a web-based Online Hospital Management System that automates patient, appointment, billing and administrative processes at HealthCare Plus, is to revolutionize the healthcare experience by leveraging cutting-edge technology.

1.3.2. Objectives.

The following are the objectives to be fulfilled so that the aim of this project is fulfilled successfully.

- To implement an automated patient registration and profile management system.
- To develop an online appointment scheduling process that prevents conflicts and reduces waiting time.
- To provide a billing and payment management module that ensures accurate and timely financial processing.
- To enable role-based access for administrators, doctors and patients.
- To generate system reports that support administrative decision-making.
- To enhance patient engagement through features such as notifications and AI-based symptom checker.

1.4. The intended audience.

Target users of the proposed Online Hospital Management System include all the key stakeholders who interact with the services of the hospital every day. First, the system will serve the hospital administration who would greatly need a system that is able to deal with patient, employee, appointments, payment as well as system generated report services. Doctors as well as employees would form the other large group of users, as the system will be helpful for accessing patient information faster, scheduling, updating, as well as assisting them with their workflow. Finally, hospital employees will greatly benefit from efficient appointment processing, fast retrieval, as well as reduced workload. Lastly, patients will form the other group, as the system will give them the ability to do online registration, appointment, as well as allow them to see their own existing personal records. The project supervisor, the evaluators and all interested parties and the developer are the intended audience of this document.

Hospital Administrators: Manage users, appointments, billing, staff records and system reports.

Doctors: Access patient information, schedules and medical reports.

Patients: Register online, book appointments, make payments and view personal medical and billing records.

The **project supervisor, evaluators and other academic reviewers** are part of the intended audience of this document.

1.5. Scope of the Project.

The Online Hospital Management System is developed to automate and simplify hospital operations by granting single-window access to patient care and administration process management. The system allows users such as patients, doctors, hospital staff and admin to view and manage health services effectively. Key functionality includes, patient registration, appointment scheduling, electronic medical records management, payment and billing and allocation of hospital resources. The system will also support to feature features like doctor and department management, prescription processing and reporting

tools to analysis and monitor. It will be accessible using both web and mobile interfaces to enable broad accessibility over all possible devices.

The primary features of Online Hospital Management System include:

- Patient Management.
 - Patient can create a user account.
 - Registered patients can login to the system using their email and password.
 - Admin can block unnecessary patients.
 - Patient can view, update and delete their profile.
- Appointment Scheduling.
 - Admin can handle booking details.
 - Patient can view booking details.
 - Patient can select doctors.
 - Patient can search doctor availability.
 - Patient can cancel the appointment booking.
 - Admin can manage appointment schedule and time table.
- Billing and Payment.
 - Patient can make payments.
 - Admin can issue the receipt.
 - Patient can collect their receipt after the payment.
- Medical Record and Report Management.
 - Doctor can view medical reports.

- Admin can generate reports.
- Patients can view annual reports.
- AI-Powered Symptom Checker.
 - Allows patients to input symptoms and receive preliminary guidance on possible condition before scheduling an appointment.
- Staff and Resource Management.
 - Staff members can login to the system.
 - Admin can add, update and delete staff members.
 - Admin can provide relevant user privileges.
 - Admin can provide user name and password.
- Feedback and Support.
 - Patients can give feedbacks by filling the feedback form.
 - Admin can reply to feedbacks.
 - Staff can view feedbacks

1.6. Approach.

Development of the HealthCare Plus Online Hospital Management system will be following the Agile Methodology, which focuses on iterative development, ongoing feedback and close collaboration with the stakeholders. Rather than providing the system as an end product in one shot, the project will be split into smaller, manageable chunks known as sprints. Each sprint will concentrate on deploying one feature like patient registration, appointment scheduling, doctor and staff management, billing or reporting. Stakeholders such as admin, patients, staff and doctors will have a review of the functional prototype at the end of each sprint in order to collect feedback. This enables the system to expand according to real world requirements and adapt to evolving needs.

The Agile implementation benefits HealthCare Plus Hospital in a variety of ways. To begin with, it provides enhanced flexibility and responsiveness, with the hospital able to deliver highest-priority features (e.g., appointment scheduling and billing) in the initial sprints and tweak or introduce new features (e.g., AI Symptom Checker and Reminder through SMS) in the subsequent iterations. This decreases the chances of failure, as stakeholders are constantly kept engaged and enhancements are incorporated along the way.

Second, Agile facilitates quicker delivery of value to the hospital. Rather than holding up the whole system for completion, functional modules are delivered in advance so that HealthCare Plus can begin experiencing the benefits of automating manual tasks, increasing patient satisfaction and minimizing administrative mistakes from day one. In general, Agile provides a disciplined but adaptive approach that synchronizes with the changing operational needs of the hospital with assurance of user-centric, high quality results.

1.7. Assumptions.

Stable Internet and Hardware Availability: This assumes that access to Internet and computer hardware in the healthcare facility is possible or available throughout the facility.

Accurate and Updated Data: The hospital should provide current and accurate data so that the data stored in the system compares favorably with the actual data.

User Participation and Feedback: All stakeholders are assumed to contribute towards requirements gathering and testing phases in terms of providing feedback.

Operational Readiness: The hospital administration's willingness to implement changes in current business processes in support of the new system and assist employees in the change from manual procedures to digital system operation is assumed.

Scalability Requirements: The system has been developed in consideration of the fact that in the future, HealthCare Plus may extend its services and the system needs to be scalable to support more modules or users.

Consistent Workflow Environment: The hypothesis here assumes that overall hospital workflows compatible with appointment scheduling and patient administration and billing functions would be relatively stable and unchanging in such a way that there would not be many structurally changing requirements.

The users (patients, staff, doctors and administrators) will have access to a stable internet connection (minimum 5Mbps bandwidth).

The system will primarily be accessed via desktop or laptop computers running Windows, Linux or Mac operating systems.

All users will have basic knowledge of operating a web-based system, including navigating web pages and filling out forms.

Users will have modern web browsers (Google Chrome or Microsoft Edge) that support the technologies used (HTML, CSS, JavaScript).

Security protocols such as HTTP and password encryption will be implemented by developers as part of best practices.

The system will run on localhost during development and on a local server or intranet upon initial deployment.

MySQL will be the database used, and data integrity will be maintained through properly designed relational schemas.

1.8. Summary.

This section gives an introduction to the project, where the objectives, intended audience, and assumptions regarding the project are discussed. This section provides the reader with an outline and a background that helps them understand the project and objectives. Also, it explains the nature of the project, which serves as an introduction into what is going to be discussed. By reading this chapter, the audience will understand the reasons behind the project, its expected outcomes and the structures approach that will be taken to achieve its aims.

2. Chapter II – Background.

2.1. Introduction.

The health care sector is undergoing a swift change spurred on by technological growth, escalating patient expectations and changing trends in health care. Conventional manual systems of patient record management, appointments and billing are becoming less and less effective and are prone to errors. This is seriously challenging the hospitals in rendering on-time quality care and maximizing resource utilization.

HealthCare Plus hospital is on a manual system at present, which is presenting some operational problems. Critical information is scattered in various documents and thus leading to inefficiency in handling of patients, coordination among the staff and reports. Administrative procedures like appointment follow-up, patient data updates and payment transaction processing are time-consuming and prone to error. All this has necessitated the need for a complete online solution, which will make the operations streamlined, provide accuracy of information and improve the quality of service.

The Online Hospital Management System is intended to resolve these problems by consolidating central hospital process (patient registration, appointment scheduling, doctor and staff management, billing, reporting and secure data storage) into a single computerized system. The centralized system will exchange isolated manual processes with an extensible, secure and user-friendly platform for retrieval of up-to-date information in real-time by all valid users, including administrators, health care providers, staff and patients.

2.2. Problem.

HealthCare Plus hospital currently uses a manual system for the management and maintenance of critical information. The current system requires numerous paper forms, with data stores spread throughout the hospital management infrastructure. Often information (on forms) is incomplete, or does not follow management standards. Multiple copies of the same information exist in the hospital and may lead to inconsistencies in data in various data stores.

There are a number of documents to be maintained in the HealthCare Plus hospital and this information typically involves, patient personal information and medical history, staff information, appointments and reports, staff scheduling and other facilities. All of this information must be managed in an efficient and cost wise fashion so that the resources can be managed effectively. The admin handles various appointments and reports of the patients. The developer should implement important functionalities and features to the system by identifying system functionalities.

Challenges include:

- Difficulty in retrieving patient records.
- Appointment scheduling conflicts and long waiting times.
- Manual billing processes prone to calculation errors.
- Limited reporting and lack of real-time operational insight.
- High administrative workload on staff and management.

2.3. Stakeholders.

There are four stakeholders in this web-based system.

2.3.1. Patients.

Role: Patients are the primary service recipients of the hospital. They use the system to register online, book appointments, view doctor availability, make payments, access receipts and review their personal medical information.

Interests: Patients are looking for an easily accessible and user-friendly system, which would help minimize waiting and the use of manual documentation. They are expecting immediate access to available appointment times, billing information and reminder

2.3.2. Doctors.

Role: The system helps doctors check patient information, appointment schedules and update doctor information as well as produce medical reports.

Interests: Doctors would like tools that provide them with reliable and easily accessible patient information for assisting the process of clinical decision-making. They would like tools that help the administrative process and assist them to view their schedules and consulting process easily. Accuracy, efficiency and less manual entry are their priorities.

2.3.3. Hospital Staff.

Role: Staff members, such as the reception and operations teams, utilize the system to process patient registration, process appointment bookings, generate receipts and process payment.

Interests: Staff would like the system to provide them with simplified administrative processes that would make their work easier and faster, as well as decrease the number of errors and the time it takes to deliver services. They would like the system to offer them simplified processes than make their work easier.

2.3.4. Administrators.

Role: Administrators are responsible for managing various operations of the system, user account management (patient, doctors and staff members), payment verification and the entire activity of the hospital system.

Interests: Management would like to find such a system that, besides the above advantages, would improve the quality of services, minimize operations costs and provide the management with real-time data. They would like the system to provide them with complete reports, increase satisfaction and increase the overall efficiency of the hospital.

2.4. Existing Solutions.

2.4.1. SimplyBook.me [1]

Cloud-based appointment booking platform used by clinics and small businesses; provides booking widgets, reminders and payment integration.

Advantages.

- Easy, fast setup for bookings and reminders (email, SMS, Telegram).

- Flexible booking channels (widget, Facebook/Instagram, own website).
- Intake forms and custom fields useful for pre-appointment triage.

Disadvantages.

- Not a full EMR/hospital information system – limited clinical record capabilities and reporting compared to EMR suites.
- Advanced billing, inventory or complex multi-role hospital workflows are limited or require integrations.

Suitability for HealthCare Plus.

Good fit for straightforward appointment scheduling and phone workload reduction (one of HealthCare Plus's highest requirements). Does not have doctor payment splits, enhanced reporting and EMR capabilities.

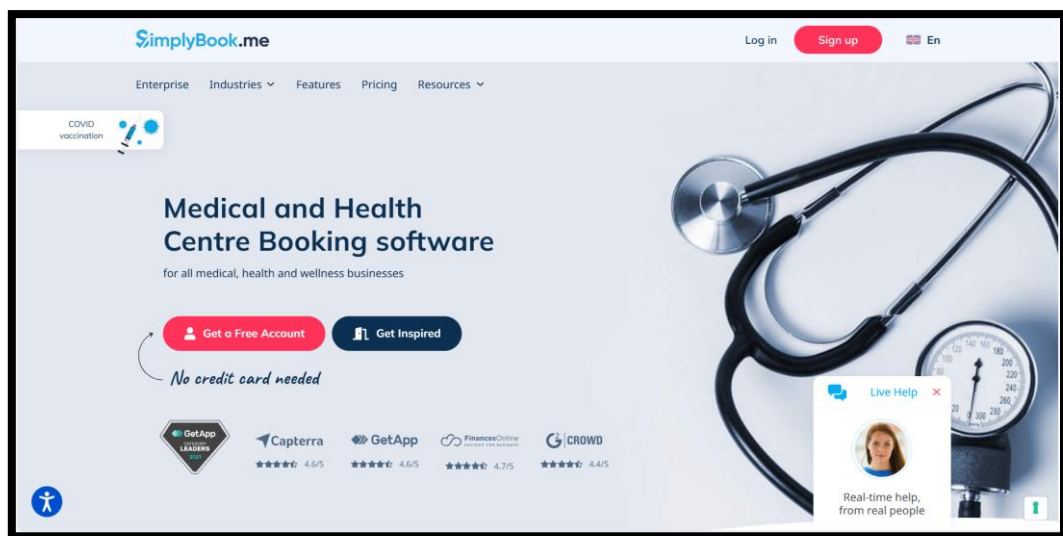


Figure 1: SimplyBook.me.

SimpleBook.me provides appointment solutions for a variety of industries including the healthcare and medical industry. This system is web-based and mobile-friendly. A web-based solution is not suited for a small practice like HealthCare Plus. The fact this system is complex and handles the entire appointment process, including payments, makes it complex for users who are switched from a manual system. [2]

2.4.2. CS Channeling [3]

CS Channeling is a Sri Lankan-developed Doctor Channeling Management System from CIS WORLD. It is designed specifically for private hospitals, channeling centers and doctors' clinics in Sri Lanka. The system is an expert in appointment scheduling, doctor channeling management, patient registration, payment and doctor charge reporting. The system is tailored towards local healthcare processes and suitable for small clinics and medium sized hospitals.

Advantages.

- Completely designed for Sri Lankan healthcare professionals, with locally implemented payment models, doctor payment plans and reports.
- Featuring doctor routing, patient registration, scheduling, payments and doctor fee management within a unified workflow.
- Has a simple user interface with minimal training for front-desk operators.
- Has real-time appointment management and receipt printing and token printing functionality.
- With local vendor support (CIS WORLD), implementation and after-sales are simpler than with global solutions.

Disadvantages.

- Mainly focused toward outpatient and channeling operations; lacks full EMR, lab and inventory feature relative to full-featured HIS solutions.
- Impeded scalability to support more sophisticated hospital functions like inpatient treatment or integrated ERP.
- Vendor dependence upon update and customization can lead to higher long-term costs.

Suitability for HealthCare Plus.

To HealthCare Plus, having by now optimized channeling of doctors, appointment booking and doctor fee handling as its priority, CS Channeling is well-suited. It is small, locally maintained and focuses directly on Sri Lankan hospital operations. But if the hospital does intend to expand out towards sophisticated clinical functions, integration into a complete HIS might subsequently become necessary.

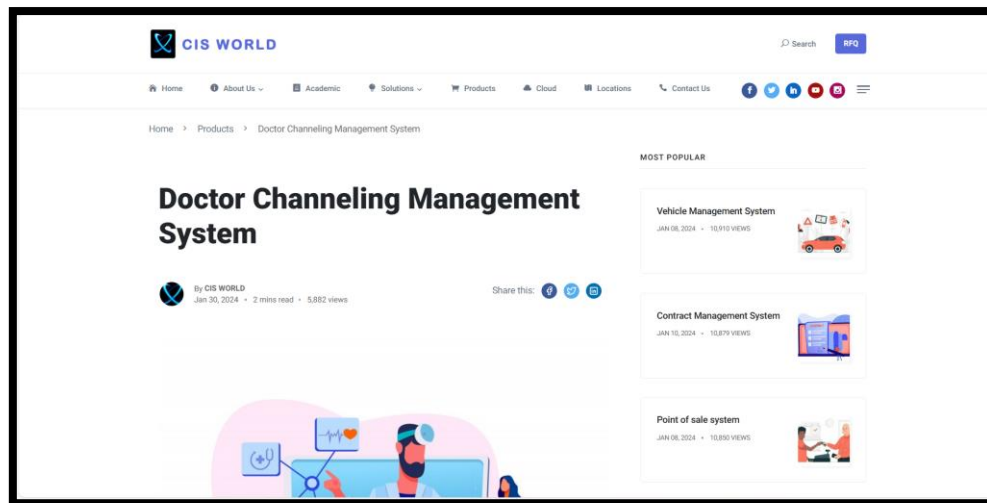


Figure 2: CS Channeling.

CS Channeling (Doctor Channeling Management System) is a Sri Lankan product by CIS WORLD that is specifically designed to automate doctor appointments among call centers, web sites and hospitals. It supports real-time booking, automatic reminders, patient and doctor administration, simple reports and even doctors' and patients' payment tracking on a daily basis and emailing of the fee for the appointment to the patient. It is easy to use and handy in simplifying paperwork, queue management optimization, enhancing communications and overall efficiency in channel centers. It lacks full EMR capability, clinical reporting and billing as is the case in hospital information systems, like other booking-oriented solutions.

That said, however, CS Channeling remarkably aligns HealthCare Plus's short-term need for streamlined booking, payment tracking and administrative ease and so is a local workable option-at least in the short term.

2.4.3. Practo. [4]

India-based healthcare platform with large doctor network; provides appointment booking, telemedicine, practice management (Practo Ray) and patient records. Designed specifically for clinics and hospitals.

Advantages.

- Comprehensive clinical and practice-management features: bookings, electronic records, billing and teleconsultation.
- Strong adoption in South Asian Markets – good local-language and payment support where Practo operates.

Disadvantages.

- Vendor is regionally focused (India). Integration and pricing models may not fit every local context; telemedicine/data residency rules differ by country.
- As a commercial product, costs and vendor lock-in can be issues for small centers.

Suitability for HealthCare Plus.

Pragmatically closer to HealthCare plus needs (appointments + records + billing). If HealthCare Plus needs full EMR and booking, Practo-like solutions are stronger than booking-only tools, but cost/regulatory fit must be examined.

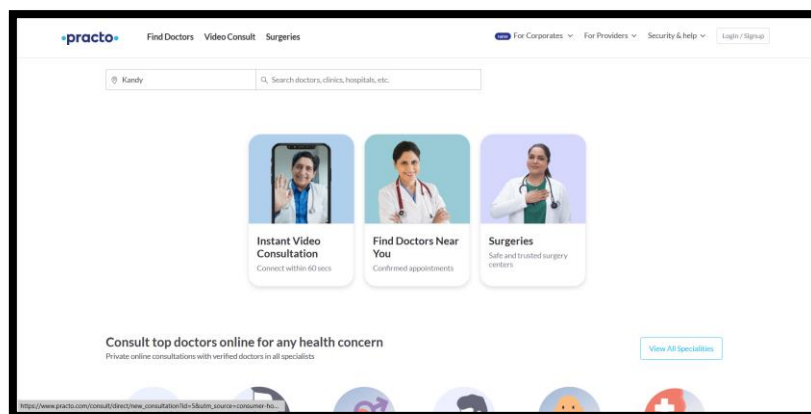


Figure 3: Practo.

Practo is a South Asia healthcare platform providing an end-to-end solution suite from online appointment scheduling to electronic patient records, billing, teleconsultation and practice management. It is optimal as a doctor-hospital administrator-facing as well as patient-convenience-advantaged healthcare workflow-based platform. Appointments are booked on the Practo portal by patients and doctors have scheduled time slots as well as access to patient history. Besides, the system also supports safe billing and reporting in close match to hospital requirements. As a proprietary product, it has associated licensing fees and potential vendor lock-in. Its regions of availability and legality might also extend differently beyond its primary market. For a hospital such as HealthCare Plus, Practo presents a strong combination of booking, EMR and billing capabilities, if the budget allows.

2.4.4. OpenMRS. [5]

Mature open-source EMR platform focused on flexible, extensible electronic medical records for low-resource settings and global health implementations. Widely used and community-driven.

Advantages.

- Designed for clinical data, configurate data model and strong community backing for low-resource settings.
- Good for structured patient records, reporting extensions and integration with lab systems.

Disadvantages.

- Not an out-of-the-box channeling/appointment product (requires UI modules or integrations for smooth patient-facing booking experiences).
- Implementation requires technical resources and configuration effort; smaller centers may find initial setup heavy.

Suitability for HealthCare Plus.

Excellent if HealthCare Plus prioritizes EMR and reporting and has ethical capacity for customization. For a lightweight channeling center that mainly needs appointment scheduling and simple records, OpenMRS alone will require front-end booking modules or integration.



Figure 4: OpenMRS.

OpenMRS is an open-source electronic medical record (EMR) software developed to be utilized in low-resource and developing country settings. OpenMRS boasts an extensible and flexible data model that enables healthcare providers to automate patient records, maintain clinical data and create structured health reports. Its largest benefit is its community-driven development and flexibility that makes it an excellent option for hospitals that require installing strong EMR systems without overcharging them. However, OpenMRS is not a ready-to-use hospital information system and demands high technical expertise to install, customize and integrate with patient-facing appointment scheduling modules. To HealthCare Plus, OpenMRS is a powerful long-term solution for medical record management and reporting but is probably too resource-intensive to immediate implementation without providing technical support.

2.4.5. Bahmni. [6]

An OpenMRS-based integrated hospital information system focused on low-resource hospitals and clinics (combines EMR, billing, lab, inventory and a simpler UI for workflows).

Advantages.

- Combines EMR, billing, lab and inventory in a single distribution (more “hospital ready” out of the box than core OpenMRS).
- Designed for low-resource settings and supports inpatient/outpatient workflows, standard reports and basic HIS features. Roadmap shows continued improvements (access control, IPD dashboards).

Disadvantages.

- Still requires technical know-how for deployment and may be heavier than a simple channeling system.
- Customization to local billing/consultant-pay models will require implementation effort (Odoo/ERP integration often used).

Suitability for HealthCare Plus.

If HealthCare Plus expects to evolve beyond channeling into richer clinical/financial operations, Bahmni provides a robust base. For immediate lightweight scheduling and consultant-charge reporting, Bahmni might be overkill unless the hospital commits to an implementation partner.

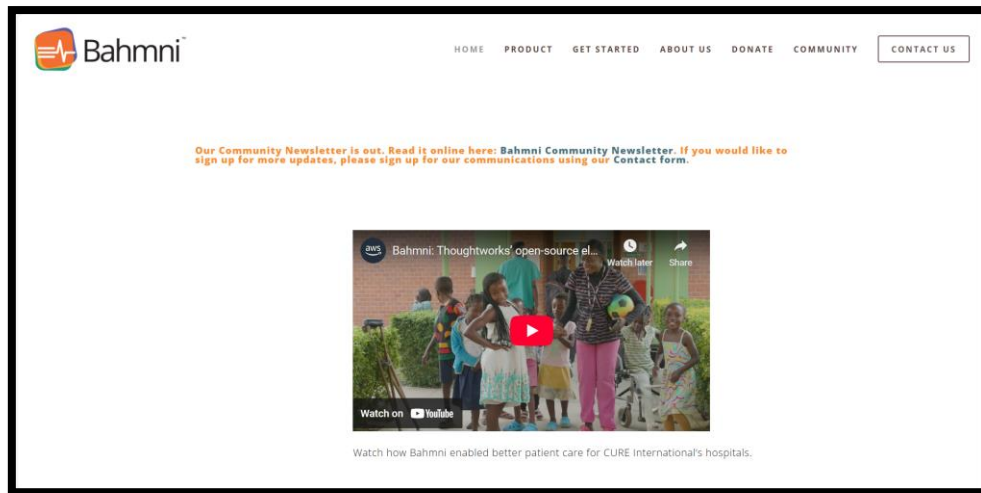


Figure 5: Bahmni.

Bahmni is an open-source hospital management system that is developed on top of OpenMRS to provide a complete hospital information system for small and medium hospitals. Unlike OpenMRS, Bahmni integrates electronic medical records, appointment scheduling, billing, laboratory integration and inventory management in a single package. This is more hospital-ready with end-to-end solutions both for clinical and administration processes. Its modularity facilitates customization across differing hospital settings with a comparatively more intuitive interface than OpenMRS. Its major disadvantage is that it entails a complicated implementation; successful deployment frequently requires specialized IT skills or implementation partner assistance. In spite of this, Bahmni can be a serious rival for HealthCare Plus in the long-term, particularly if the hospital intends to enhance its activities away from simple channeling operations into extensive clinical management.

System	Primary Focus	Patient Booking	EMR / Clinical Records	Billing & Receipts	Ease of Setup	Suitability for HealthCare Plus
SimplyBook.me	Appointment booking	Excellent	Limited	Basic (Payments)	Very Easy	Good for immediate online booking needs; lacks EMR & consultant reporting.
Practo (Ray)	Clinic/hospital practice management	Good	Good	Good	Medium (Vendor Integration)	Strong fit for clinics, booking and billing; check cost/regional fit.
OpenMRS	EMR platform	Needs front-end	Excellent	Needs modules	Requires development resources	Great for medical records; booking front-end must be added.
Bahmni	EMR + HIS	Possible with modules	Very Good	Built-in	High	Robust long-term HIS; heavier setup for a small channeling center.
CS Channeling (CIS WORLD)	Doctor channeling management	Good	Limited	Basic with doctor payments	Medium	Addresses local Sri Lanka channeling needs; useful for booking and doctor payments but lacks full EMR/reporting.

Figure 6: Comparison Table.

System	Key Focus	Strengths	Limitations	Overall Suitability
SimplyBook.me	Appointment booking	• Easy appointment scheduling and reminders • Multi-channel booking support	• No integrated medical records • Limited billing and reporting. • Not designed for full hospital operations	Reduces booking workload but inadequate for integrated hospital management
CS Channeling	Doctor channeling (Sri Lanka)	• Localized workflows and payment models. • Simple interface for front-desk use	• Focused mainly on outpatient channeling • Limited scalability and EMR support • Vendor-dependent customization	Suitable for short-term appointment needs but lacks long-term hospital capability
Practo	Appointment, billing, and EMR	• Comprehensive features including EMR and billing • Well-established platform	• High licensing costs • Limited customization for local workflows • Vendor lock-in risks	Feature-rich but financially and operationally unsuitable for smaller hospitals
OpenMRS	Electronic Medical Records (EMR)	• Strong clinical data management • Highly customizable	• Requires significant technical expertise • Not appointment-focused by default	Excellent for clinical records but complex for immediate hospital deployment
Bahmni	Hospital Information System	• Integrated EMR, billing, and inventory • Designed for low-resource hospitals	• Complex implementation • Requires technical partners	Powerful solution but excessive for HealthCare Plus's current scale

Figure 7: Review of Existing Systems.

2.5. Research Gap.

An analysis of the current appointment management system and the hospital management system that exists highlights the absence of an integrated system that suits the operational requirements of the health organizations that fall within the category of the medium-scale hospital, such as HealthCare Plus Hospital. Though the current solutions offered by SimplyBook.me or CS Channeling are very useful and efficient for managing appointments, they are not equipped with the functionality of managing electronic medical records, billing, or overall reporting at the hospital level. On the contrary, solutions such as Practo, OpenMRS, or Bahmni are very feature-rich but suffer from the drawback that these are either very expensive or are very complicated to implement at the hospital level, along with the fact that they are not very customizable according to the working requirements at the health organizations at the hospital level.

2.6. Review Objectives.

Whereas comparing the given alternatives to the given Online Hospital Management System for HealthCare Plus, it is clear that all of the given alternatives address only a few aspects of the hospital's requirement. Scheduling appointments, reminders and simple reporting provided by SimplyBook.me and CS Channeling are significantly helpful in offsetting the existing paper-based workload at HealthCare Plus but do not have integrated EMR fully and have limited features of hospital-wide reports. Practo has a more comprehensive solution with appointment scheduling, billing and teleconsultation, but more applicable to hospital workflows, but its local market focus and license fee might hold it back from adoption. Alternatively, OpenMRS and Bahmni are robust open-source systems uniquely designed for medical records and hospital information systems, with Bahmni more comprehensive for hospitals with its integrated billing and lab modules; but both are high on technical expertise and customizations needed. Whereas HealthCare Plus is exclusively tailored to address the hospital's very specific operational needs-mashing appointment scheduling, consultant fee management, billing, secure data processing and reporting into one integrated, light-weight native workflow-optimized single platform. This guarantees it extends the advantages of computerization and digitalization without the undue complication, excessive expense or customization hassle inherent in current alternatives.

2.7. Methods and Tools.

2.7.1. Methodology.

Development of the HealthCare Plus Online Hospital Management system will be following the Agile Methodology, which focuses on iterative development, ongoing feedback and close collaboration with the stakeholders. Rather than providing the system as an end product in one shot, the project will be split into smaller, manageable chunks known as sprints. Each sprint will concentrate on deploying one feature like patient registration, appointment scheduling, doctor and staff management, billing or reporting. Stakeholders such as admin, patients, staff and doctors will have a review of the functional prototype at the end of each sprint in order to collect feedback. This enables the system to expand according to real-world requirements and adapt to evolving needs.

The Agile implementation benefits HealthCare Plus Hospital in a variety of ways. To begin with, it provides enhanced flexibility and responsiveness, with the hospital able to deliver highest-priority features (e.g., appointment scheduling and billing) in the initial sprints and tweak or introduce new features (e.g., AI Symptom Checker) in the subsequent iterations. This decreases the chances of failure, as stakeholders are constantly kept engaged and enhancements are incorporated along the way. Second, Agile facilitates quicker delivery of value to the hospital. Rather than holding up the whole system for completion, functional modules are delivered in advance so that HealthCare Plus can begin experiencing the benefits of automating manual tasks, increasing patient satisfaction and minimizing administrative mistakes from day one. In general, Agile provides a disciplined but adaptive approach that synchronizes with the changing operational needs of the hospital with assurance of user-centric, high-quality results.

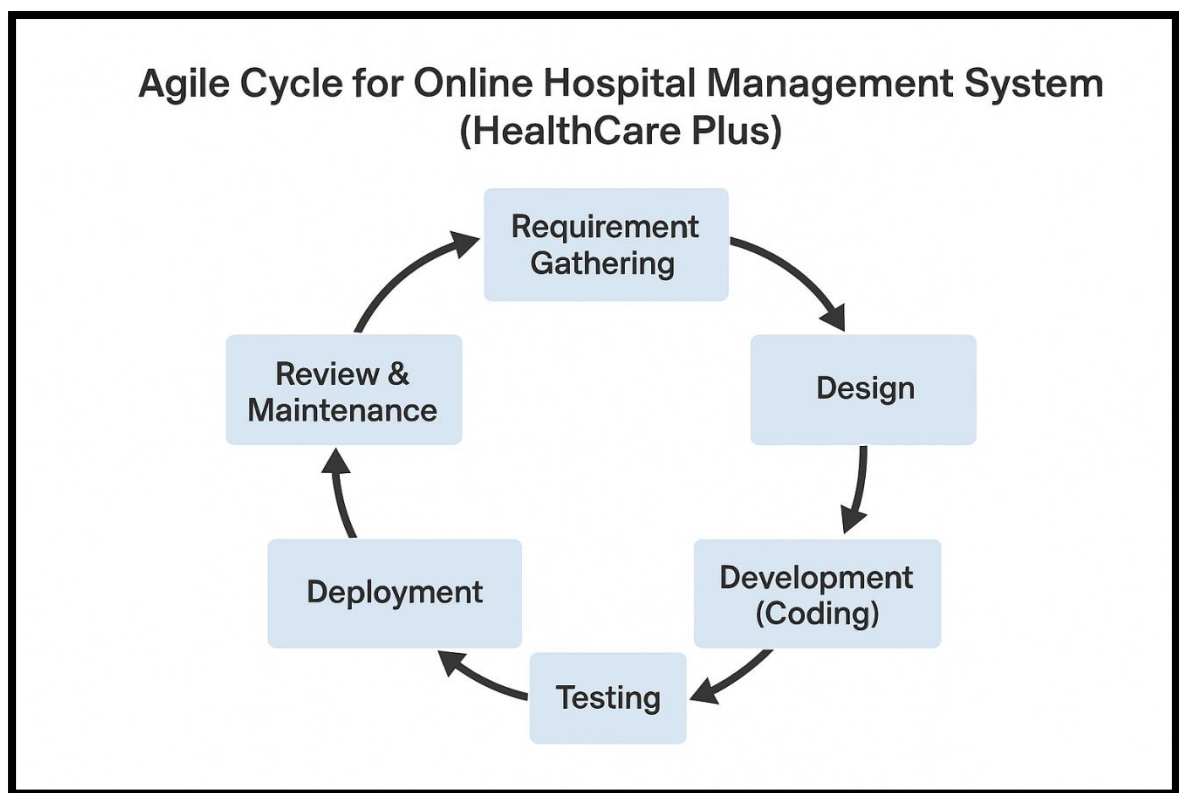


Figure 8: Agile Methodology.

1. Requirement Gathering:

Meet with stakeholders (doctors, admin, patients, staff (nurses)).

Identify and refine user stories such as:

- “As a patient, want to book appointments online.”
- “As a doctor, want to view patient medical history.”

Define sprint scope (what features will be built this iteration).

Estimate time, resources and budget for selected user stories.

2. Design.

For the user stories selected in the sprint:

Create high-level and detailed system architecture diagrams.

Design workflows for modules (examples: Patient Registration, Appointment Scheduling).

Define database structures (tables for patients, doctors, appointments, billing).

Plan User Interface designs for screens such as, Login, Dashboard, Appointment portal, Report Viewer.

3. Development.

During the sprint execution:

Developers write code for the features defined in this sprint.

Implement backend modules (APIs for appointments, billing, pharmacy, reports).

Implement User Interface components (forms, dashboards, notifications).

Unit test every component to verify core functionality.

4. Testing.

Integration Testing: Check interaction between modules (example: appointment → billing → report).

System Testing: Validate the full workflow within the sprint.

User Acceptance Testing: Hospital staff test new features and confirm requirements are met.

Performance Testing: Ensure modules run smoothly under load (example: many simultaneous appointment bookings).

5. Deployment.

After testing:

Deploy the newly developed features to a staging or production environment.

Release sprint increments such as:

Sprint 1: Patient Registration module.

Sprint 2: Appointment Scheduling.

Sprint 3: Doctor Dashboard.

Ensure features work in a real-world hospital environment.

Provide training/guide updates to end-users.

6. Review and Maintenance.

At the end of each sprint:

Review sprint deliverables with stakeholders.

Collect feedback for improvements.

Identify bugs, performance gaps or user experience issues.

Add enhancement requests and fixes to the Product Backlog.

Release patches, updates or improvements as needed.

2.7.2. Tools.

Software Components.

Operating System	Windows 10
Web Server	XAMPP – SQL Server 2021 – v3.3.0
Web Browser	Chrome and Microsoft Edge
Code Editor	Microsoft Visual Studio Code, Notepad++

Table 1: Software Components.

Hardware Components.

Component	Recommended
Device	Laptop
Processor	11 th Gen Intel® Core™ i7-1165G7@2.80GHz 2.80GHz
Application Window Size	1920 × 1080
Screen Resolution	1920 × 1080
Hard Disk	512 GB
RAM	8 GB
System	64-bit operating system, ×64-based processor

Table 2: Hardware Components.

This system needs a computer or a laptop. If the users use a computer, the users should use a properly working keyboard and a mouse as input devices. A minimum requirement for the processor should be 11th Gen Intel(R) Core (TM) 2Duo 3.00GHz. The maximum 11th Gen Intel(R) Core (TM) i7-1165G7 @ 2.80GHz generation processor can be used.

Further, there should be a minimum 1GB space from the hard disk to run this system. The primary memory (RAM) should have a minimum of 4GB. There can be maximum 16GB RAM used to run without interruption.

There should be a minimum 1024×680 display size of monitor to view this system. There can be used monitors with above this display size like 1920×1080 . Moreover, the screen resolution should be at least 1024×768 . There can be an upper screen resolution monitor to use this system.

2.7.3. Technologies.

Frontend Technologies.

HTML (HyperText Markup Language): Utilized as the system's web page structure framework, such as patient registration, appointment scheduling, dashboards and reports.

CSS (Cascading Style Sheets): Provides style and design to professionalize the user interface (UI) and ensure it is user-friendly, as well as accessible on devices. CSS formats the fonts, colors, layout and responsiveness for patients, doctors, staff and admin users.

JavaScript: Provides interactivity to the system, e.g., form validation, dynamic updates (e.g., availability of an appointment) and making web pages responsive without reloading the entire page.

Backend Technologies.

PHP (Hypertext Preprocessor): It is the server-side scripting language used to handle system logic, interact with the database, handle sessions and handle user requests. It handles tasks such as patient authentication, booking appointments, staff/doctor handling and billing operations.

API Integrations: PHP will also include third-party APIs such as email notification APIs and the symptom checker based on AI to further increase the functionality of the hospital system.

Database.

MySQL: Used as the relational database management system (RDBMS) to manage and store all hospital data such as patient data, staff information, doctor profiles, schedules, billing records and system logs. MySQL provides data consistency, relational queries and secure storage with role-based access controls.

Tools and Supporting Technologies.

Email Notification Service (via API): To send reminders for appointments, confirmations and medication reminders to patients' mobile phones in real-time.

AI Integration (Symptom Checker): Functions from an AI module that is supplied with patient-input symptoms and provides preliminary guidance before scheduling.

XAMPP Server.

XAMPP is an open-source package of web solutions that contains the Apache server, MariaDB, PHP, Perl and command-line executables along with distributions for many more servers. Before publishing a website to the main server, XAMPP enables a local host or server to test its website and clients via desktop and laptop PCs. It is a platform that provides an appropriate setting for testing and confirming the operation of projects based on Apache, Perl, MySQL and PHP through the host's system. Perl is a computer language used for web development, PHP is a backend scripting language and MariaDB is the most popular database created by MySQL among these technologies. [7]

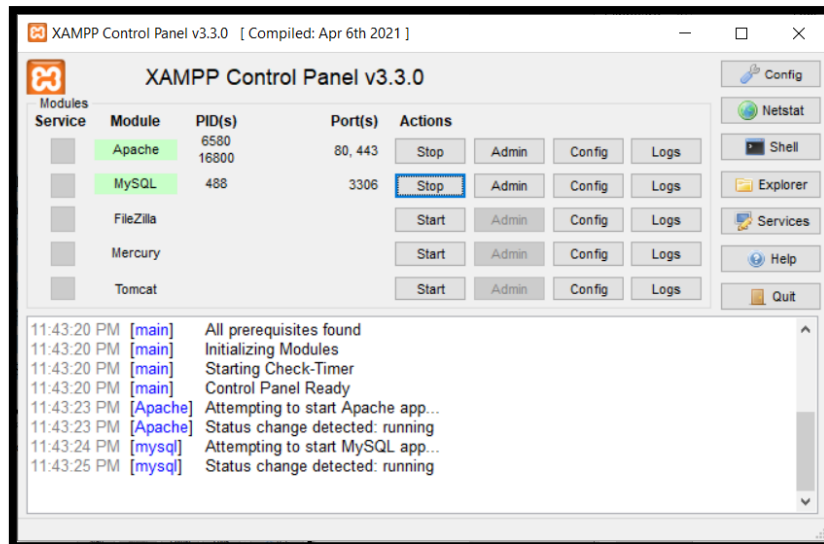


Figure 9: XAMPP Server.

Notepad++.

A free text editor for Windows called Notepad++ provides more functionality nor present in the default Notepad text editor for Windows. Notepad++ specially made for editing source code. In programming languages like C, C++, Java and JavaScript, the increment operator is represented by the prefix “++” in the name. [8]

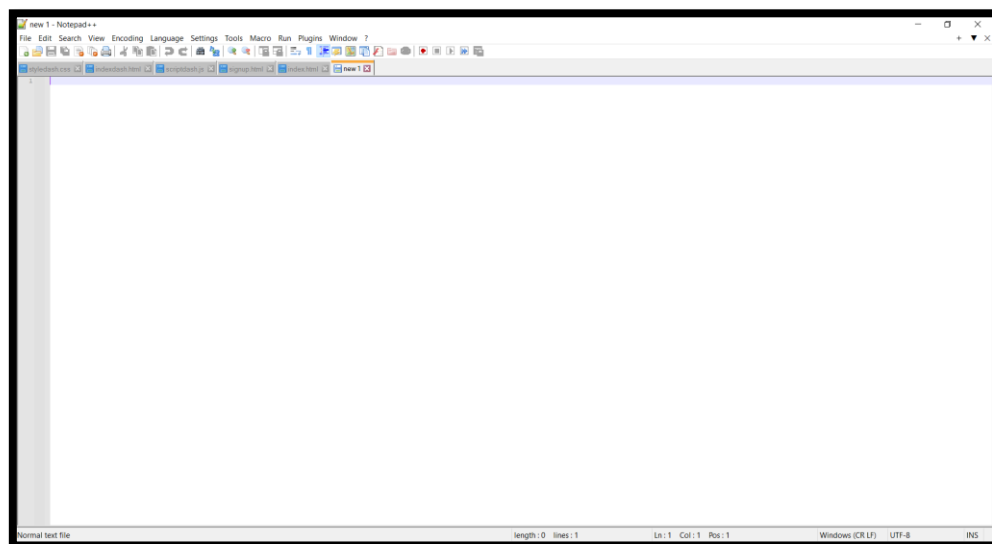


Figure 10: Notepad++.

Visual Studio Code.

Visual Studio Code is a free, open source code editor developed by Microsoft. It is available on Windows, macOS and Linux. VS Code is known for its lightweight design, speed and flexibility, making it one of the most popular code editors among developers around the world. [9]

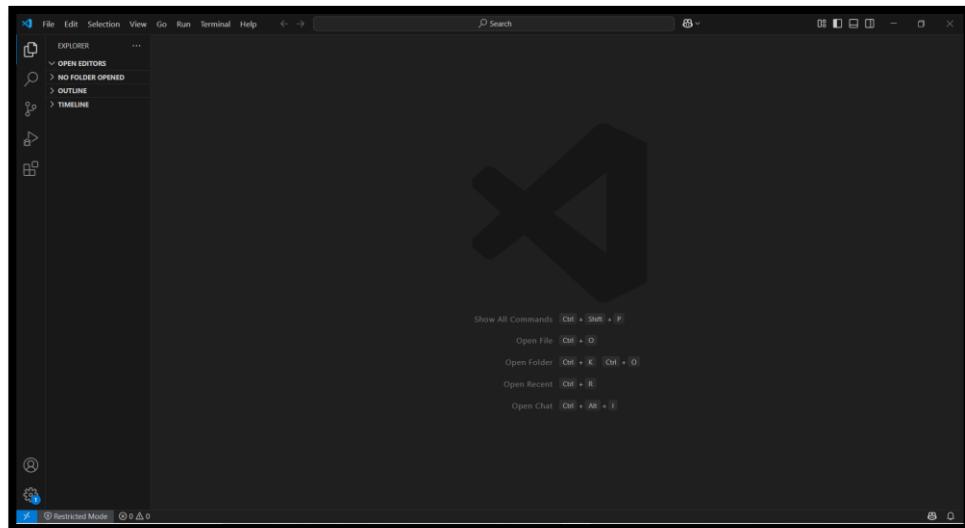


Figure 11: Visual Studio Code.

2.8. Summary.

This chapter has furnished a background for why this system needs to be developed for HealthCare Plus Hospital. This includes a broader background of why digitization in healthcare has become a requirement and a detailed description of problems associated with healthcare that a particular healthcare institution may be facing. This chapter has identified other existing systems for healthcare management in similar healthcare setups. This chapter has also pointed out why those existing systems are inadequate for this particular healthcare institution. This chapter describes who are the major players involved while implementing such a healthcare system. This chapter has furnished explanations about theories involved in managing healthcare systems. This chapter has furnished explanations about methods involved in managing healthcare. This chapter has identified those research questions that need to be answered.

3. Chapter III – Specification and Design.

3.1. Introduction.

This chapter shows the complete specification and design of the Online Hospital Management System for the HealthCare Plus Hospital. How the system architecture, tools and structural elements are to be selected will be discussed in order to ensure full satisfaction of the project objectives through the adopted solution design. The chapter will illustrate the viewpoints applied in developing the system specification and design, such as user interface view, data flow view, dynamic view, algorithmic view, data structure view and static architecture view, to give a comprehensive insight into what the system intends to accomplish and how the design be implemented.

3.2. User Interface.

The User Interface in the system has been designed to make the system simple and accessible to the users. The user interface design is responsive and has a flat design concept to make the application easily accessible by those with little technical know-how of the system.

Login Page.

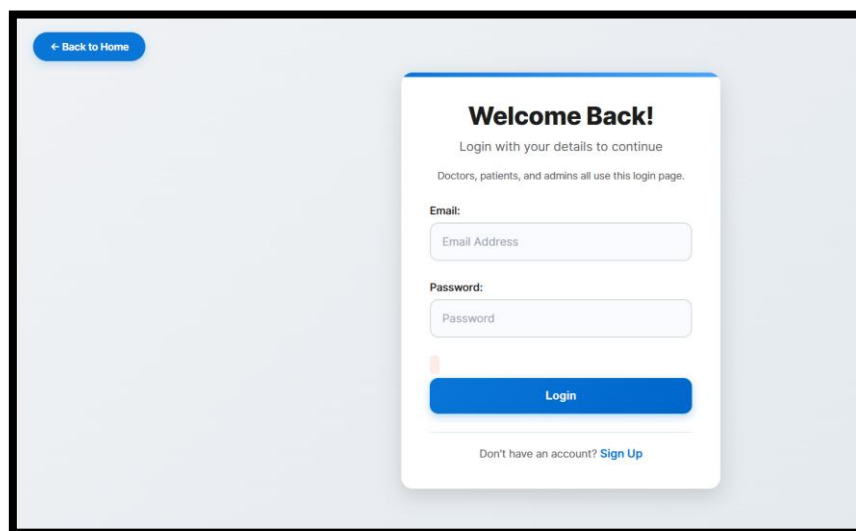
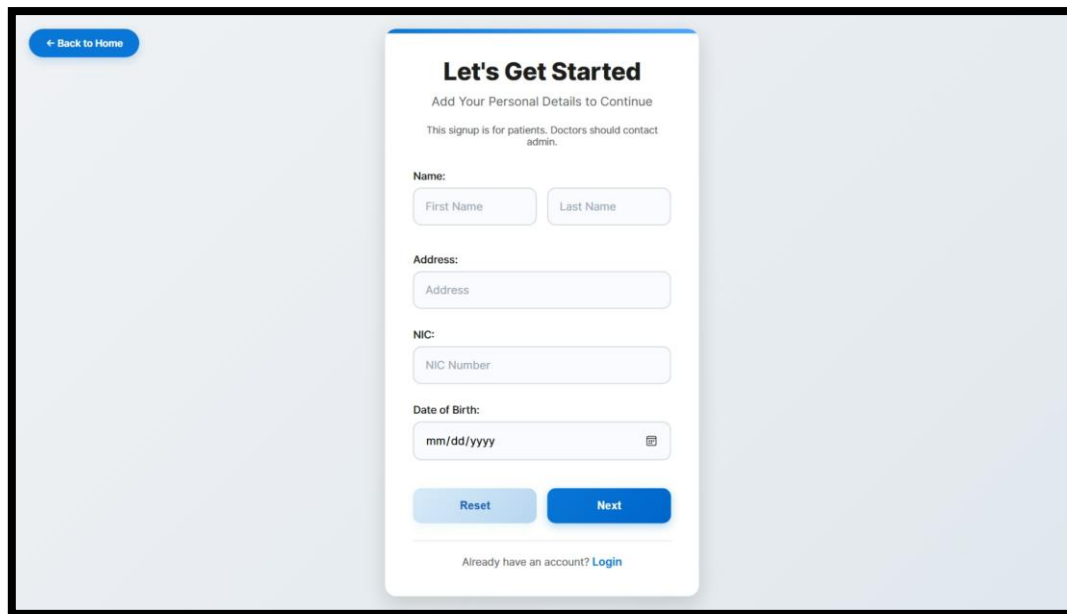


Figure 12: Login Page.

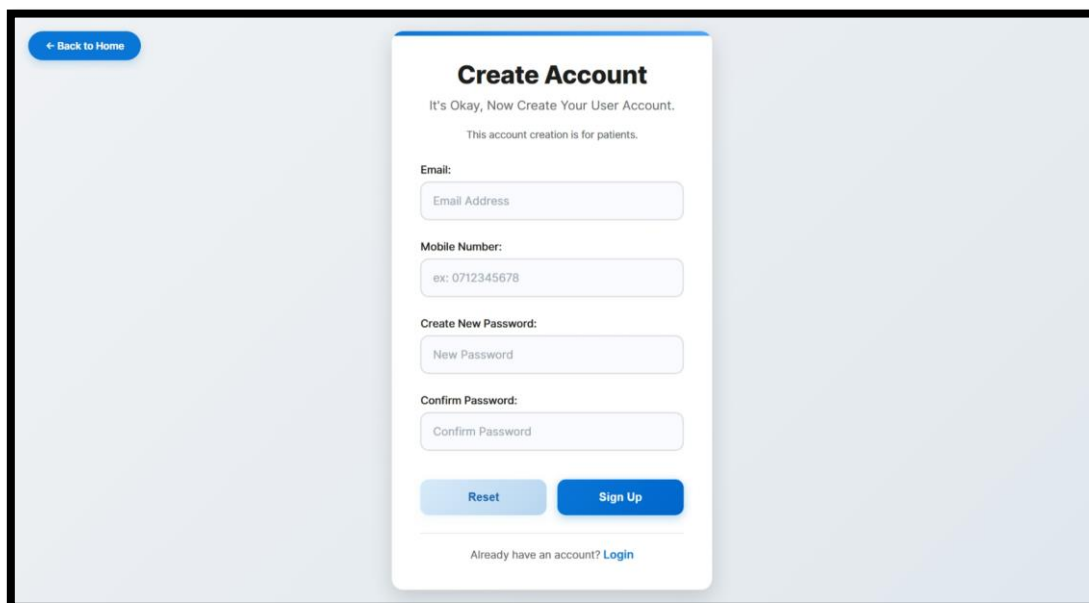
This is the Login page of the system. There are four categories in this system to login. They are Admin, Patient, Doctor and Staff. These levels should enter username and password to login to the system. They would be wrong and if the any fields would be empty or incorrect when click the Login button will throw errors. After, the users enter the correct username and password the system shows “Login Successful Message”.

Registration Page.



The registration page features a central white card on a light blue background. At the top left of the card is a blue button labeled "← Back to Home". The card's title is "Let's Get Started" in bold, followed by the subtitle "Add Your Personal Details to Continue". A note states: "This signup is for patients. Doctors should contact admin." The form includes fields for "Name:" (split into "First Name" and "Last Name"), "Address:", "NIC:" (with a sub-label "NIC Number"), and "Date of Birth:" (with a sub-label "mm/dd/yyyy" and a calendar icon). At the bottom of the form are "Reset" and "Next" buttons. A link "Already have an account? Login" is at the very bottom.

Figure 13: Registration Page.



The sign-up page features a central white card on a light blue background. At the top left of the card is a blue button labeled "← Back to Home". The card's title is "Create Account" in bold, followed by the subtitle "It's Okay, Now Create Your User Account." A note states: "This account creation is for patients." The form includes fields for "Email:" (with a sub-label "Email Address"), "Mobile Number:" (with a sub-label "ex: 0712345678"), "Create New Password:", and "Confirm Password:". At the bottom of the form are "Reset" and "Sign Up" buttons. A link "Already have an account? Login" is at the very bottom.

Figure 14: Sign Up Page.

Here is the registration page of the system for new users. Users should enter their name, address, Identity number, date of birth, email, contact number and password. Users should confirm their password again.

Home Page.

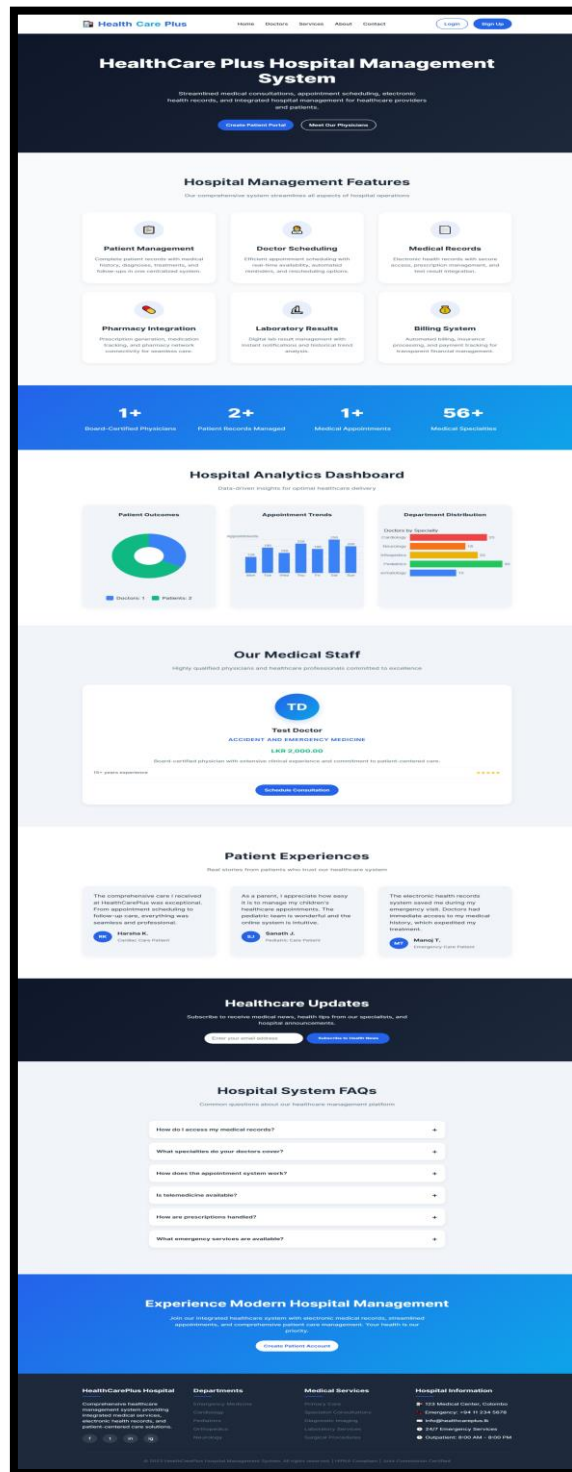


Figure 15: Home Page.

This is the home page of the system. there is a navigation bar including Home, Doctors, Services, About and Contact tabs. Users can see the features of the system, hospital analytics dashboard, staff of the hospital, comments of patients in this home page. Users can see the guidance when using the system.

Admin Dashboard.

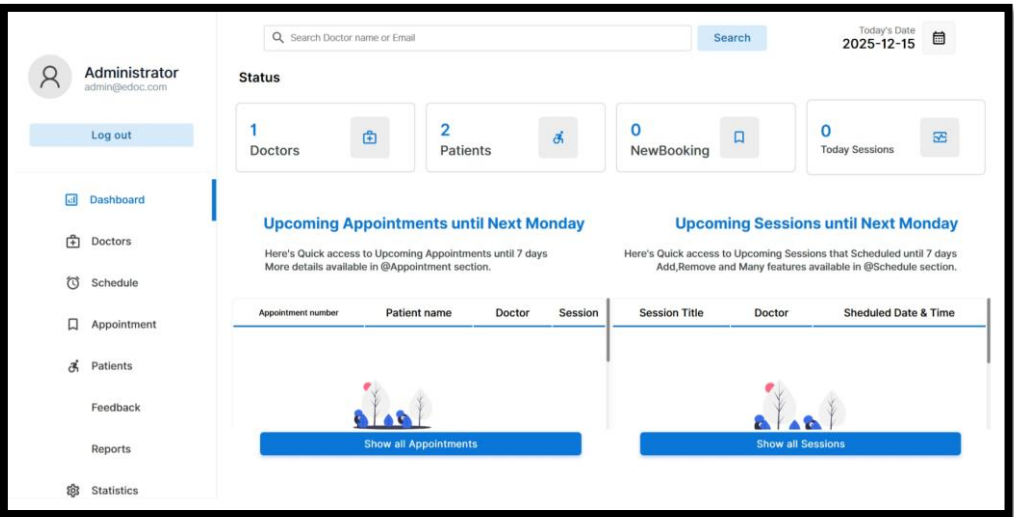


Figure 16: Admin Dashboard.

If Admin login to the system successfully, Admin can see the Admin Dashboard. Admin can see the Total active doctors, Total appointments and Today sessions. Admin can see the registered patients also.

Doctor Management.

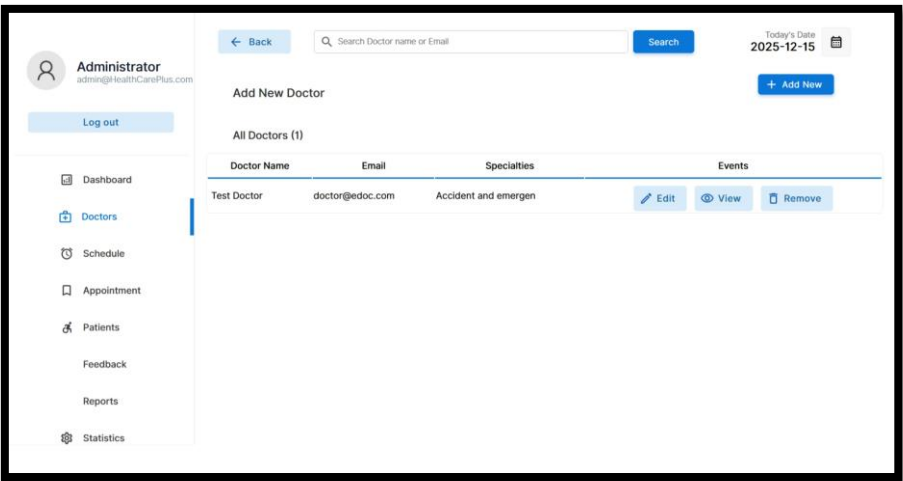


Figure 17: Doctor Management.

In this page, Admin can add, update and delete doctors. Admin should fill doctor details to add, update or delete a doctor. After adding, updating or deleting the details, the results will be shown in the bottom view of the page. Then the admin can search for a doctor in the search bar.

Appointment Management.

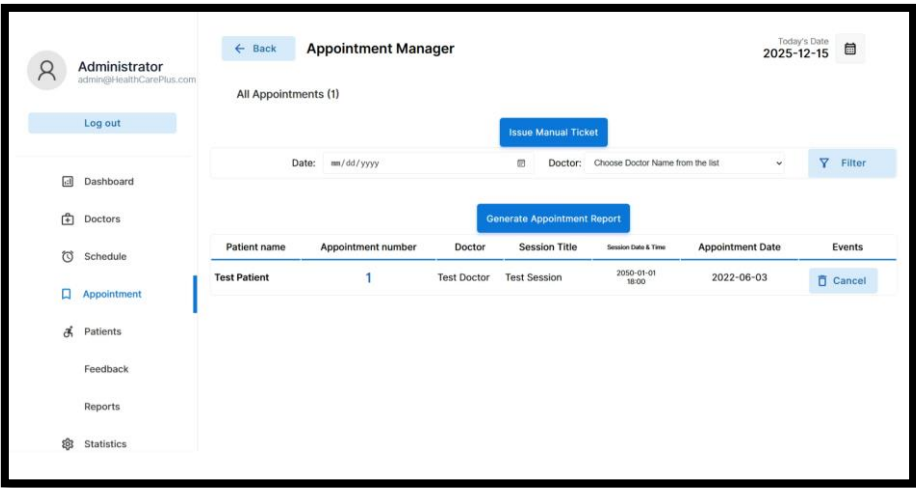


Figure 18: Appointment Management Page.

Here is the admin appointment management form. Admin can confirm the appointments, cancel the appointment and update appointment status. Admin can see the appointment list of patients.

Schedule Management.

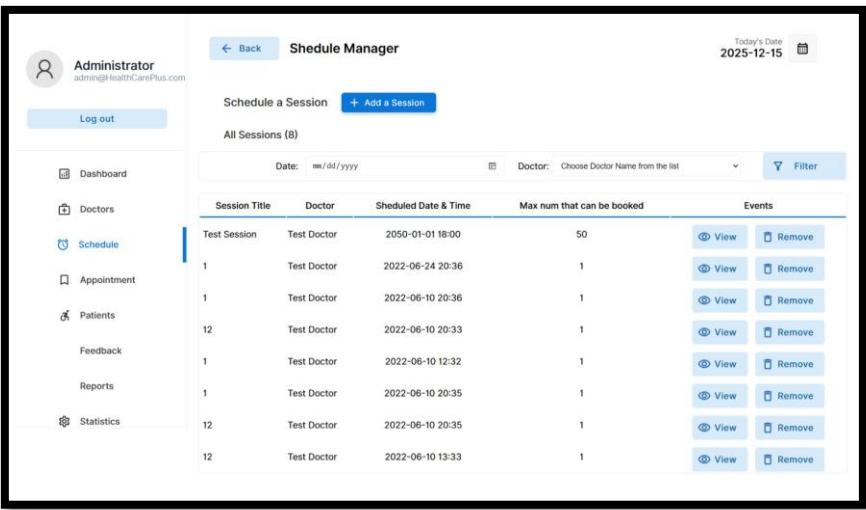


Figure 19: Schedule Management.

Admin can schedule new sessions. Admin can view the schedules, remove schedules and view all details of sessions.

Patient Management.

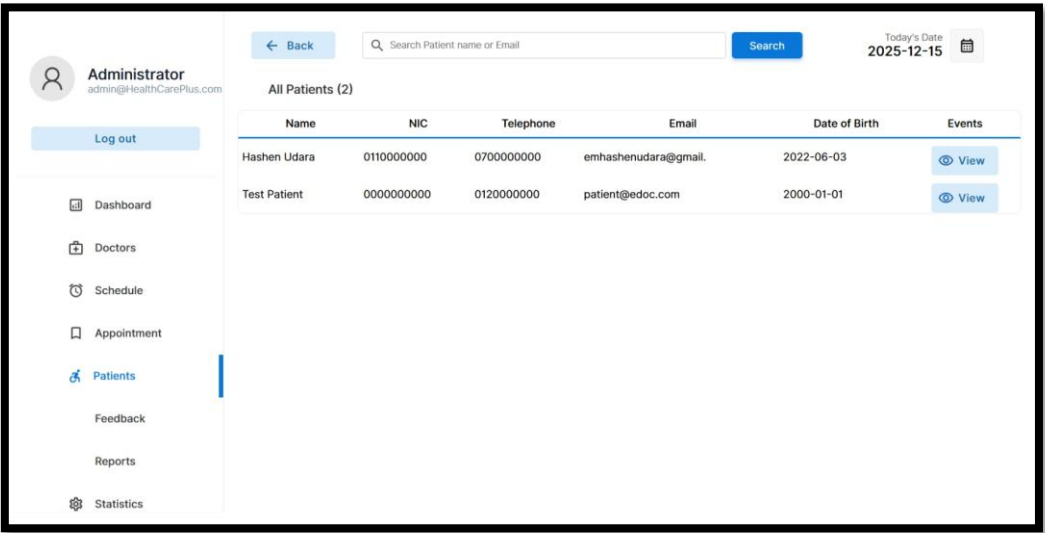


Figure 20: Patient Management Page.

In this page, Admin can view patients that registered to the system. The system shows all details of the patient including name, identity number, contact number, email and date of birth. Admin can view all details of patients.

Feedback Management.

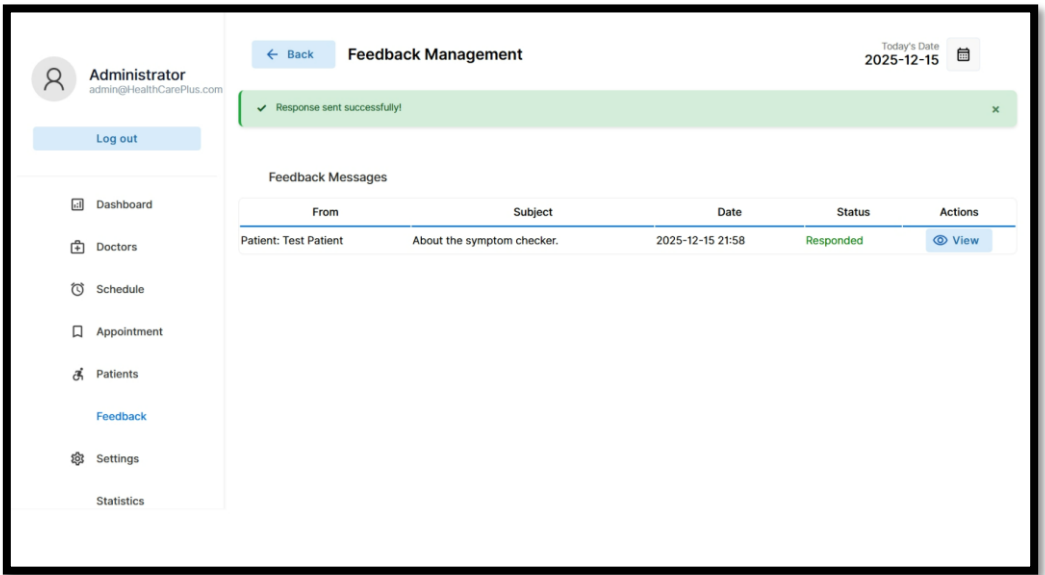


Figure 21: Feedback Management.

Admin can view feedbacks that patients gave to the system. The system shows the feedback including patient name, subject, date and status.

Report Generation.

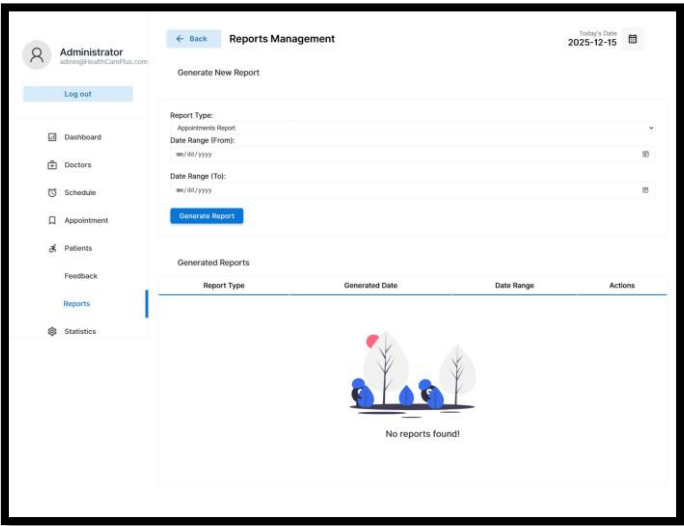


Figure 22: Report Generation.

Admin can generate reports. Admin can generate appointment reports, payment reports, doctor reports, patient reports and patient documents.

System Statistics.

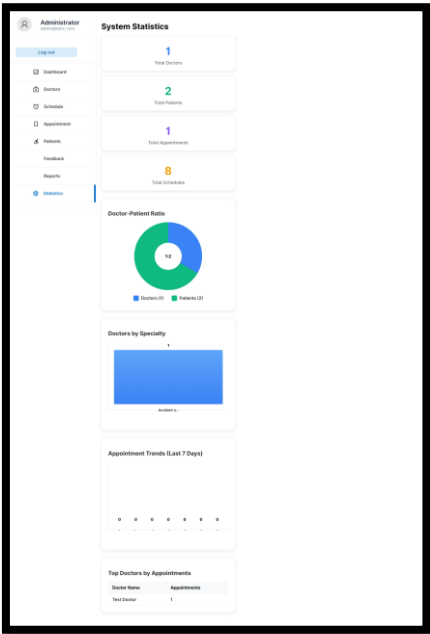


Figure 23: System Statistics.

Here is the system statistics page. The system shows the current total of doctors, patients, appointments and schedules. Admin can see the doctor-patient ratio, doctors by specialty, appointment trends and top doctors by appointments.

Patient Dashboard.

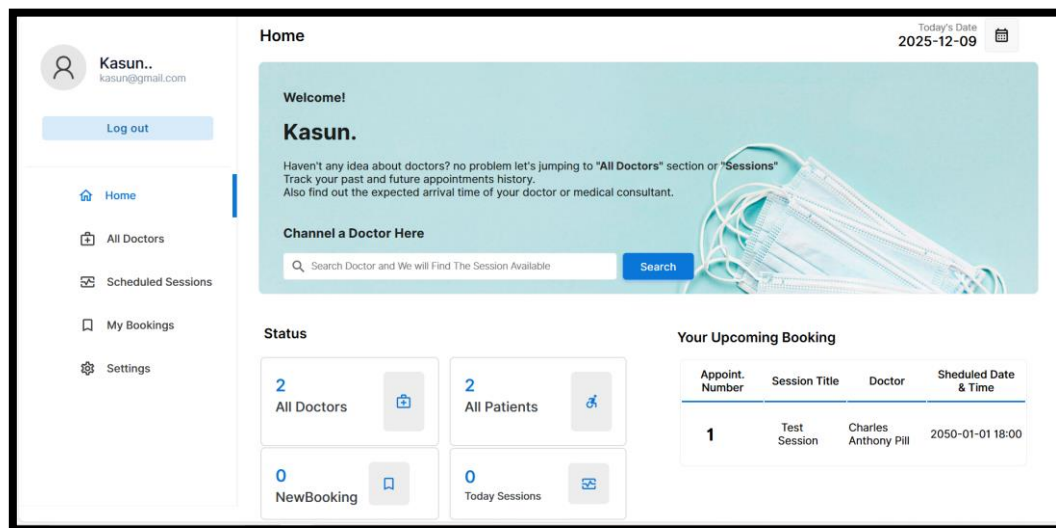


Figure 24: Patient Dashboard.

After the login, patient can see their dashboard including, count of all doctors, patients, bookings and the sessions. Patient can make an appointment. The system shows the doctor availability list to the patient. Patient can see their booking history. Patient can update their profile details using settings tab.

Book Appointment.

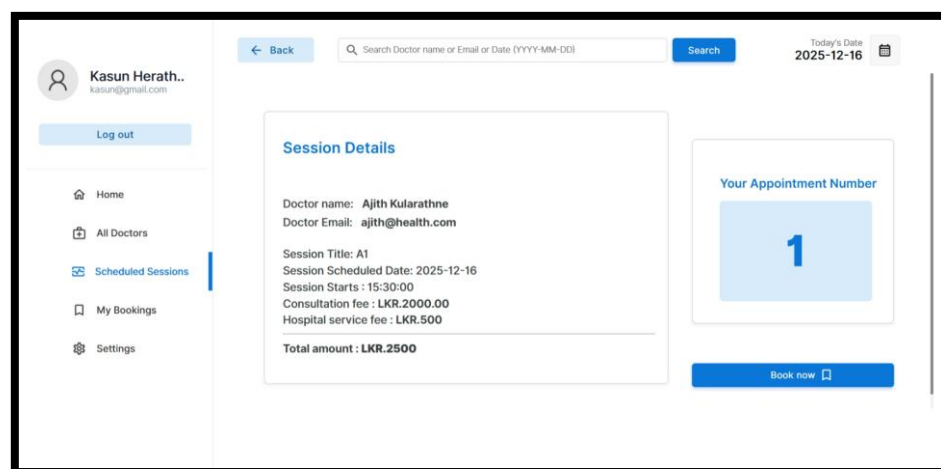


Figure 25: Book Appointment.

When patient booking an appointment, the system shows the session details including, doctor name, doctor email, session title, scheduled date, consultation fee, hospital fee and the appointment number.

Payment.

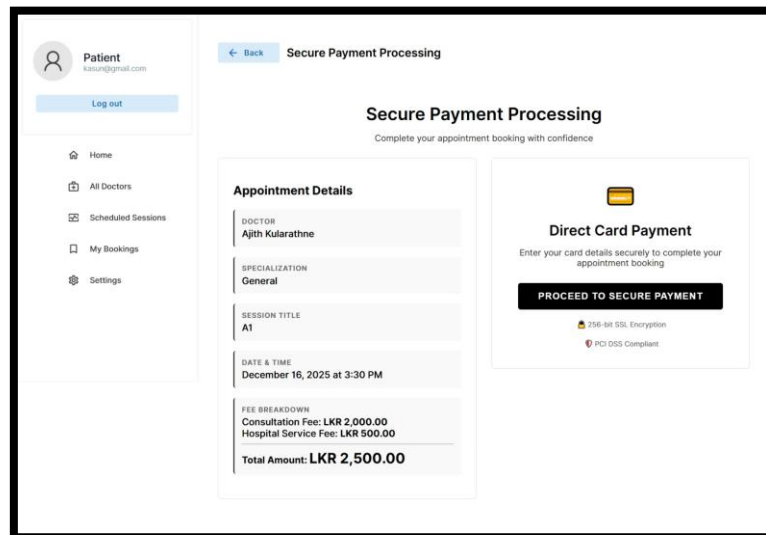


Figure 26: Payment.

Patients can make payments using the payment page. The system shows the full amount that the patient should pay. Patient can see the appointment details in their payment process. Patient can make their payments using online transaction.

Booking History.

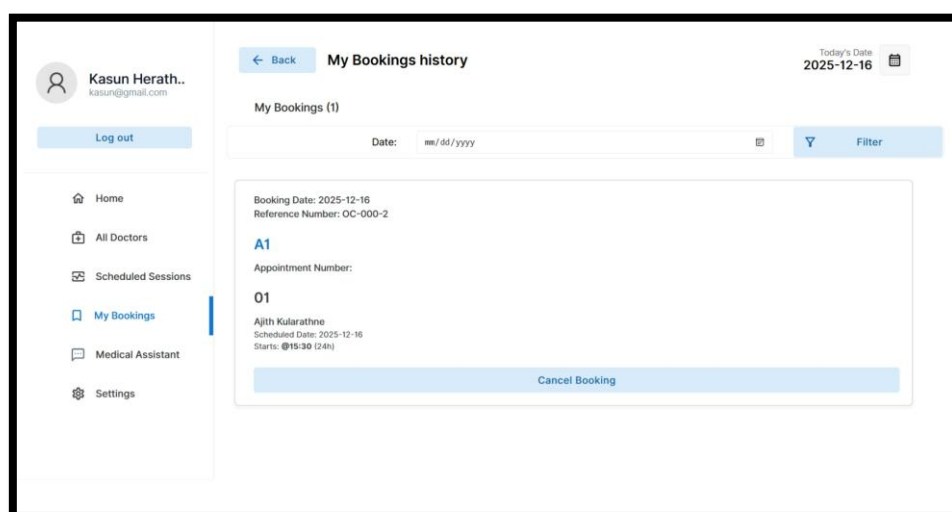


Figure 27: Booking History.

Patient can view their booking history. Patient can view the booking summary including booking date, reference number, appointment number, doctor name, scheduled date and status. Patient can cancel their appointments.

AI-Powered Symptom Checker.

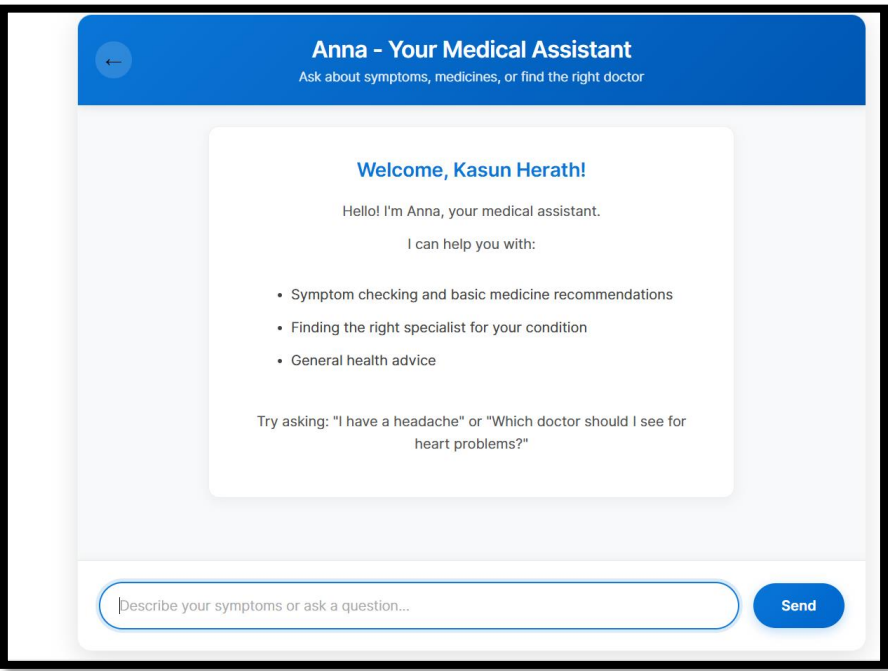


Figure 28: AI-Powered Symptom Checker.

Patient can search the doctors and take the advice about their symptoms using the AI-Powered Symptom Checker.

Doctor Dashboard.

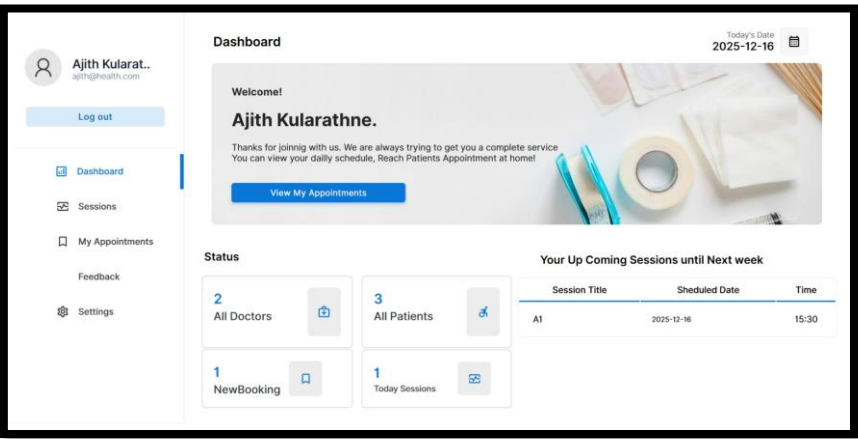


Figure 29: Doctor Dashboard.

Here is the doctor dashboard in the system. doctor can see the total amount of all doctors, all patients, bookings and the sessions. Doctor can view their sessions, appointments and feedbacks. The system shows the upcoming session to the doctors.

Doctor Schedules.

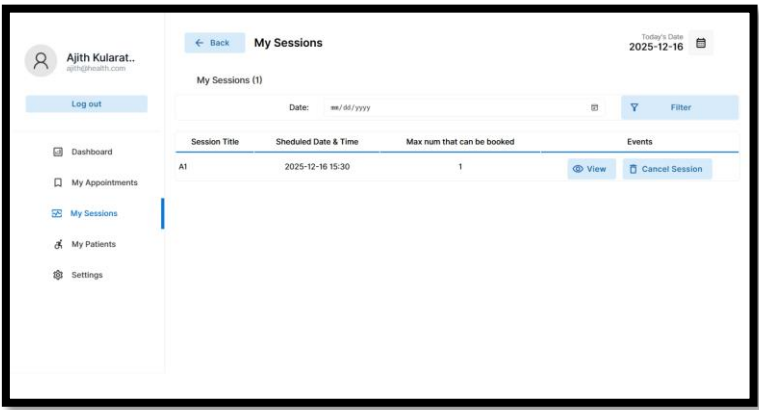


Figure 30: Doctor Schedules.

Doctor can see their sessions including session title, scheduled date and time, maxed number that can be booked and events.

Patient List.

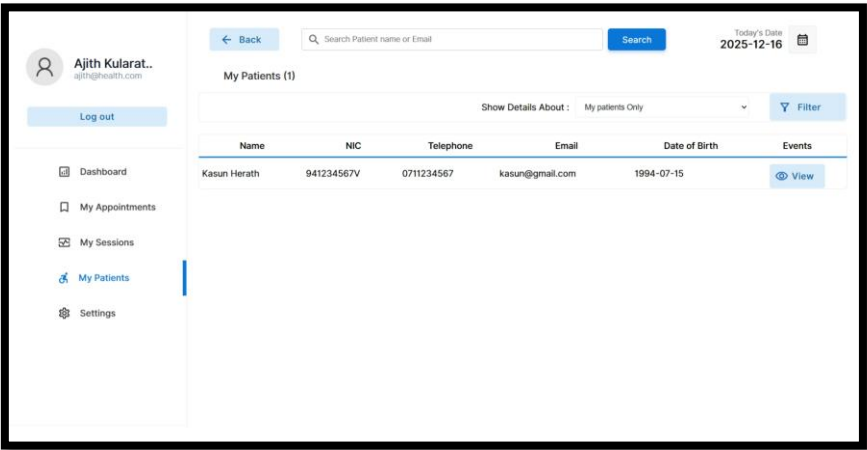


Figure 31: Patient List.

Doctor can see the patients that booked an appointment. The system shows the patient details including name, identity number, contact number, email and date of birth of the patient.

3.3. Dynamic Behavior of the System.

This dynamic behavior describes how the system reacts to user input and real-time events. The system follows a modular structure, where each module interacts through secure database transactions and logical workflows.

1. User Registration Flow.

- User enters personal details.
- System validates required fields.
- Account is created and stored in the database.
- Confirmation message is displayed.

2. Login Authentication.

- User enters username and password.
- System validates credentials using hashing.
- Role-based dashboard loads.
- Invalid attempts generate alerts.

3. Appointment Scheduling Workflow.

- Patient searches doctor availability.
- System checks doctor's schedule in the database.
- Patient selects time and date.
- Appointment is stored with "Pending" or "Confirmed" status.
- Payment processing updates final confirmation.
- Email notification is sent automatically.

4. Payment Processing Behavior.

- Patient or admin enters payment details.
- System calculates doctor fees, hospital commission and taxes.
- Receipt is generated.
- Payment history updates in patient profile.

5. Feedback System Behavior.

- Patient submits feedback.
- Admin receives notification.
- Admin replies.
- Patient views the response on dashboard.

6. AI Symptom Checker Behavior.

- User enters symptoms.
- System sends data to AI model.
- AI returns possible conditions.
- User is advised to book an appointment.

7. Email Notification Behavior.

- Appointment confirmation.
- Cancellations.
- Medication Reminders.
- Daily schedules.

3.4. System Architecture.

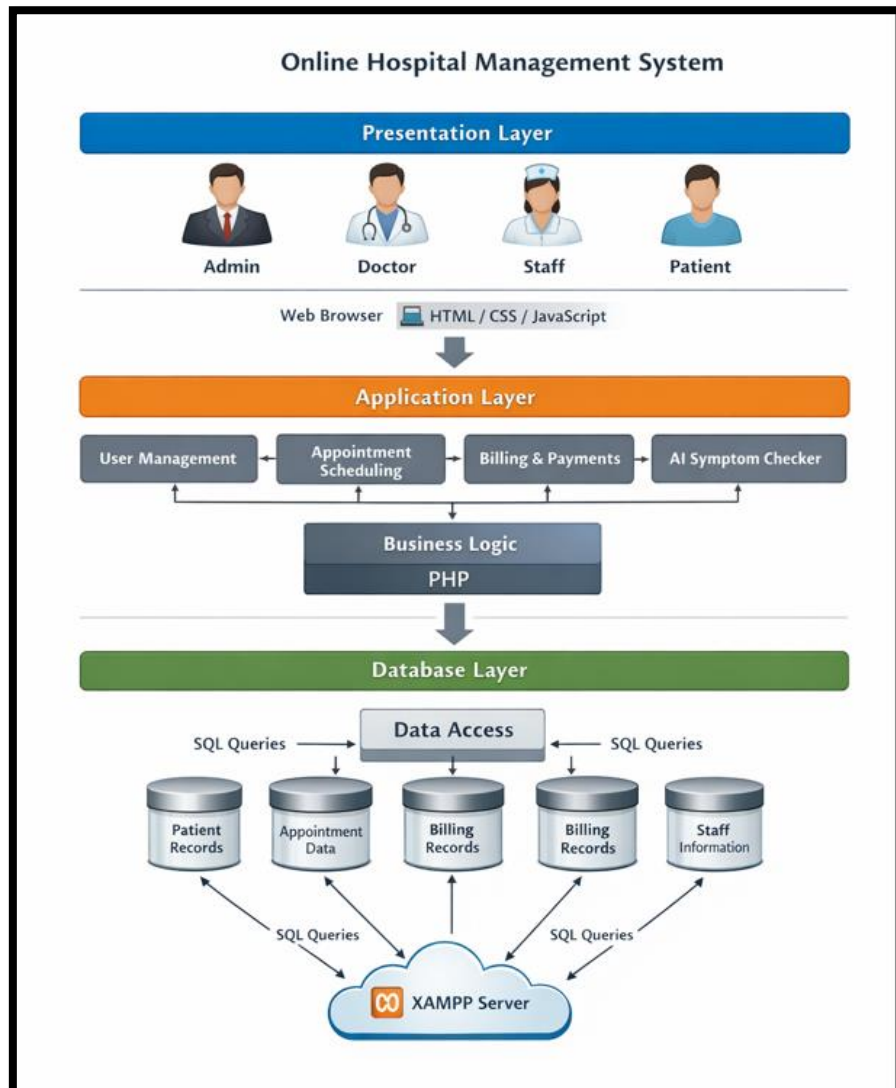


Figure 32: System Architecture.

This architecture diagram of the Online Hospital Management System for HealthCare Plus hospital represents a three-tier layered architecture, which separates the system into Presentation Layer, Application Layer and Database Layer. This representation improves maintainability, scalability, security and performance of the system.

3.5. Diagrams.

Use Case Diagram.

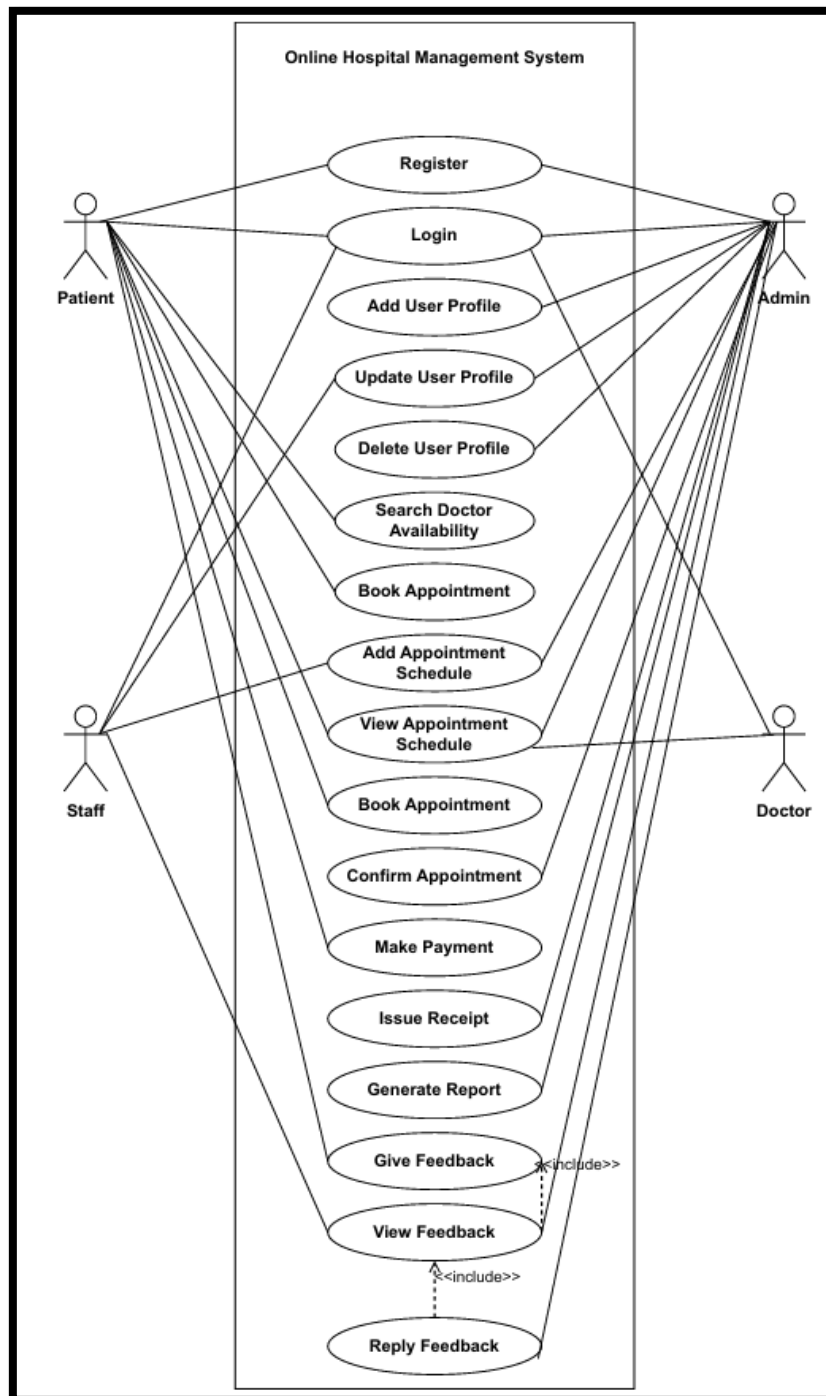


Figure 33: Use Case Diagram.

The developer created the below use case diagram to show the actors and how they impact the hospital system. There are four actors in this diagram. They are Patient, Staff, Admin and Doctor. Patients can register and login to the system, manage their own profiles, search for doctor availability, book appointments, make payments, provide feedback and can view feedback replies from the admin. Admin has the highest authority in the system. Admin can manage all user profiles, manage appointment schedules and confirm appointments, issue receipts, generate reports and reply feedback. So, the Patients are the main users of booking, payment and feedback services. Admin is the most powerful actor, capable of managing all functions. Doctors mainly interact with appointment schedules and medical report generation.

Enhanced Entity Relationship Diagram (EERD).

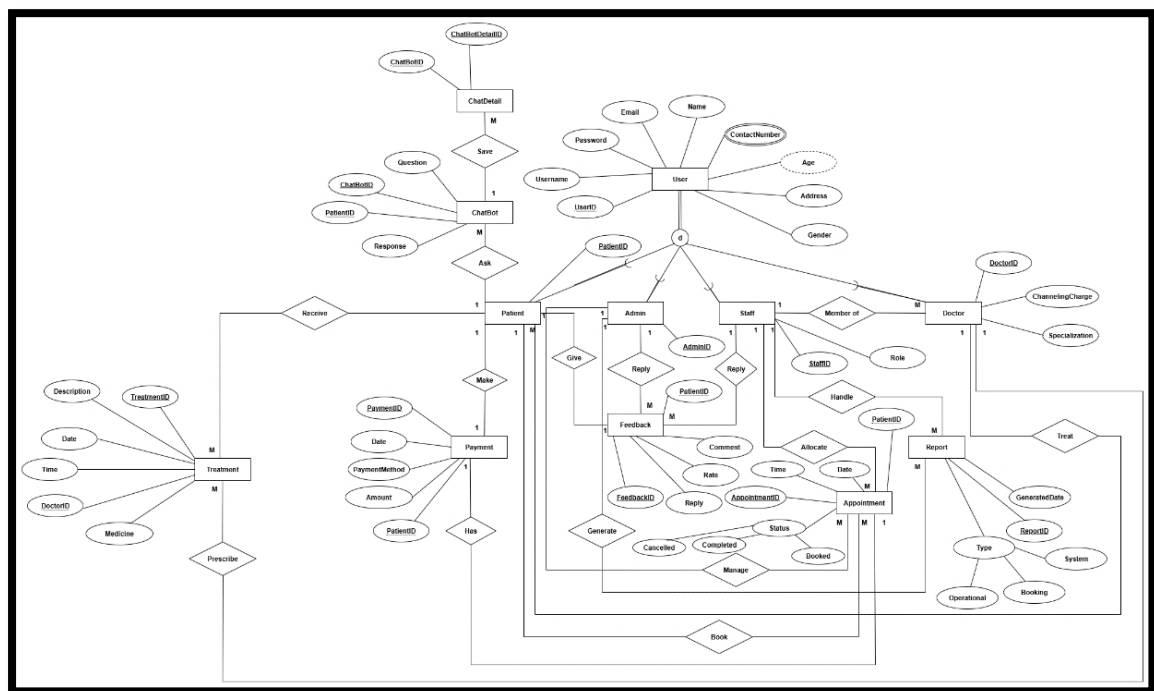


Figure 34: Enhanced Entity Relationship Diagram.

Relational Schema for Online Hospital Management System based on the Enhanced Entity Relationship Diagram.

Patient (PatientID, Name, ContactNumber, Address, Age, AppointmentID*, Report ID*)

Appointment (AppointmentID, Date, Time, Status)

Doctor (DoctorID, Name, Category, ChannelingCharge, Specialization, Username, Password, Email, ContactNumber, Gender)

Admin (AdminID, ContactNumber, Username, Password, Address, Email, Age, Gender, Name)

Payment (Payment ID, Amount, Date, PaymentMethod, PatientID*)

Staff (Staff ID, Name, Address, ContactNumber, Username, Password, Email, Age)

Report (ReportID, GeneratedDate, Type)

Treatment (TreatmentID, Medicine, Date, Description, Time, DoctorID*)

ChatBot (ChatBotID, Question, Response, PatientID*)

ChatDetail (ChatDetailID, ChatBotID*)

Feedback (FeedbackID, Comment, Rate, Reply, PatientID*)

Entities, Attributes and Relationships.

Patient.

Primary Key: PatientID

Attributes: Name, Age, Address, ContactNumber

Foreign Keys: ReportID, AppointmentID, PaymentID

Relationships:

One Patient → Many Appointments (One-to-Many)

One Patient → Many Reports (One-to-Many)

One Patient → One Account (One-to-One)

Doctor.

Primary Key: DoctorID

Attributes: Name, Address, Category, ChannelingCharge, Rate

Foreign Keys: AppointmentID

One Doctor → Many Appointments (One-to-Many)

One Doctor → Many Reports (One-to-Many)

Appointment.

Primary Key: AppointmentID

Attributes: Date

Foreign Keys: PatientID, DoctorID, AdminID

Relationships:

One Appointment → Belongs to One Patient (Many-to-One)

One Appointment → Belongs to One Doctor (Many-to-One)

One Appointment → One Reports (One-to-Many)

One Appointment → One Payment (One-to-One)

Report.

Primary Key: ReportID

Attributes: Medicine, OperationDate, DateAdmitted, DateDischarged

Foreign Keys: PatientID, DoctorID

Relationships:

One Report → Linked to One Patient (Many-to-One)

One Report → Linked to One Doctor (Many-to-One)

One Report → Linked to One Appointment (One-to-One)

Payment.

Primary Key: PaymentID

Attributes: Date, PaymentMethod (Cash/Online), Amount

Foreign Keys: PatientID

Relationships:

One Payment → Linked to One Patient (One-to-One)

Admin.

Primary Key: AdminID

Attributes: ContactNumber

Foreign Keys: AppointmentID, StaffID

Relationships: Admin can manage Patients, Appointments, Staff

Staff.

Primary Key: StaffID

Attributes: Name, Address, ContactNumber

Foreign Keys: AdminID

Relationships: Staff assist in managing Appointments and Payments.

Treatment.

Primary Key: TreatmentID

Attributes: Description, Medicine, TreatmentDate, TreatmentTime,

Foreign Keys: DoctorID, PatientID

Relationships: Doctor prescribe Treatment, Patient receive Treatment.

Feedback.

Primary Key: FeedbackID

Attributes: Comment, Rate, Reply

Foreign Keys: PatientID

Relationships: Patient give Feedback, Admin reply to Feedback.

ChatBot.

Primary Key: ChatBotID

Attributes: Question, Response

Foreign Keys: PatientID

Relationships: Patient ask ChatBot, ChatBot save ChatDetail.

Assumptions.

- Each user of the system must have a unique account with a unique **UserID, Username, and Email address.**
- A user account can belong to **only one role at a time**, such as Patient, Doctor, Admin, or Staff.
- Each **Patient** is uniquely identified by a **PatientID** and must be associated with exactly one user account.

- A patient must register and create an account before booking appointments or accessing hospital services.
- A patient can book **multiple appointments** with different doctors at different times.
- A patient **cannot book overlapping appointments** with the same doctor at the same date and time.
- Each **Doctor** has a unique **DoctorID** and must be linked to one user account.
- A doctor can handle **multiple appointments**, but each appointment is assigned to only one doctor.
- Doctors are responsible for generating **medical reports** for patients after consultations.
- Each **Admin** has a unique **AdminID** and is linked to one user account.
- Admin users have full system access and are responsible for managing patients, doctors, staff, appointments, and reports.
- Each **Staff** member has a unique **StaffID** and is associated with one user account.
- Staff members act as support users and assist Admins in managing appointments and payment-related activities.
- Each **Appointment** has a unique **AppointmentID** and is booked by exactly one patient.
- Each appointment is managed by an Admin or Staff member and attended by one doctor.
- An appointment has a defined **date, time, and status** (e.g., Booked, Completed, Cancelled).
- Each appointment can generate **only one payment record**.

- Each **Payment** has a unique **PaymentID** and is linked to one appointment and one patient.
- Payments can be made using **cash or online payment methods**.
- A patient can make multiple payments over time, but each payment corresponds to a single appointment.
- Each **Report** has a unique **ReportID** and is generated by one doctor for one patient.
- A patient can have **multiple medical reports**, generated from different appointments.
- Each report is associated with **only one appointment**.
- Reports are generated **after the consultation is completed**.
- Each **Treatment** record is associated with one patient and one doctor.
- A doctor can prescribe multiple treatments for a patient over time.
- Feedback can be provided **only by registered patients**.
- Each patient can submit **multiple feedback entries**, but each feedback belongs to one patient.
- The chatbot is available only to registered patients for basic queries and assistance.
- Each chatbot interaction is stored with a unique **ChatBotID** and linked to one patient.
- Detailed chatbot conversations are recorded through **ChatDetail** entries.
- System enforces **data integrity, access control, and role-based permissions**.
- Deleted records (patients, doctors, appointments) are logically handled to maintain historical data integrity.

Relationships and Cardinalities.

Relationship	Cardinality
User → Patient	1: 1
User → Doctor	1: 1
User → Admin	1: 1
User → Staff	1: 1
Patient → Appointment	1: M
Doctor → Appointment	1: M
Appointment → Payment	1: 1
Patient → Report	1: M
Doctor → Report	1: M
Appointment → Report	1: 1
Patient → Feedback	1: M
Patient → Chatbot	1: M

Table 3: Relationships and Cardinalities.

3.6. Data Structures and Algorithms Implemented.

Function	Algorithm Used	Purpose
Appointment allocation	Time-slot verification algorithm	Prevent double-booking
Authentication	Hash comparison algorithm	Secure login
Payment calculation	Fee computation logic	Doctor charge + hospital fee
Feedback routing	Role-based routing	Ensure admin-only visibility
AI symptom analysis	API-based inference	Provide guidance

Table 4: Algorithms Overview.

3.7. Constraints and Design Justification.

Technical Constraints.

- Must run on local server (XAMPP).
- Must use open-source technologies.
- Must support limited hardware in clinics.
- Must be lightweight for fast loading.

Technology	Why Selected
PHP	Easy integration, fast processing, widely used in Sri Lanka.
MySQL	Reliable, secure, supports relational healthcare data.
HTML/CSS/JavaScript	Lightweight, responsive, accessible.
Email API	Essential for patient reminders.
AI API	Adds modern features to enhance patient interaction.

Table 5: Justification of Technology Choices.

3.8. Summary.

This chapter introduced the specifications and design of the Online Hospital Management System for HealthCare Plus Hospital. This chapter discussed the user interfaces, dynamic features, static structure, algorithm, processes and design rationale of the system. Through the study of the system using various viewpoints: UI, diagrams, database, behavior and architecture, this chapter provides a comprehensive platform for the implementation of the system.

4. Chapter IV – Implementation.

4.1. Introduction.

The implementation of the system design, the theory of concepts and ideas into real systems, will be addressed in this chapter. This includes the use of developed concepts and the challenges encountered during the implementation, along with the approaches used to tackle these challenges. It also highlights the unforeseen challenges that come up and how these challenges are solved.

4.2. Problems during Implementation.

In the initial phase, the overlapping of various user roles that have similar session variables led to unintended session overriding. For instance, a user logs into the system using the patient role and then changes the role without logging out, but it still shows the patient's data.

How it was fixed: Role-based session variables like `patient_session`, `admin_session` and etc. were added. Session regeneration was used after the login process for protecting session fixation attacks.

The initial form allowed two patients to book appointments with the same doctor for the same time slot if both patients filled out the form at the exact moment. This was due to the fact that it involved two different queries – check availability and save.

Solution: A combined atomic SQL transaction was used. This ensured that availability was checked in the same transaction that inserted an appointment.

Because of the cascade connections, deleting a patient resulted in breaking appointments and reports.

Solution: Foreign keys were modified using, `ON DELETE RESTRICT` for critical data. Soft delete implemented through status fields rather than delete operations.

The original dashboard layout was not user-friendly.

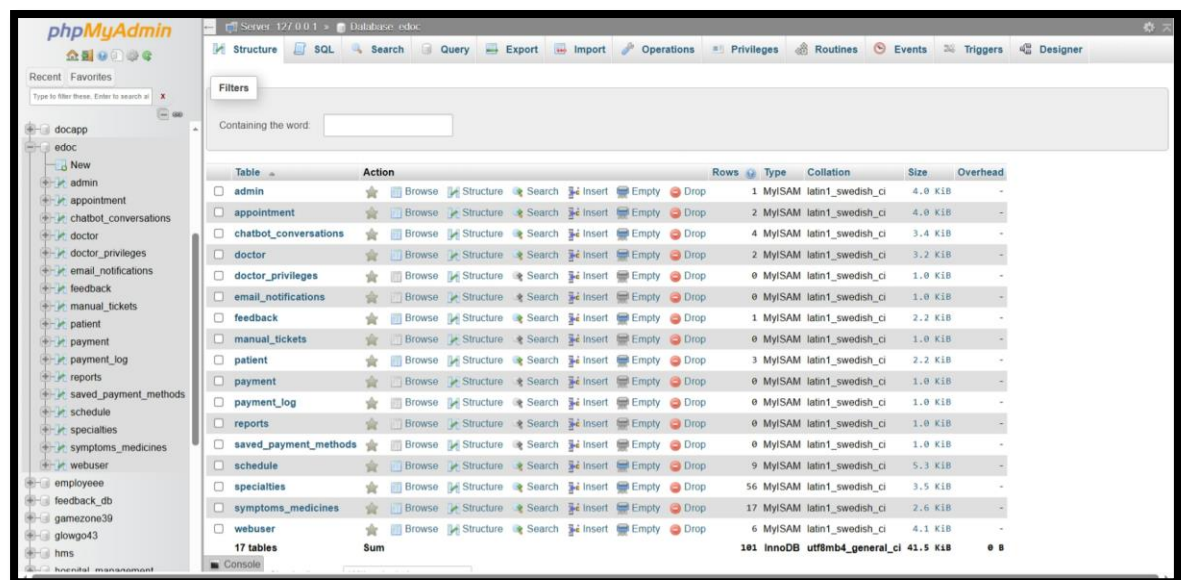
Solution: CSS Grid and Flexbox replaced fixed size layouts. It was also optimized for display resolution standards at 1024 x 768 and 1366 x 768.

4.3. Coding Segment.

4.3.1. Database Connection.

This system employs MySQL as the relational database management system and utilizes PHP for developing the connection for the database. The database connection works as the communication interface between the application logic and the stored data of the hospital, allowing secure, efficient and reliable transaction of data.

Database for the System.



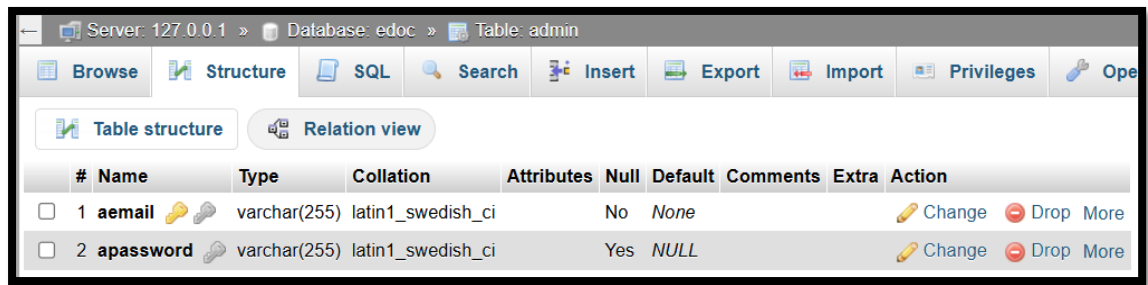
The screenshot shows the phpMyAdmin interface for a database named 'edoc'. The left sidebar lists various tables including admin, appointment, chatbot_conversations, doctor, doctor_privileges, email_notifications, manual_tickets, patient, payment, payment_log, reports, saved_payment_methods, specialties, symptoms_medicines, webuser, employeee, feedback_db, gamezone39, glowgo43, and hms. The main panel displays a table structure for 'edoc' with columns: Table, Action, Rows, Type, Collation, Size, and Overhead. The table structure is as follows:

Table	Action	Rows	Type	Collation	Size	Overhead
admin	Browse Structure Search Insert Empty Drop	1	MyISAM	latin1_swedish_ci	4.0 KIB	-
appointment	Browse Structure Search Insert Empty Drop	2	MyISAM	latin1_swedish_ci	4.0 KIB	-
chatbot_conversations	Browse Structure Search Insert Empty Drop	4	MyISAM	latin1_swedish_ci	3.4 KIB	-
doctor	Browse Structure Search Insert Empty Drop	2	MyISAM	latin1_swedish_ci	3.2 KIB	-
doctor_privileges	Browse Structure Search Insert Empty Drop	0	MyISAM	latin1_swedish_ci	1.0 KIB	-
email_notifications	Browse Structure Search Insert Empty Drop	0	MyISAM	latin1_swedish_ci	1.0 KIB	-
feedback	Browse Structure Search Insert Empty Drop	1	MyISAM	latin1_swedish_ci	2.2 KIB	-
manual_tickets	Browse Structure Search Insert Empty Drop	0	MyISAM	latin1_swedish_ci	1.0 KIB	-
patient	Browse Structure Search Insert Empty Drop	3	MyISAM	latin1_swedish_ci	2.2 KIB	-
payment	Browse Structure Search Insert Empty Drop	0	MyISAM	latin1_swedish_ci	1.0 KIB	-
payment_log	Browse Structure Search Insert Empty Drop	0	MyISAM	latin1_swedish_ci	1.0 KIB	-
reports	Browse Structure Search Insert Empty Drop	0	MyISAM	latin1_swedish_ci	1.0 KIB	-
saved_payment_methods	Browse Structure Search Insert Empty Drop	0	MyISAM	latin1_swedish_ci	1.0 KIB	-
schedule	Browse Structure Search Insert Empty Drop	9	MyISAM	latin1_swedish_ci	5.3 KIB	-
specialties	Browse Structure Search Insert Empty Drop	56	MyISAM	latin1_swedish_ci	3.5 KIB	-
symptoms_medicines	Browse Structure Search Insert Empty Drop	17	MyISAM	latin1_swedish_ci	2.6 KIB	-
webuser	Browse Structure Search Insert Empty Drop	6	MyISAM	latin1_swedish_ci	4.1 KIB	-
17 tables	Sum	101	InnoDB	utf8mb4_general_ci	41.5 KIB	0 B

Figure 35: Database for the System.

There are twenty-two tables in the system database including admin, allergies, appointment, chatbot_conversations, doctor, doctor_privileges, family_history, feedback, immunizations, lab_tests, manual_tickets, medical_documents, medical_records, medications, patient, reports, schedule, specialties, surgical_historhy, symptoms_medicines, vital_signs, webuser.

Admin Table.



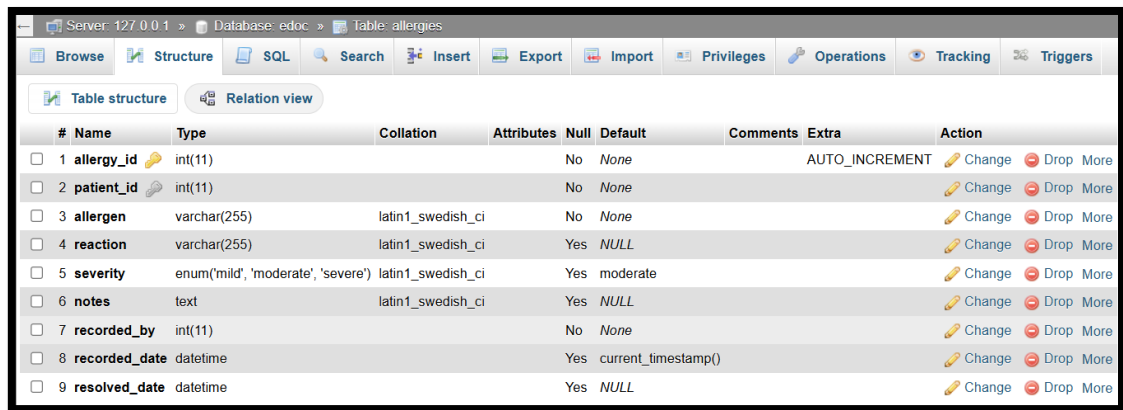
Server: 127.0.0.1 » Database: edoc » Table: admin

Table structure | Relation view

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	aemail	varchar(255)	latin1_swedish_ci		No	None			Change Drop More
☐ 2	apassword	varchar(255)	latin1_swedish_ci		Yes	NULL			Change Drop More

Figure 36: Admin Table.

Allergies Table.



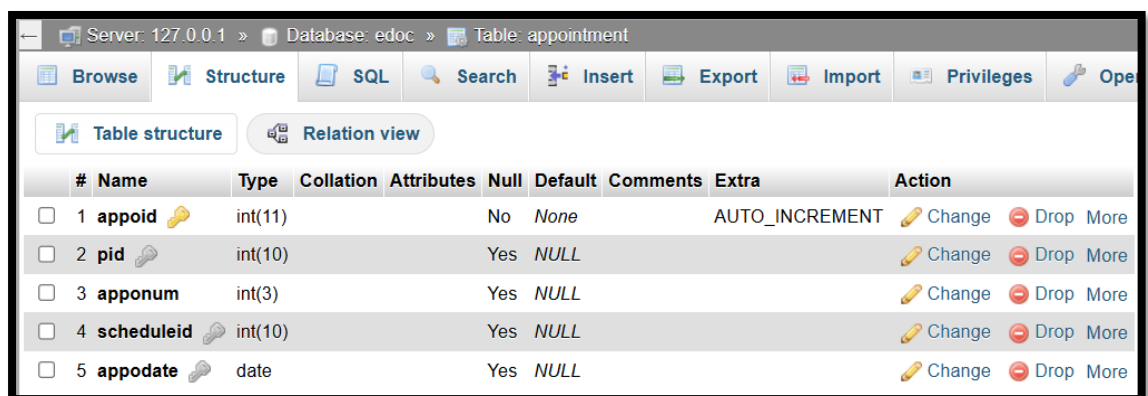
Server: 127.0.0.1 » Database: edoc » Table: allergies

Table structure | Relation view

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	allergy_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐ 2	patient_id	int(11)			No	None			Change Drop More
☐ 3	allergen	varchar(255)	latin1_swedish_ci		No	None			Change Drop More
☐ 4	reaction	varchar(255)	latin1_swedish_ci		Yes	NULL			Change Drop More
☐ 5	severity	enum('mild', 'moderate', 'severe')	latin1_swedish_ci		Yes	moderate			Change Drop More
☐ 6	notes	text	latin1_swedish_ci		Yes	NULL			Change Drop More
☐ 7	recorded_by	int(11)			No	None			Change Drop More
☐ 8	recorded_date	datetime			Yes	current_timestamp()			Change Drop More
☐ 9	resolved_date	datetime			Yes	NULL			Change Drop More

Figure 37: Allergies Table.

Appointment Table.



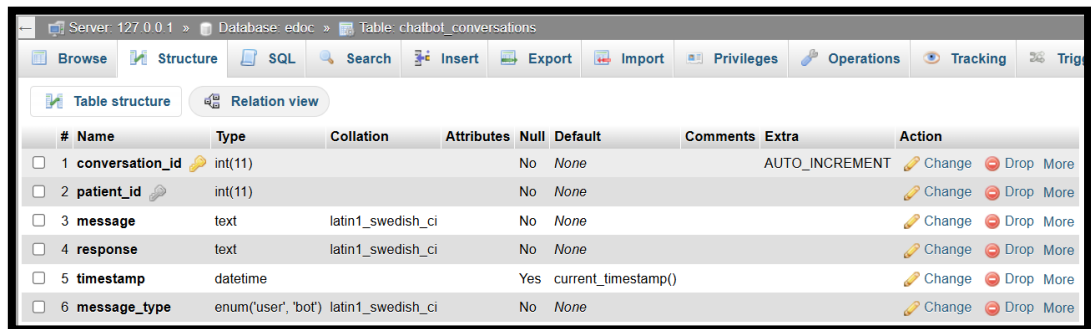
Server: 127.0.0.1 » Database: edoc » Table: appointment

Table structure | Relation view

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	appoid	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐ 2	pid	int(10)			Yes	NULL			Change Drop More
☐ 3	apponum	int(3)			Yes	NULL			Change Drop More
☐ 4	scheduleid	int(10)			Yes	NULL			Change Drop More
☐ 5	appodate	date			Yes	NULL			Change Drop More

Figure 38: Appointment Table.

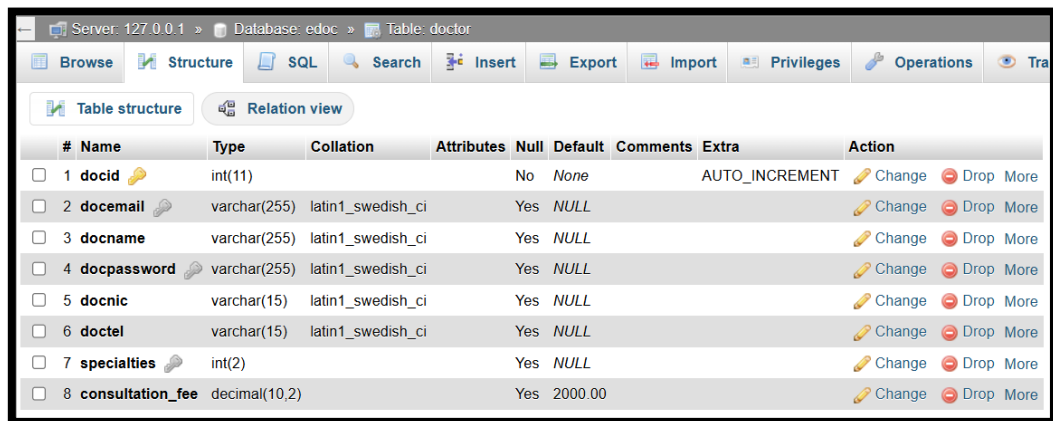
Chatbot Conversation Table.



#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	conversation_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	patient_id	int(11)			No	None			Change Drop More
3	message	text	latin1_swedish_ci		No	None			Change Drop More
4	response	text	latin1_swedish_ci		No	None			Change Drop More
5	timestamp	datetime			Yes	current_timestamp()			Change Drop More
6	message_type	enum('user', 'bot')	latin1_swedish_ci		No	None			Change Drop More

Figure 39: Chatbot Conversation.

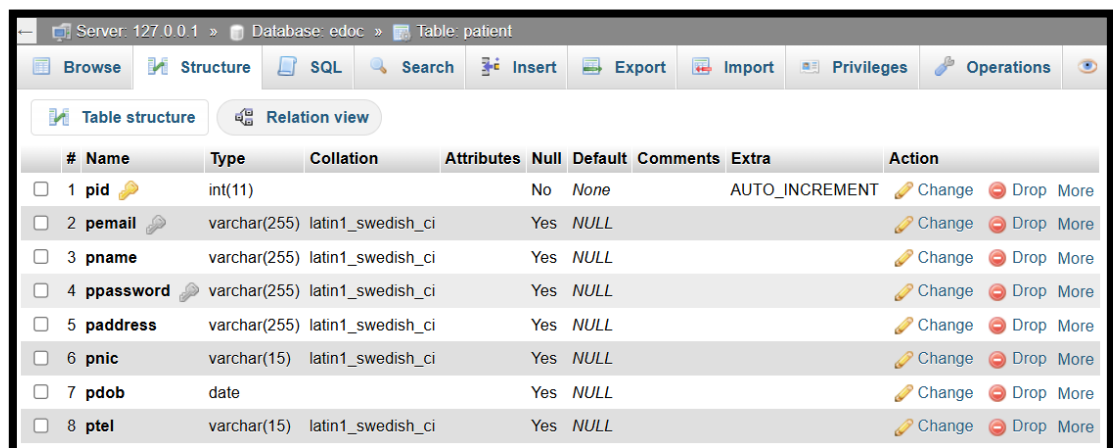
Doctor Table.



#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	docid	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	docemail	varchar(255)	latin1_swedish_ci		Yes	NULL			Change Drop More
3	docname	varchar(255)	latin1_swedish_ci		Yes	NULL			Change Drop More
4	docpassword	varchar(255)	latin1_swedish_ci		Yes	NULL			Change Drop More
5	docnic	varchar(15)	latin1_swedish_ci		Yes	NULL			Change Drop More
6	doctel	varchar(15)	latin1_swedish_ci		Yes	NULL			Change Drop More
7	specialties	int(2)			Yes	NULL			Change Drop More
8	consultation_fee	decimal(10,2)			Yes	2000.00			Change Drop More

Figure 40: Doctor Table.

Patient Table.



#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	pid	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	pemail	varchar(255)	latin1_swedish_ci		Yes	NULL			Change Drop More
3	pname	varchar(255)	latin1_swedish_ci		Yes	NULL			Change Drop More
4	ppassword	varchar(255)	latin1_swedish_ci		Yes	NULL			Change Drop More
5	paddress	varchar(255)	latin1_swedish_ci		Yes	NULL			Change Drop More
6	pnice	varchar(15)	latin1_swedish_ci		Yes	NULL			Change Drop More
7	pdob	date			Yes	NULL			Change Drop More
8	ptel	varchar(15)	latin1_swedish_ci		Yes	NULL			Change Drop More

Figure 41: Patient Table.

Feedback Table.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	feedback_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	user_id	int(11)			Yes	NULL			Change Drop More
3	user_type	enum('patient', 'doctor', 'admin')	latin1_swedish_ci		Yes	NULL			Change Drop More
4	subject	varchar(255)	latin1_swedish_ci		Yes	NULL			Change Drop More
5	message	text	latin1_swedish_ci		Yes	NULL			Change Drop More
6	response	text	latin1_swedish_ci		Yes	NULL			Change Drop More
7	responded_by	int(11)			Yes	NULL			Change Drop More
8	responded_date	datetime			Yes	NULL			Change Drop More
9	created_date	datetime			Yes	current_timestamp()			Change Drop More

Figure 42: Feedback Table.

Reports Table.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	report_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	patient_id	int(11)			Yes	NULL			Change Drop More
3	report_type	enum('appointments', 'payments', 'doctors', 'patient', 'reports')	latin1_swedish_ci		Yes	NULL			Change Drop More
4	generated_by	int(11)			Yes	NULL			Change Drop More
5	generated_date	datetime			Yes	current_timestamp()			Change Drop More
6	report_data	longtext	latin1_swedish_ci		Yes	NULL			Change Drop More
7	file_path	varchar(255)	latin1_swedish_ci		Yes	NULL			Change Drop More

Figure 43: Reports Table.

Schedule Table.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	scheduleid	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	docid	varchar(255)	latin1_swedish_ci		Yes	NULL			Change Drop More
3	title	varchar(255)	latin1_swedish_ci		Yes	NULL			Change Drop More
4	scheduledate	date			Yes	NULL			Change Drop More
5	scheduletime	time			Yes	NULL			Change Drop More
6	nop	int(4)			Yes	NULL			Change Drop More
7	session_fee	decimal(10,2)			Yes	NULL			Change Drop More

Figure 44: Schedule Table.

The database component of the system is the fundamental code that is being used to store and manipulate data about patient, doctor, appointment, admin, staff, report, payment and account. It has been coded with MySQL so that it becomes secure, scalable, query-efficient and reliable. The database schema includes normalized tables, foreign keys and primary keys so that referential integrity can be preserved and redundancy can be eliminated.

4.4. Implementation Code.

4.4.1. Authentication.

```

if($_POST){
    $email=$_POST['useremail'];
    $password=$_POST['userpassword'];

    $error=<label for="promter" class="form-label"></label>;

    $stmt = $database->prepare("select usertype from webuser where email=?");
    $stmt->bind_param("s", $email);
    $stmt->execute();
    $result = $stmt->get_result();

    if($result->num_rows==1){
        $utype=$result->fetch_assoc()['usertype'];
        if ($utype=='p'){
            $stmt = $database->prepare("select * from patient where pemail=? and ppassword=?");
            $stmt->bind_param("ss", $email, $password);
            $stmt->execute();
            $checker = $stmt->get_result();
            if ($checker->num_rows==1){
                $_SESSION['user']=$email;
                $_SESSION['usertype']='p';
                header('location: patient/index.php');
                exit();
            }
            $error=<label for="promter" class="error-message">Wrong credentials: Invalid email or password</label>;
        }
        elseif($utype=='a'){
            $stmt = $database->prepare("select * from admin where aemail=? and apassword=?");
            $stmt->bind_param("ss", $email, $password);
            $stmt->execute();
            $checker = $stmt->get_result();
            if ($checker->num_rows==1){
                $_SESSION['user']=$email;
                $_SESSION['usertype']='a';
                header('location: admin/index.php');
            }
        }
    }
}

```

Figure 45: Code for Authentication.

Registration Page.

```

<?php if ($error != ''): ?>
<div class="error-message"><?php echo $error; ?></div>
<?php endif; ?>

<form action="" method="POST" >
<input type="hidden" name="step" value="personal_details">
<div class="label-td">
<label for="name" class="form-label">Name: </label>
</div>
<div style="display: flex; gap: 15px; margin-bottom: 20px;">
<input type="text" name="fname" class="input-text" placeholder="First Name" required style="flex: 1;"
value=<?php echo isset($_SESSION['personal']['fname']) ? htmlspecialchars($_SESSION['personal']['fname']) : ''; ?>
<input type="text" name="lname" class="input-text" placeholder="Last Name" required style="flex: 1;"
value=<?php echo isset($_SESSION['personal']['lname']) ? htmlspecialchars($_SESSION['personal']['lname']) : ''; ?>
</div>
<div class="label-td">
<label for="address" class="form-label">Address: </label>
<input type="text" name="address" class="input-text" placeholder="Address" required
value=<?php echo isset($_SESSION['personal']['address']) ? htmlspecialchars($_SESSION['personal']['address']) : ''; ?>
</div>
<div class="label-td">
<label for="nic" class="form-label">NIC: </label>
<input type="text" name="nic" class="input-text" placeholder="NIC Number" required
value=<?php echo isset($_SESSION['personal']['nic']) ? htmlspecialchars($_SESSION['personal']['nic']) : ''; ?>
</div>
<div class="label-td">
<label for="dob" class="form-label">Date of Birth: </label>
<input type="date" name="dob" class="input-text" required
value=<?php echo isset($_SESSION['personal']['dob']) ? htmlspecialchars($_SESSION['personal']['dob']) : ''; ?>
</div>
<div style="display: flex; gap: 15px; margin-top: 10px;">
<input type="reset" value="Reset" class="login-btn" style="background: linear-gradient(135deg, #D8EBFA 0%, #B8D6F0 100%); color:
<input type="submit" value="Next" class="login-btn">
</div>
<div class="signup-link">
<span class="sub-text">Already have an account? </span>
<a href="login.php" class="hover-link">Login</a>
</div>

```

Figure 46: Code for Registration Page.

Home Page.

```
$sql_specialties = "SELECT COUNT(*) as total FROM specialties";
$result_specialties = $database->query($sql_specialties);
$specialties_count = $result_specialties->fetch_assoc()['total'];

$sql_appointments = "SELECT COUNT(*) as total FROM appointment";
$result_appointments = $database->query($sql_appointments);
$appointments_count = $result_appointments->fetch_assoc()['total'];

$sql_patients = "SELECT COUNT(*) as total FROM patient";
$result_patients = $database->query($sql_patients);
$patients_count = $result_patients->fetch_assoc()['total'];

$database->close();
-?>

<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>HealthCarePlus - Advanced Medical Care</title>
  <link rel="stylesheet" href="css/font-inter.css">
  <style>
    :root {
      --primary: #2563eb;
      --primary-dark: #1d4ed8;
      --secondary: #0ea5e9;
      --success: #10b981;
      --warning: #f59e0b;
      --danger: #ef4444;
      --dark: #1e293b;
      --gray: #64748b;
      --light: #f1f5f9;
      --white: #ffffff;
    }
  </style>
</head>
```

Figure 47: Code for Home Page.

Admin Dashboard.

```
<table class="menu-container" border="0">
  <tr>
    <td style="padding:10px" colspan="2">
      <table border="0" class="profile-container">
        <tr>
          <td width="30%" style="padding-left:20px">
            
          </td>
          <td style="padding:0px;margin:0px;">
            <p class="profile-title">Administrator</p>
            <p class="profile-subtitle">admin@edoc.com</p>
          </td>
        </tr>
        <tr>
          <td colspan="2">
            <a href="..logout.php" ><input type="button" value="Log out" class="logout-btn btn-primary-soft btn"></a>
          </td>
        </tr>
      </table>
    </td>
  </tr>
  <tr class="menu-row">
    <td class="menu-btn menu-icon-dashbord menu-active menu-icon-dashbord-active">
      <a href="index.php" class="non-style-link-menu non-style-link-menu-active"><div><p class="menu-text">Dashboard</p></div></a>
    </td>
  </tr>
  <tr class="menu-row">
    <td class="menu-btn menu-icon-doctor">
      <a href="doctors.php" class="non-style-link-menu"><div><p class="menu-text">Doctors</p></div>
    </td>
  </tr>
  <tr class="menu-row">
    <td class="menu-btn menu-icon-schedule">
      <a href="schedule.php" class="non-style-link-menu"><div><p class="menu-text">Schedule</p></div>
    </td>
  </tr>
  <tr class="menu-row">
    <td class="menu-btn menu-icon-patient">
      <a href="patients.php" class="non-style-link-menu"><div><p class="menu-text">Patients</p></div>
    </td>
  </tr>
  <tr class="menu-row">
    <td class="menu-btn menu-icon-report">
      <a href="reports.php" class="non-style-link-menu"><div><p class="menu-text">Reports</p></div>
    </td>
  </tr>
  <tr class="menu-row">
    <td class="menu-btn menu-icon-settings">
      <a href="settings.php" class="non-style-link-menu"><div><p class="menu-text">Settings</p></div>
    </td>
  </tr>
  <tr class="menu-row">
    <td class="menu-btn menu-icon-help">
      <a href="help.php" class="non-style-link-menu"><div><p class="menu-text">Help</p></div>
    </td>
  </tr>
  <tr class="menu-row">
    <td class="menu-btn menu-icon-logout">
      <a href="logout.php" class="non-style-link-menu"><div><p class="menu-text">Logout</p></div>
    </td>
  </tr>
</table>
```

Figure 48: Code for Admin Dashboard.

Patient Dashboard.

```
<div class="container">
  <div class="menu">
    <table class="menu-container" border="0">
      <tr>
        <td style="padding:10px; colspan="2">
          <table border="0" class="profile-container">
            <tr>
              <td width="30%" style="padding-left:20px">
                <?php if(isset($userfetch["profile_picture"])) && $userfetch["profile_picture"]): ?>
                  " alt="" width="" height="" />
                <?php else: ?>
                  
                <?php endif: ?>
              </td>
              <td style="padding:0px;margin:0px;">
                <p class="profile-title"><?php echo substr($username,0,13) ?>..</p>
                <p class="profile-subtitle"><?php echo substr($useremail,0,22) ?></p>
              </td>
            </tr>
            <tr>
              <td colspan="2">
                <a href="..../logout.php" ><input type="button" value="Log out" class="logout-btn btn-primary-soft btn"></a>
              </td>
            </tr>
          </table>
        </td>
        <tr class="menu-row">
          <td class="menu-btn menu-icon-home menu-active menu-icon-home-active">
            <a href="index.php" class="non-style-link-menu non-style-link-menu-active"><div><p class="menu-text">Home</p></div></a>
          </td>
        </tr>
        <tr class="menu-row">
          <td class="menu-btn menu-icon-doctor">
            <a href="doctors.php" class="non-style-link-menu"><div><p class="menu-text">All Doctors</p></div></a>
          </td>
        </tr>
      </tr>
    </table>
  </div>
</div>
```

Figure 49: Code for Patient Dashboard.

Doctor Dashboard.

```
<td colspan="4">
  <center>
    <div class="abc scroll">
      <table width="93%" class="sub-table scrolldown" border="0">
        <thead>
          <tr>
            <th class="table-headin">
              Doctor Name
            </th>
            <th class="table-headin">
              Email
            </th>
            <th class="table-headin">
              Specialties
            </th>
            <th class="table-headin">
              Events
            </th>
          </tr>
        </thead>
        <tbody>
          <?php
            $result= $database->query($sqlmain);
            if($result->num_rows==0){
              echo '
                <tr>
                  <td colspan="4">
                    <br><br><br><br>
                    <center>
                      
                    </center>
                    <br>
                    <p class="heading-main12" style="margin-left: 45px;font-size:20px;color:rgb(49, 49, 49)">We couldnt find anything related to your k
                    <a class="non-style-link" href="doctors.php"><button class="login-btn btn-primary-soft btn" style="display: flex;justify-content:
                    </a>
                    </center>
                    <br><br><br><br>
                  </td>
                </tr>';
            }
          </?php>
        </tbody>
      </table>
    </div>
  </center>
</td>
</tr>';
```

Figure 50: Code for Doctor Dashboard.

Report Generation.

```
<?php
if (isset($_POST['generate_report'])) {
    $report_type = $_POST['report_type'];
    $date_from = $_POST['date_from'];
    $date_to = $_POST['date_to'];
    $admin_id = 1;
    $report_data = "";
    switch($report_type) {
        case 'appointments':
            $stmt = $database->prepare("SELECT COUNT(*) as count FROM appointment WHERE appodate BETWEEN ? AND ?");
            $stmt->bind_param("ss", $date_from, $date_to);
            $stmt->execute();
            $result = $stmt->get_result();
            $count = $result->fetch_assoc()['count'];
            $report_data = "Appointments Report ($date_from to $date_to)\n\n";
            $report_data .= "Total appointments in period: $count\n\n";
            $report_data .= "Detailed appointment data would be listed here in a full implementation.";
            break;
        case 'payments':
            $report_data = "Payments Report ($date_from to $date_to)\n\n";
            $report_data .= "Payment data would be listed here in a full implementation.";
            break;
        case 'doctors':
            $stmt = $database->prepare("SELECT COUNT(*) as count FROM doctor");
            $stmt->execute();
            $result = $stmt->get_result();
            $count = $result->fetch_assoc()['count'];
            $report_data = "Doctors Report ($date_from to $date_to)\n\n";
            $report_data .= "Total doctors: $count\n\n";
            $report_data .= "Detailed doctor data would be listed here in a full implementation.";
            break;
        case 'patients':
            $stmt = $database->prepare("SELECT COUNT(*) as count FROM patient");
            $stmt->execute();
            $result = $stmt->get_result();
            $count = $result->fetch_assoc()['count'];
            $report_data = "Patients Report ($date_from to $date_to)\n\n";
            $report_data .= "Total patients: $count\n\n";
            $report_data .= "Detailed patient data would be listed here in a full implementation.";
```

Figure 51: Code for Report Generation.

Payment Page.

```
<td colspan="2">
    <div class="payment-container">
        <div class="section-header">
            <h1>Secure Payment Processing</h1>
            <p>Complete your appointment booking with confidence</p>
        </div>
        <div class="details-grid">
            <div class="details-card">
                <h2 style="color: #000000; margin-bottom: 25px; font-size: 1.5rem;">Appointment Details</h2>
                <div class="detail-item">
                    <div class="detail-label">Doctor</div>
                    <div class="detail-value"><?php echo htmlspecialchars($schedule['docname']);?></div>
                </div>
                <div class="detail-item">
                    <div class="detail-label">Specialization</div>
                    <div class="detail-value">
                        <?php
                            $spec_sql = "SELECT sname FROM specialties WHERE id=?";
                            $spec_stmt = $database->prepare($spec_sql);
                            $spec_stmt->bind_param("i", $schedule['specialties']);
                            $spec_stmt->execute();
                            $spec_result = $spec_stmt->get_result();
                            $specialty = $spec_result->fetch_assoc();
                            echo htmlspecialchars($specialty['sname'] ?? 'General');
                        <?php
                            $spec_stmt->close();
                        <?php
                    </div>
                </div>
                <div class="detail-item">
                    <div class="detail-label">Session Title</div>
                    <div class="detail-value"><?php echo htmlspecialchars($schedule['title']);?></div>
                </div>
            </div>
        </div>
    </div>
```

Figure 52: Code for Payment Page.

Email Configuration.

```
class EmailUtility {

    private static $apiKey = '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';
    private static $apiUrl = 'https://api.brevo.com/v3/smtpp/email';

    public static function sendAppointmentConfirmation($patientEmail, $patientName, $doctorName, $appointmentDate, $appointmentTime, $appointmentNumber) {
        $subject = 'Appointment Confirmation - HealthCarePlus';
        $htmlContent = self::getAppointmentConfirmationTemplate($patientName, $doctorName, $appointmentDate, $appointmentTime, $appointmentNumber);

        return self::sendEmailViaAPI($patientEmail, $patientName, $subject, $htmlContent);
    }

    public static function sendWelcomeEmail($patientEmail, $patientName) {
        $subject = 'Welcome to HealthCarePlus!';
        $htmlContent = self::getWelcomeEmailTemplate($patientName);

        return self::sendEmailViaAPI($patientEmail, $patientName, $subject, $htmlContent);
    }

    public static function sendPaymentReceipt($patientEmail, $patientName, $appointmentNumber, $doctorName, $appointmentDate, $totalFee) {
        $subject = 'Payment Receipt - Appointment #' . $appointmentNumber;
        $htmlContent = self::getPaymentReceiptTemplate($patientName, $appointmentNumber, $doctorName, $appointmentDate, $totalFee);

        return self::sendEmailViaAPI($patientEmail, $patientName, $subject, $htmlContent);
    }

    private static function sendEmailViaAPI($toEmail, $toName, $subject, $htmlContent) {
        $data = array(
            'sender' => array(
                'name' => 'HealthCarePlus',
                'email' => 'senurasamintha377@gmail.com'
            ),
            'to' => array(
                array(
                    'email' => $toEmail,
                    'name' => $toName
                )
            ),
            'subject' => $subject,
            'htmlContent' => $htmlContent
        );
    }
}
```

Figure 53: Code for Email.

Chatbot Connection.

```
function saveConversation($database, $patient_id, $message, $response) {
    // Save user message
    $stmt = $database->prepare("INSERT INTO chatbot_conversations (patient_id, message, response, message_type) VALUES (?, ?, 'user')");
    $stmt->bind_param("is", $patient_id, $message);
    $stmt->execute();

    // Save bot response
    $stmt = $database->prepare("INSERT INTO chatbot_conversations (patient_id, message, response, message_type) VALUES (?, 'bot')");
    $stmt->bind_param("is", $patient_id, $response);
    $stmt->execute();
}

function getMedicineRecommendation($database, $symptom) {
    $stmt = $database->prepare("SELECT recommended_medicine, specialty_id FROM symptoms_medicines WHERE symptom LIKE ?");
    $search_term = "%" . strtolower($symptom) . "%";
    $stmt->bind_param("s", $search_term);
    $stmt->execute();
    $result = $stmt->get_result();

    if ($result->num_rows > 0) {
        return $result->fetch_assoc();
    }
    return null;
}

function getSpecialtyName($database, $specialty_id) {
    $stmt = $database->prepare("SELECT sname FROM specialties WHERE id = ?");
    $stmt->bind_param("i", $specialty_id);
    $stmt->execute();
    $result = $stmt->get_result();

    if ($result->num_rows > 0) {
        $row = $result->fetch_assoc();
        return $row['sname'];
    }
    return "General Practitioner";
}

function getDoctorsBySpecialty($database, $specialty_id) {
    $stmt = $database->prepare("SELECT docid, docname FROM doctor WHERE specialties = ? LIMIT 3");
    $stmt->bind_param("i", $specialty_id);
}
```

Figure 54: Code for Chatbot.

Feedback Page.

```
$sqlmain= "select * from patient where pemail=?";
$stmt = $database->prepare($sqlmain);
$stmt->bind_param("s",$useremail);
$stmt->execute();
$userrow = $stmt->get_result();
$userfetch=$userrow->fetch_assoc();
$pid= $userfetch["pid"];
$name=$userfetch["pname"];

if ($_POST) {
    if (isset($_POST['submit_feedback'])) {
        $subject = $_POST['subject'];
        $message = $_POST['message'];

        $sql = "INSERT INTO feedback (user_id, user_type, subject, message) VALUES (?, 'patient', ?, ?)";
        $stmt = $database->prepare($sql);
        $stmt->bind_param("iss", $pid, $subject, $message);
        if ($stmt->execute()) {
            setMessage("Feedback submitted successfully! We'll review your message and respond soon.", "success");
        } else {
            setMessage("There was an error submitting your feedback. Please try again.", "error");
        }
        $stmt->close();

        header("Location: feedback.php");
        exit();
    }
}

?>
<div class="container">
    <div class="menu">
        <table class="menu-container" border="0">
            <tr>
                <td style="padding:10px" colspan="2">
                    <table border="0" class="profile-container">
                        <tr>
                            <td width="30%" style="padding-left:20px" >
                                
                            </td>
                            <td style="padding:0px;margin:0px;">
```

Figure 55: Code for Feedback.

4.5. Summary.

Implementation of this system entails the transformation of the designed architecture into working components using HTML, CSS, JavaScript and PHP. The most critical components of the health IT system for authentication, appointment management, payment management, report generation and notification systems were designed using effective and optimal logic. Despite encountering several issues involving API integration problems, issues involving session management conflicts, and database limitations, all challenges were addressed through intensive debugging and enhancement of health IT systems architectural design. The following chapter shall analyze the performance assessment of the health IT solution.

5. Chapter V – Result and Evaluation.

5.1. Introduction.

In this chapter, the results and analysis of the online hospital management system developed for HealthCare Plus Hospital are presented. The main aim of this phase is to check if the system satisfies the requirements of functional and non-functional specifications and determine what extent the objectives of the project have been achieved. Structured testing is adopted in order to confirm the reliability and functionality of the developed system.

5.2. Plan of Evolution.

The testing phase incorporated a number of testing methods so that the online hospital management system can perform appropriately. In view of limitations of time as well as resources, only limited stress testing and deployment tests could be accomplished, but adequate functionality tests were achieved.

These tests were conducted by:

Unit Testing: Individual major modules (Login, Patient Management, Appointment Scheduling and Payment Processing) have been tested rigorously.

Integration Testing: The interaction of modules (Appointment, Payment, Login, Dashboard) has been tested for ensuring a proper flow of data.

Functional Testing: Each functional requirement was verified for its expected output.

User Interface Testing: UI responsiveness, form validation, navigation flow and error handling were also assessed.

All these testing techniques played an important role in ensuring that the system functions well under normal working environment conditions.

5.3. Test Plan and Test Cases.

The purpose of a test case is to define a certain situation with which one can check the expected behavior or function of a system. every test case has its own set of inputs, execution steps, expected results and actual outcomes. In order to make the testing process simpler, the system is segment into manageable modules.

Test and Deployment Plan.

System Testing Plan.

Logic Testing.

The logical design of the system must be tested to guarantee proper processing of data and proper execution of workflows.

Unit Testing: Every module (Patient Registration, Appointment Scheduling, Billing) will be tested separately to guarantee inputs lead to proper outputs.

Example: Cross-checking that billing module is correctly computing total payment including discount and pending balance.

Integration Testing: Work to ensure various modules are correctly passing data between each other.

Example: Cross-checking that appointment creations will refresh the patient's record as well as the doctor's schedule.

Database Testing: verify CRUD (Add, Update, Delete) operations are functioning correctly. Verify data consistency is being preserved across multiple tables (deleting a patient removes dependent appointments, but billing history is archived).

GUI Testing.

The user interface will be tested for usability, accessibility and consistency.

Navigation Testing: Ensure that all the buttons, links and menus navigate to the proper pages.

Example: “View Appointment” on the patient dashboard should show all appointments booked.

Input Validations: Ensure that the mandatory fields are correctly validated (avoiding duplicate usernames, validating phone numbers and emails).

Cross-Device and Browser Testing: Test for responsiveness on desktops, tablets and mobile. Ensure cross-browser compatibility with leading browsers (Chrome, Edge, Firefox, Safari).

User Experience Testing: Assess for ease for patients, doctors and admins. Dashboards display only the correct options based on role.

Functional Testing.

Patient Module: Registering, updating and deleting patient records. Viewing medical records.

Doctor Module: Managing availability schedules. Reviewing appointments and updating treatments.

Appointments Module: Scheduling, rescheduling and cancelling appointments. Verify status changes.

Billing Module: Generating invoices. Updating payment status.

User Account Management: Secure login and authentication.

Deployment Plan.

Preparation Phase: Configure required servers, networks and security measures. Install database and backup systems.

Pilot Deployment: Deploy in a single department and collect feedback.

Full Deployment: Roll out to all departments. Migrate historical patient/doctor records.

Post-Deployment Monitoring: Track performance, fix initial bugs and provide continuous support.

Maintenance Plan.

Corrective Maintenance: Bug fixing and error resolution.

Adaptive Maintenance: System updates on regulations or hospital needs.

Perfective Maintenance: Performance and usability enhancements.

Preventive Maintenance: Regular backups, patch updates and security audits.

Modules that were extensively tested:

- User Login and Registration Module.
- Appointment Scheduling Module.
- Payment Processing Module.
- Report and Dashboard Module.
- Doctor Management Module.

Project Name	Online Hospital Management System for HealthCare Plus Hospital.			
Release Version	1.0			
Module Name	User Login			
Preconditions	Internet connection available, user account exists.			
Test Case ID	Test Case Description	Input	Expected Output	Result
01	User Login	Empty Username and Password.	Error message “Invalid Credentials”.	Pass
		Incorrect Username and Password.	Error message “We can’t find any account for this email.”	Pass
		Correct Username and Password.	Successfully Message “Login Successfully”. User redirected to role-based dashboard.	Pass
Output	 Figure 56: Test Case 01 Output.			
Evaluation	This test confirms that the authentication mechanism functions correctly and securely, ensuring only authorized users can access the system.			

Table 6: Test Case 01.

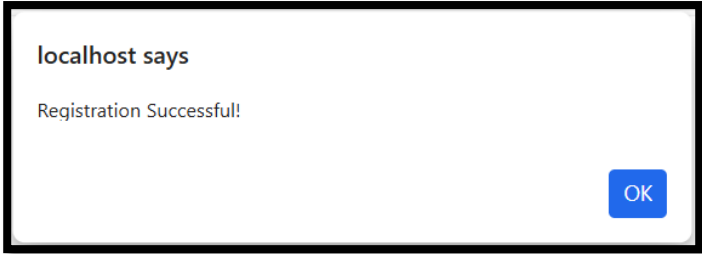
Project Name	Online Hospital Management System for HealthCare Plus Hospital.			
Release Version	1.0			
Module Name	User Registration.			
Preconditions	Internet connection available, Patient logged in, Doctor profile available.			
Test Case ID	Test Case Description	Input	Expected Output	Result
02	User Registration.	Empty required fields.	Error message “Please fill all required fields.”.	Pass
		Invalid email format.	Error message “Invalid email address.”	Pass
		Password and confirm password mismatch.	Error message “Passwords do not match.”	Pass
Output	 Figure 57: Test Case 02 Output.			
Evaluation	This registration module ensures data integrity and prevents invalid or incomplete registrations.			

Table 7: Test Case 02.

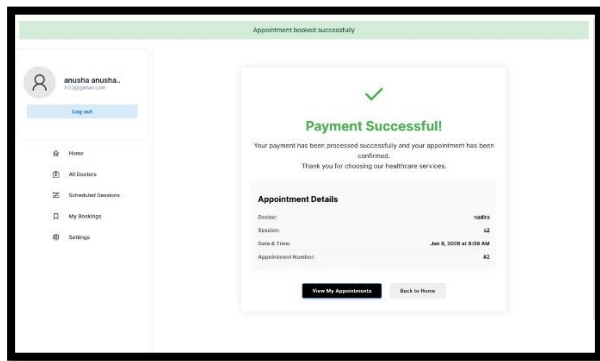
Project Name	Online Hospital Management System for HealthCare Plus Hospital.			
Release Version	1.0			
Module Name	Appointment Booking.			
Preconditions	Internet connection available, patient logged in, doctor profile available.			
Test Case ID	Test Case Description	Input	Expected Output	Result
03	Book Appointment	Doctor not selected.	Error message “Please Select a Doctor”.	Pass
		Date not selected.	Error message “Please Select Appointment Date.”	Pass
		Time slot already booked.	Error message “Selected Time Slot Unavailable”.	Pass
Output	 Figure 58: Test Case 03 Output.			
Evaluation	The test verifies correct appointment scheduling and conflict prevention.			

Table 8: Test Case 03.

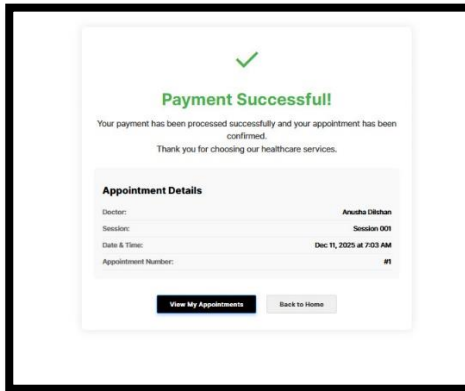
Project Name	Online Hospital Management System for HealthCare Plus Hospital.			
Release Version	1.0			
Module Name	Online Payment.			
Preconditions	Internet connection available, user account exists.			
Test Case ID	Test Case Description	Input	Expected Output	Result
04	Make Payment.	Incorrect Payment Details.	Error message “Invalid Credentials”.	Pass
		Correct details insufficient balance.	Error message “Insufficient Balance.”	Pass
		Correct details and sufficient balance.	Payment Successful and Receipt Generated.	Pass
Output	 Figure 59: Test Case 04 Output.			
Evaluation	The test confirms reliability and security of the payment process.			

Table 9: Test Case 04.

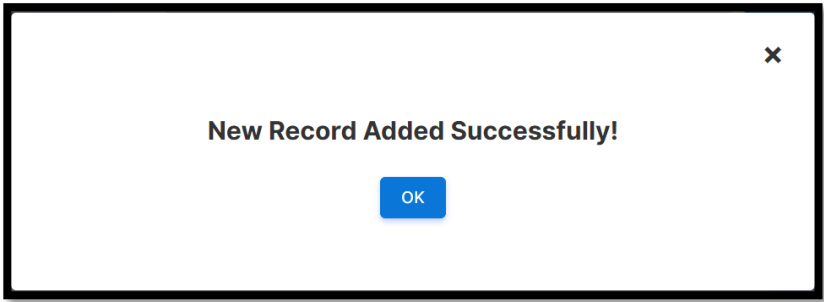
Project Name	Online Hospital Management System for HealthCare Plus Hospital.			
Release Version	1.0			
Module Name	Doctor Management.			
Preconditions	Internet connection available, admin logged in.			
Test Case ID	Test Case Description	Input	Expected Output	Result
05	Make Payment.	Missing mandatory fields.	Error message “Please fill required fields”.	Pass
		Invalid Specialization.	Error message “Invalid Specialization.”	Pass
		All valid details.	Doctor profile added successfully.	Pass
Output	 Figure 60: Test Case 05 Output.			
Evaluation	Ensures only valid and complete doctor records are maintained.			

Table 10: Test Case 05.

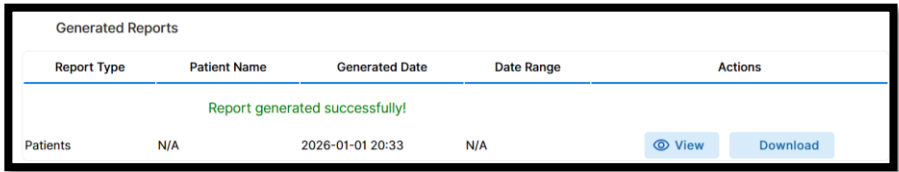
Project Name	Online Hospital Management System for HealthCare Plus Hospital.			
Release Version	1.0			
Module Name	Report Management.			
Preconditions	Internet connection available, admin logged in.			
Test Case ID	Test Case Description	Input	Expected Output	Result
06	Generate Reports.	No records available.	Error message “No records found”.	Pass
		Appointment data available.	Correct appointment report generated.	Pass
		Payment data available.	Accurate revenue report generated.	Pass
Output	 The screenshot shows a web interface titled 'Generated Reports'. It contains a table with columns: Report Type, Patient Name, Generated Date, Date Range, and Actions. A green message 'Report generated successfully!' is displayed above the table. The table has one row with 'Patients' as the Report Type, 'N/A' as Patient Name, '2026-01-01 20:33' as Generated Date, 'N/A' as Date Range, and 'View' and 'Download' buttons as Actions. Figure 61: Test Case 06 Output.			
Evaluation	This test confirms the accuracy and usefulness of administrative reports.			

Table 11: Tetest Case 06.

6. Chapter VI – Future Work.

6.1. Introduction.

This chapter introduces the future improvement possibilities that can be incorporated for the advancement of the system. as for the current development of the project, it has already accommodated all the necessary requirements for its development. Unfortunately, because of the constraints of time and resources, some of the future concepts could not be included in the development of this project. This is the essence of this chapter, in which future possibilities will be introduced.

6.2. Future Works.

Mobile Application Development.

Presently, the system is an online platform. Future upgrades may involve applications for smartphones running on the Android and iOS operating system. this will give patients the flexibility to use their smartphones to book appointments, send notifications, view medical reports, among functions.

Online Consultation and Telemedicine Support.

To cater to this increasing need for telehealth services, certain features of telemedicine can be implemented in this system. the addition of a video conferencing function would enable doctors to conduct virtual sessions, which would benefit patients in distant and emergency circumstances.

Enhanced Appointment Scheduling with AI Assistance.

Future updates might bring features like the use of AI for scheduling appointments based on the doctor's availability, the urgency of the patient's needs, and past trends. Waiting times and resource allocation in the hospital can thus be optimized.

Interface with Pharmacy and Lab Systems.

The system can also be expanded to incorporate other modules, such as managing pharmacies and laboratories. Handling prescriptions and uploading results for lab tests would go a long way in enhancing cooperation and ease within hospital divisions.

Multilingual and Localization Capabilities.

Although the current system is able to perform the basic functionality, some future developments that could be introduced into the program include multilingual support, thereby benefiting patients who hail from varying linguistic backgrounds.

Cloud Deployment and Multi-Hospital Capability.

Future improvement can then be centered around aspects such as cloud technology, where multiple branches of a hospital can share the same computer system for security purposes.

6.3. Summary.

Concluding this chapter, the future enhancements or additions that can be included in the development of the system were addressed. The proposed enhancements covered in this chapter will serve as guidelines for future researchers and developers who wish to continue from the point where this project has left off.

7. Chapter VII – Conclusions.

7.1. Introduction.

This chapter summarizes the conclusion of the project. It gives an overview of the aims of the project, identifies the major achievements and provides a reflection of the entire project. This chapter also marks the final end of both the project writing and analysis.

7.2. Conclusion.

Development of the Online Hospital Management System was a challenging yet rewarding experience. The project has successfully overcome the shortcomings associated with manual hospital management as it provides a safe and efficient system for managing various operations related to patients, doctors and billing. The system has been able to provide role-based access.

In the course of developing the project, technical and real-world challenges were encountered. Some of these challenges include changes in the system requirements, the integration of various components in the system and time constraints. The solutions to these challenges have improved the knowledge of full-stack system development and the implementation of secure applications.

The system has immense potential for increased efficiency in hospitals, reducing the burden of administration and increasing the level of patient satisfaction. By computerizing the basic functionalities in the hospital, HealthCare Plus Hospital will achieve quicker and more trustworthy and transparent health care. In summary, the system has met its intended goals and serves as a good foundation for future developments.

7.3. Summary.

This chapter dealt with a summary of results and accomplishments of this project. The experiences and lessons acquired during this project will be discussed again in the next reflection chapter.

8. Chapter VIII – Reflection.

8.1. Introduction.

This chapter is a reflection on the learning experience that have been garnered throughout the process of developing the Online Hospital Management System. this has been done to assess the learnings that have been garnered.

8.2. Lessons Learned.

This project has given good experience with system development and has helped to improve analytical and problem-solving skills. It has given hands-on experience in analysis and design of systems, normalization of databases and web application development. apart from this, this project has helped to improve the documentation, presentation and time management skills.

One of the most essential lessons learned was the need to know the user requirements in depth before implementing. The assumptions initially developed in the early stages of the project had to be changed when feedback was obtained from the stakeholders.

This project has also made realize and appreciate the need for system security, data privacy and ethics in medical applications. The skills and experience acquired have motivated to learn more about new technologies and pursue a career in relation to information systems and software development.

8.3. Summary.

This chapter concluded the project by looking back on the skills developed and the challenges which were experienced throughout the duration of the project. This skill can serve as a platform for the project's future academic work.

References

- [1] SimplyBook.me, "Online booking system - SimplyBook.me," SimplyBook.me Ltd, 2025. [Online]. Available: <https://simplybook.me/>. [Accessed 5 December 2025].
- [2] SimplyBook.me, "Medical and Health Centre Booking software," SimplyBook.me Ltd, 2025. [Online]. Available: <https://simplybook.me/en/online-appointment-software-for-medical-companies-clinics-doctors-and-healthcare>. [Accessed 5 December 2025].
- [3] CIS WORLD, "Doctor Channeling Management System," CIS WORLD, 30 January 2024. [Online]. Available: <https://www.cisworld.lk/doctor-channeling-management-system>. [Accessed 5 December 2025].
- [4] Practo, "Practo - Book doctor appointments, teleconsultation and practice management," Practo Technologies Pvt. Ltd., 2025. [Online]. Available: <https://www.practo.com/>. [Accessed 5 December 2025].
- [5] OpenMRS, "OpenMRS - Open source medical record system," OpenMRS Inc., 2025. [Online]. Available: <https://openmrs.org/>. [Accessed 5 December 2025].
- [6] Bahmni, "Bahmni - Open source EMR & hospital information system," ThoughtWorks/Bahmni Community, 2025. [Online]. Available: <https://www.bahmni.org/>. [Accessed 5 December 2025].
- [7] TpointTech, "XAMPP TUTORIAL," JavaTPoint, 2023. [Online]. Available: <https://www.tpointtech.com/xampp>. [Accessed 8 December 2025].
- [8] Computer Hope, "Notepad++," Compute Hope, 2023. [Online]. Available: <https://www.computerhope.com/jargon/n/notepad-plus-plus.htm>. [Accessed 8 December 2025].
- [9] Microsoft, "Visual Studio Code - Code Editing," Microsoft, [Online]. Available: <https://code.visualstudio.com/>. [Accessed 8 December 2025].

Appendices.

Appendix A: Use Case Diagrams.

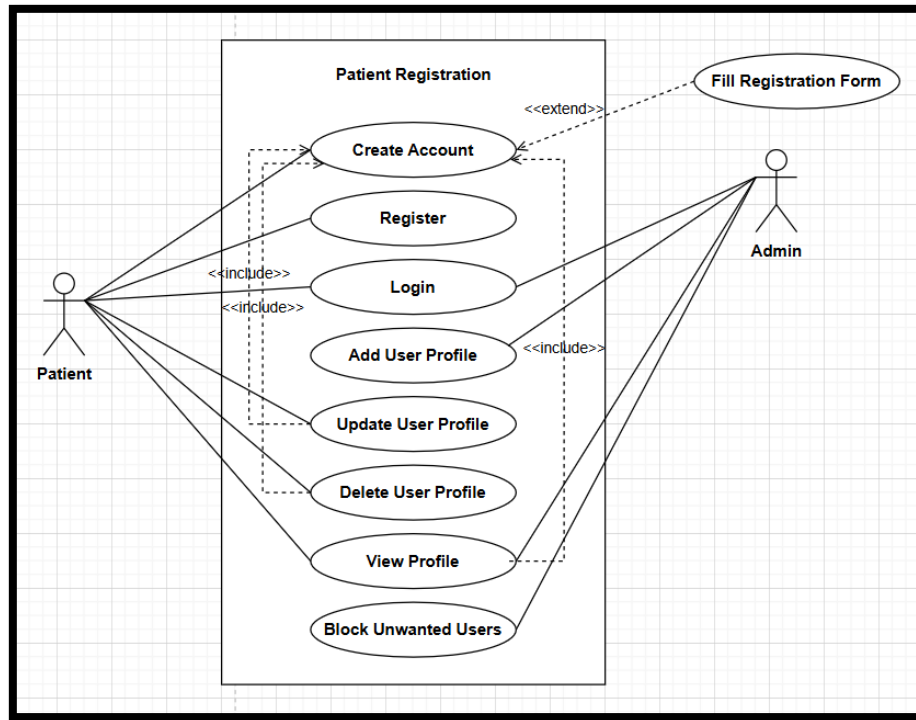


Figure 62: Use Case for Patient Registration.

Use Case for Booking Appointment.

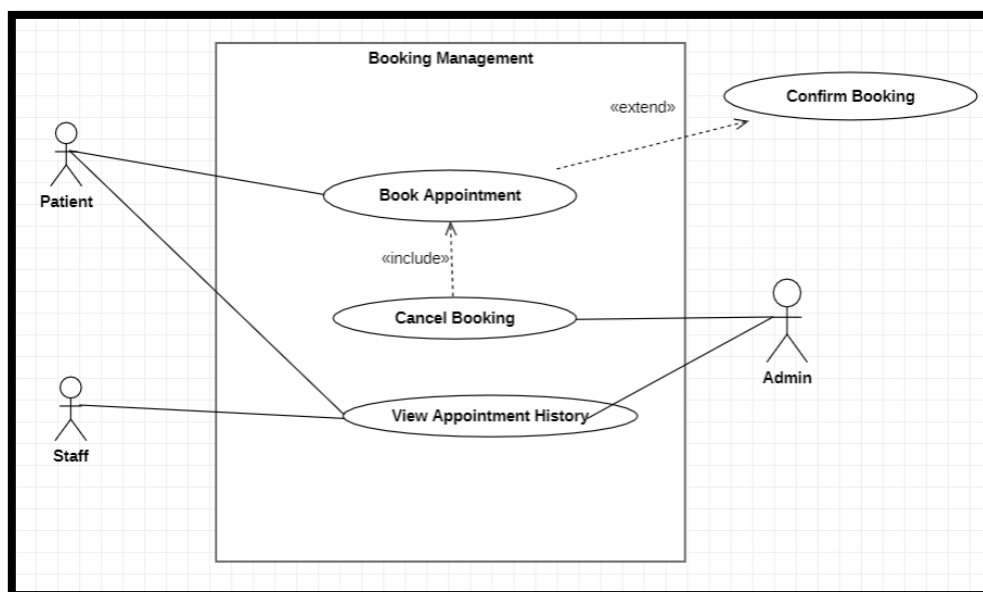


Figure 63: Use Case for Booking Appointment.

Use Case for Payment.

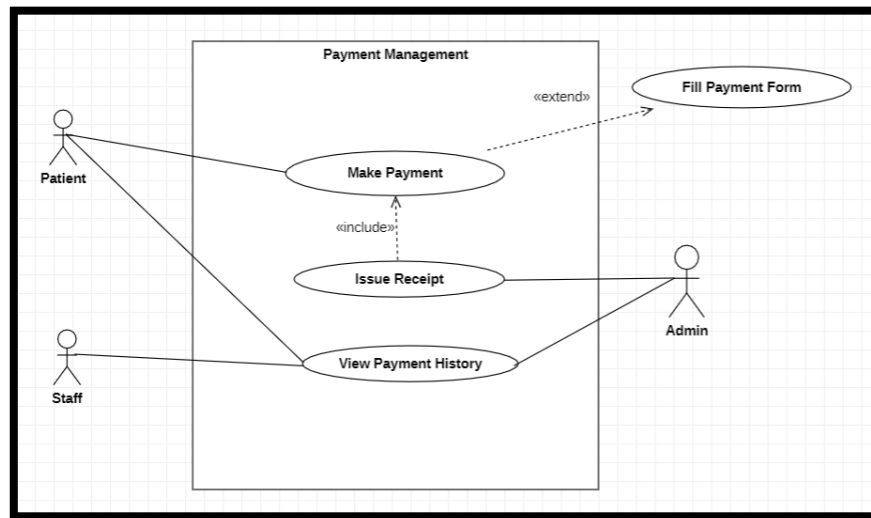


Figure 64: Use Case for Payment.

Use Case for Feedback and Notification.

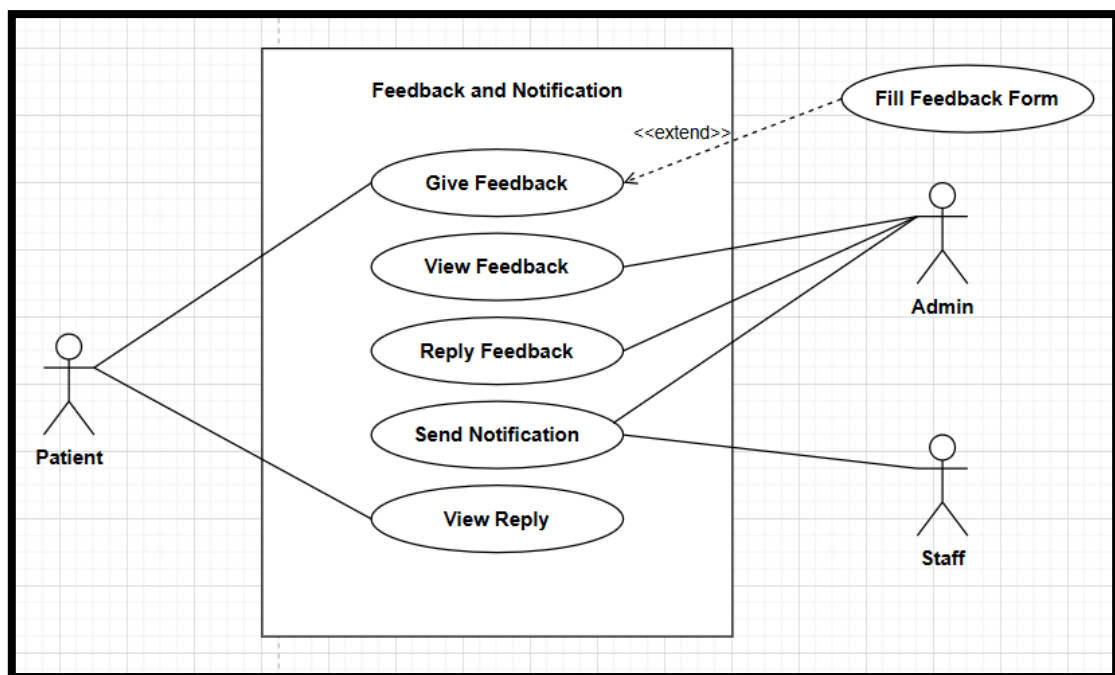


Figure 65: Use Case for Feedback and Notification.

Appendix B: Activity Diagrams.

This is the Activity Diagram of this system. That visually represents the process in the system. It shows the sequence of activities that a user follows to successfully make an appointment. This activity diagram clarifies the steps required for successful booking and helps developers and analysts what needs to be implemented in the system.

Activity Diagram for Checking Doctor Availability.

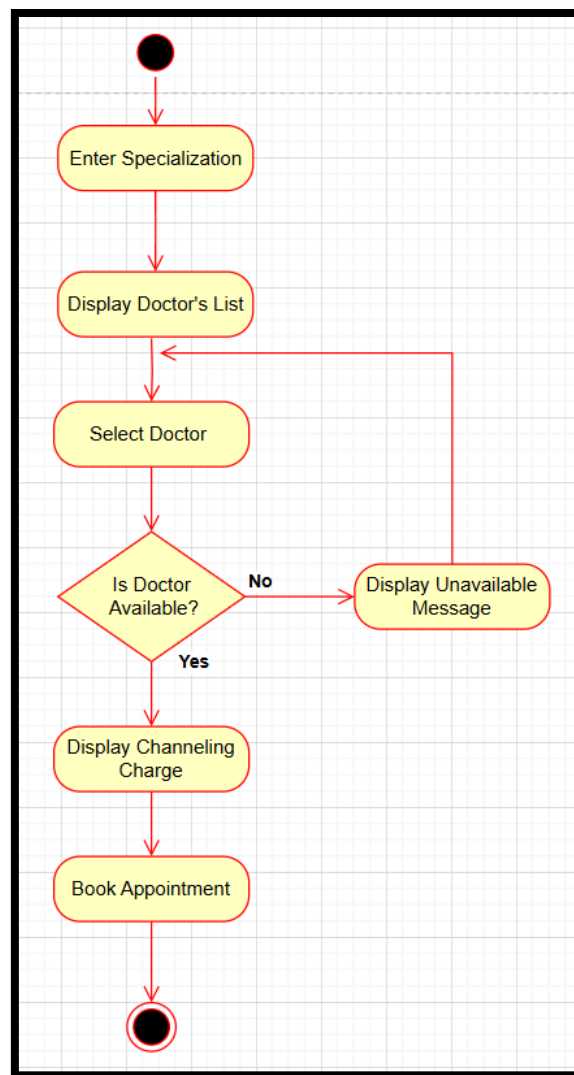


Figure 66: Activity Diagram for Checking Doctor Availability.

Activity Diagram for Payment.

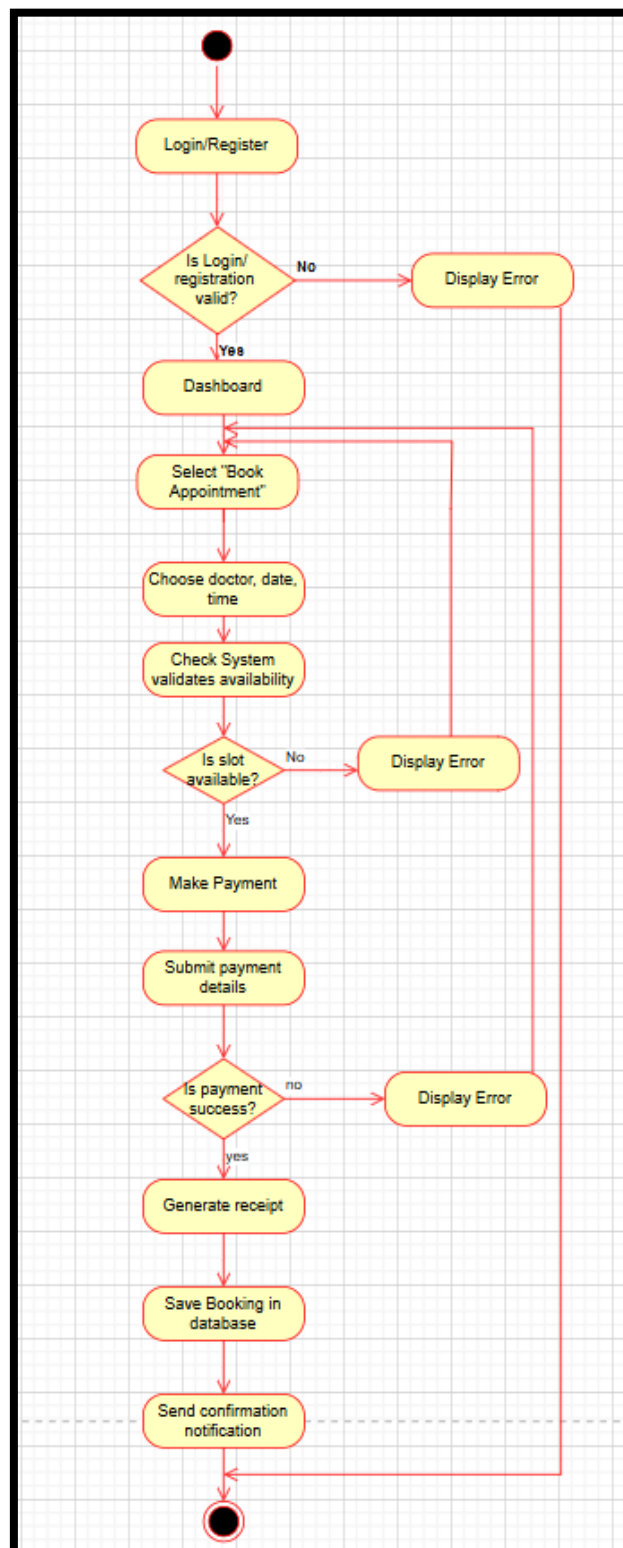


Figure 67: Activity Diagram for Payment.

Activity Diagram for Booking Appointment.

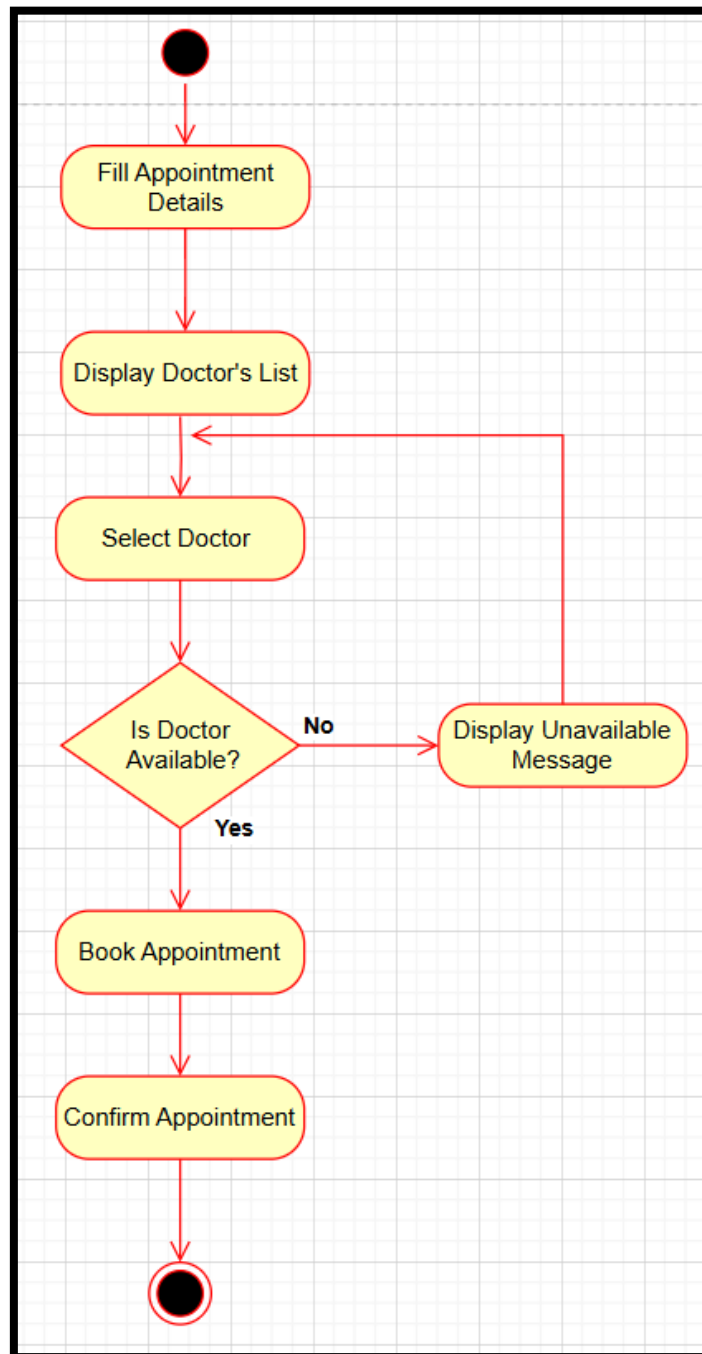


Figure 68: Activity Diagram for Booking Appointment.

Appendix C: Source Code for the System.

The complete source code for the HealthCare Plus Web-based Hospital Management System is maintained in a public GitHub repository for transparency and future enhancement purposes.

Repository Name: Health-Care-Plus

Repository URL: <https://github.com/dini99-edirisinghe/Health-Care-Plus>

Branch used for Final Submission: Main.

Technology Stack: HTML, CSS, JavaScript, PHP, MySQL

Appendix D: Software Requirement Specification (SRS).

Software Requirements Specification
For
Online Hospital Management System for
HealthCare Plus Hospital.

Version 1.0

Prepared by – Edirisinghe EMMGDB (E2240189)
Bachelor of Information Technology (External Degree)
Faculty of Information Technology
University of Moratuwa

Created: 2025.06.20

Acknowledgement.

This project named Online Hospital Management System for HealthCare Plus Hospital will be developed by Edirisinghe EMMGDB who is a student from University of Moratuwa in BIT 24S1 Batch. The developer gratefully thanking the people who helped and gave valuable ideas and advices to successfully complete this Software Requirements Specification document. Specially Miss. Thumashi De Silva as the supervisor who contributed her maximum effort to improve our knowledge and gave the clear and suitable ideas and advices to finish this assignment in a limited time.

Thank you,

Edirisinghe EMMGDB.

Table of Contents.

Acknowledgement	87
Table of Contents.....	88
List of Figures.....	89
List of Tables.....	90
1. Introduction.....	91
1.1 Purpose.....	91
1.2 Document Conventions.....	91
1.3 Intended Audience and Reading Suggestions.....	92
1.4 Project Scope.....	92
1.5 References.....	92
2. Overall Description.....	93
2.1 Product Perspective.....	93
2.2 Product Features.....	94
2.3 User Classes and Characteristics.....	95
2.4 Operating Environment.....	96
2.5 Design and Implementation Constraints.....	97
2.6 User Documentation.....	98
2.7 Assumptions and Dependencies.....	99
3. System Features.....	101
3.1 Patient Management.....	101
3.2 Staff Management.....	102
3.3 Appointment and Booking Management.....	103
3.4 Feedback Management.....	104
3.5 Report Management.....	105
4. External Interface Requirements.....	106
4.1 User Interfaces.....	106
4.2 Hardware Interfaces.....	111
4.3 Software Interfaces.....	112
4.4 Communications Interfaces.....	113
5. Other Nonfunctional Requirements.....	114
5.1 Performance Requirements.....	114
5.2 Safety Requirements.....	114
5.3 Security Requirements.....	115
5.4 Software Quality Attributes.....	115
6. Other Requirements.....	117
Appendix A: Glossary.....	119
Appendix B: Analysis Models.....	120
Appendix C: Issues List.....	127

List of Figures.

Figure 1: Block Diagram of Proposed System.	93
Figure 2: User Registration Page.	106
Figure 3: Login Page.....	106
Figure 4: Admin Dashboard.....	107
Figure 5: Staff Management.	107
Figure 6: Doctor Management.	108
Figure 7: Appointment Management.	108
Figure 8: Confirmed Appointments.	109
Figure 9: General Reports.....	109
Figure 10: Patient Management.	110
Figure 11: Patient Registration Form.....	110
Figure 12: Payment Receipt Form.	111
Figure 13: Enhanced Entity Relationship Diagram.	120
Figure 14: Class Diagram.	121
Figure 15: Use Case Diagram.	122
Figure 16: Use Case for Passenger Registration.....	123
Figure 17: Use Case for Booking Appointment.	123
Figure 18: Use Case for Payment.	124
Figure 19: Activity Diagram for Booking Appointment.	124
Figure 20: Activity Diagram for Payment.	125
Figure 21: Sequence Diagram for Booking Appointment.	126
Figure 22: Sequence Diagram for Payment.	126

List of Tables.

Table 1: Document Conventions.	91
Table 2: User Classes and Characteristics.	95
Table 3: Operating Environment.	96
Table 4: Design and Implementation Constraints.....	97
Table 5: User Manuals.	99
Table 6: Technical Capabilities.	99
Table 7: Hardware Interfaces.....	112
Table 8: Software Interfaces.	113

1. Introduction.

This chapter provides an overview of the Hospital Management System by outlining the purpose, scope and context of the software requirements. It introduces key terms, acronyms and abbreviations used throughout the document to ensure a common understanding among all stakeholders.

1.1 Purpose.

The purpose of this Software Requirement Specification (SRS) document is to provide a detailed description of the Online Hospital Management System to be developed. This document is intended to outline the system's functionality, performance and design requirements, serving as a guide for developers, testers and stakeholders throughout the development process. The goal of the Online Hospital Management System is to enable users to manage patient records, schedule appointments, maintain staff information, handle billing processes and access medical reports through an integrated online platform, ensuring efficient, secure and user-friendly experience.

1.2 Document Conventions.

In general, this document follows the IEEE formatting requirements. This document contains Times New Roman template font size throughout. The author has used underlines for title headings. The document text is 1.5 lines spaced and sub titles have used Heading 2 with bold font style and normal texts have used Heading 3 with 12 font size.

Software	Microsoft Word
Font	Times New Roman
Heading font size	18pt
Sub heading font size	14pt
Content font size	12pt

Table 12: Document Conventions.

1.3 Intended Audience and Reading Suggestions.

The project supervisor, the evaluators and all interested parties, and me as the project developer are the intended audience of this document.

1.4 Project Scope.

The Online Hospital Management System is developed to automate and simplify hospital operations by granting single-window access to patient care and administration process management. The system allows users such as patients, doctors, hospital staff and admin to view and manage health services effectively. Key functionality includes, patient registration, appointment scheduling, electronic medical records management, payment and billing and allocation of hospital resources. The system will also support to feature features like doctor and department management, prescription processing and reporting tools to analysis and monitor. It will be accessible using both web and mobile interfaces to enable broad accessibility over all possible devices.

1.5 References.

- [1] IEEE Software Engineering Standards Committee, "IEEE Recommended Practice for Software Requirements Specifications.," *IEEE Standards*, Vols. IEEE Std 830-1998, 1998.
- [2] J. Karlsson, "A Cost-Value Approach for Prioritizing Requirements," *Norges Teknisk Naturvitenskapelige University*, 1997.
- [3] GeeksforGeeks, "Software Requirements Specification (SRS) format," Sanchhaya Education Private Limited, 20 September 2023. [Online]. Available: <https://www.geeksforgeeks.org/software-requirement-specification-srs-format/>. [Accessed 01 September 2024].

2. Overall Description.

2.1 Product Perspective.

This Hospital Management System has a backend and a frontend. This system is self-contained system that manages activities of the hospital as Patients Info. Various stakeholders are involved in the Hospital Management System. This system is independent as both back-end and front-end are developed by the developer. The patient is the person who handles the front-end. All patients' activities are conducted in the front-end. Patients can visit the website, booking appointments, view and download reports, give feedback and view doctor's availability list through the front-end. Moreover, patients can see notifications sent from the back-end.

Admin handles the back-end. All management activities are contributed in the back-end. This includes report generation, staff member registration and booking handling. Moreover, staff members can view some front-end activities through the back-end that patients given to process, such as view bookings and view patient details. Both the back-end and front-end are connected to the database. All the data is passes to the database. Here is the block diagram of the proposed system.

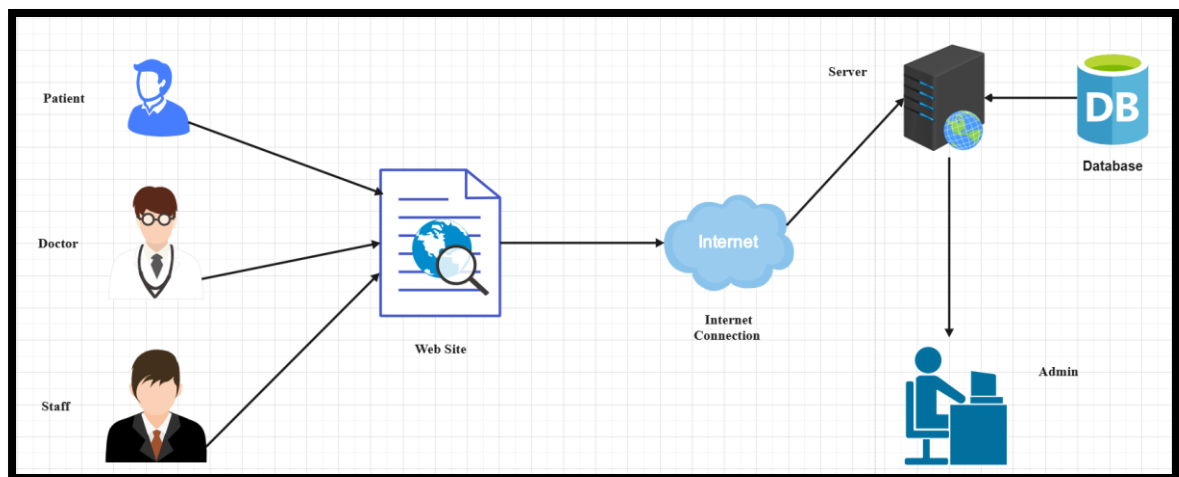


Figure 69: Block Diagram of Proposed System.

2.2 Product Features.

- Patient Management.
 - Patient can create a user account.
 - Registered patients can login to the system using their email and password.
 - Admin can block unnecessary patients.
 - Patient can view, update and delete their profile.
- Appointment Scheduling.
 - Admin can handle booking details.
 - Patient can view booking details.
 - Patient can select doctors.
 - Patient can search doctor availability.
 - Patient can cancel the appointment booking.
 - Admin can manage appointment schedule and time table.
- Billing and Payment.
 - Patient can make payments.
 - Admin can issue the receipt.
 - Patient can collect their receipt after the payment.
- Medical Record and Report Management.
 - Doctor can view medical reports.
 - Admin can generate reports.

- Patients can view annual reports.
- AI-Powered Symptom Checker.
 - Allows patients to input symptoms and receive preliminary guidance on possible condition before scheduling an appointment.
- Staff and Resource Management.
 - Staff members can login to the system.
 - Admin can add, update and delete staff members.
 - Admin can provide relevant user privileges.
 - Admin can provide user name and password.
- Feedback and Support.
 - Patients can give feedbacks by filling the feedback form.
 - Admin can reply to feedbacks.
 - Staff can view feedbacks

2.3 User Classes and Characteristics.

User Class	Education	Technical Skill	Experience
Admin	Medium	High	1 year
Staff	Low	Medium	None
Patient	Low	Low	None
Doctor	High	Low	Varied

Table 13: User Classes and Characteristics.

2.4 Operating Environment.

Component	Details
Client Hardware	• Laptop/Desktop (Minimum 4GB RAM, 1GB HDD) • Display: Minimum 1024×680, Recommended 1920×1080 • Minimum Intel Core i3
Server Hardware	• Local or LAN server. • Minimum Quad-core CPU • 8GB RAM • 100GB HDD
Client Operating System	• Windows 10 or above, Linux, MacOS.
Web Browser	• Chrome, Microsoft Edge.
Web Server	• XAMPP (Apache, PHP, MySQL)
Compatible Apps	• Microsoft Office • Antivirus Software • Backup Tools • Printer Drivers

Table 14: Operating Environment.

2.5 Design and Implementation Constraints.

Technology and Tools	Frontend	HTML, CSS, JavaScript
	Backend	PHP with MySQL
	Database	MySQL
	Web Server	Apache (through XAMPP)
Development Methodology	Agile Methodology	
Platform Constraints	Devices	Desktop and laptop.
	Operating System	Windows 10 and above, modern Linux distributions (Ubuntu 20.04 or higher).
	Browser	Google Chrome, Microsoft Edge
	Network	Localhost environment with IP 127.0.0.1
Regulatory and Privacy Policies	- Local healthcare data privacy laws (patient confidentiality, secure access control). - Not accessible by unauthorized users.	
Communication and Network Protocol Constraints	- HTTP protocol - The network is encrypted. - Minimum bandwidth of 5 Mbps.	

Table 15: Design and Implementation Constraints.

2.6 User Documentation.

User	Manuals
Admin	• System overview. • User access control. • Managing staff, doctors and appointments. • Generating and exporting reports. • Handling feedback and communication.
Staff	• Logging in and managing profile. • Viewing and confirming appointments. • Handling patient feedback. • Managing payment history.
Doctor	• Accessing doctor schedule. • Viewing appointments. • Updating availability.
Patient	• Account registration and login. • Booking and cancelling appointments. • Making payments. • Providing feedback. • Downloading appointment receipts.

Table 16: User Manuals.

Role-specific manuals will be provided in PDF format and made available through the system's interface or internal hospital network.

Online Help and Tooltips.	• Frequently Asked Questions (FAQs). • Step-by-step articles for common tasks. • Tooltips on buttons and form fields for guidance. • Troubleshooting suggestions for common user issues.
Video Tutorials.	• How to register as a new user. • How to book or cancel appointments. • How to make a secure payment. • How to use the admin dashboard.
In-System Notifications and Guides.	• This system will provide real-time success and error notifications ("Appointment booked successfully").
Delivery Formats.	• PDF Manuals. • HTML Help Pages. • Video Files.

Table 17: Technical Capabilities.

2.7 Assumptions and Dependencies.

- Assumptions.
 - The users (patients, staff, doctors and administrators) will have access to a stable internet connection (minimum 5Mbps bandwidth).

- The system will primarily be accessed via desktop or laptop computers running Windows, Linux or Mac operating systems.
 - All users will have basic knowledge of operating a web-based system, including navigating web pages and filling out forms.
 - Users will have modern web browsers (Google Chrome or Microsoft Edge) that support the technologies used (HTML, CSS, JavaScript).
 - Security protocols such as HTTP and password encryption will be implemented by developers as part of best practices.
 - The system will run on localhost during development and on a local server or intranet upon initial deployment.
 - MySQL will be the database used, and data integrity will be maintained through properly designed relational schemas.
- Dependencies.
 - The hospital management system is dependent on the availability and correct configuration of XAMPP or equivalent web server with support for PHP and MySQL.
 - The system requires a stable local network or internet connectivity for real-time features such as appointment booking and feedback submission.
 - Online payment gateway integration (PayPal) will require third-party APIs and may involve compliance with PCI DSS standards.
 - The reliability of scheduled tasks (Report generation) depends on the correct configuration of server-side cron jobs or scheduling scripts.

- **System Features.**

- **Patient Management.**

- Description and Priority.

- This feature allows patients to register, authenticate, manage their account and interact with the hospital system.

- Priority: High

- Stimulus/Response Sequences.

- Patient selects “Create Account” → System prompts for patient details → Patient submit details → System confirms account creation.

- Patient enters username/password → System verifies credentials → Patient dashboard is displayed.

- Patient requests to update/delete account → System fetches details → Changes saved/account removed.

- Admin selects patient from list → System displays patient info → Admin blocks patient → System prevents login access.

- Patient clicks “Logout” → System ends session and redirects to login page.

- Functional Requirements.

- REQ-01: The system allows to patients for create a new account entering patient details such as name, address, contact number, username and password.

- REQ-02: The system allows authenticate patient to login their username and password to the system.

- REQ-03: The system allows to add, update and delete patients.

- REQ-04: The system allows to logout from the system.

- **Staff Management.**

- Description and Priority.

- This feature allows admin to manage hospital staff accounts including registration, credential assignment and access privileges.
- Priority: High

- Stimulus/Response Sequences.

- Admin selects "Register Staff" → Enters staff details → System confirms account creation.
- Admin accesses staff list → Edits or deletes selected account → System saves or removes data.
- Admin assigns username/password → System updates account credentials.
- Staff logs in using credentials → Dashboard is displayed.
- Admin sets user roles → System saves assigned privileges.

- Functional Requirements.

- REQ-05: The system shall allow the admin to register staff members with name, contact, address, etc.
- REQ-06: The system shall allow the admin to add, update, or delete staff records.
- REQ-07: The system shall allow the admin to provide usernames and passwords to staff.
- REQ-08: The system shall allow staff to log in with assigned credentials.
- REQ-09: The system shall allow the admin to assign access privileges to each staff account.

○ **Appointment and Booking Management.**

▪ Description and Priority.

- This feature manages appointment scheduling between patients and doctors, booking confirmations, payment processing, receipt generation and cancellations.
- Priority: High

▪ Stimulus/Response Sequences.

- Patient logs in → Views available slots/doctors → Selects appointment → Confirms booking.
- System confirms availability → Accepts payment → Generates appointment ticket.
- Patient views/cancels upcoming bookings → System processes request.
- Admin confirms or cancels bookings → System updates records.

▪ Functional Requirements.

- REQ-10: The system shall allow the admin to manage all booking records.
- REQ-11: The system shall allow patients to view their own appointment bookings.
- REQ-12: The system shall allow patients to select preferred doctor/specialization and date/time.
- REQ-13: The system shall allow patients to make online payments for appointments.
- REQ-14: The system shall allow patients to check appointment slot availability.
- REQ-15: The system shall allow the admin to issue appointment confirmation tickets.

- REQ-16: The system shall allow patients to view/print ticket after successful booking.
- REQ-17: The system shall allow patients to cancel their booked appointments.

○ **Feedback Management.**

- Description and Priority.
 - This feature handles patient feedback submissions, staff views of feedback and admin replies to feedback messages.
 - Priority: Medium
- Stimulus/Response Sequences.
 - Patient submits feedback → System stores and notifies relevant staff.
 - Staff views feedback → Takes note of patient comments.
 - Admin selects a feedback entry → Sends response → System notifies patient.

3.4.3 Functional Requirements.

- REQ-18: The system will allow the admin to reply to feedback messages.
- REQ-19: The system will allow staff to view feedback received.
- REQ-20: The system will allow patients to submit feedback through their account.

- **Report Management.**

- Description and Priority.

- This feature generates reports on bookings, patient activities, staff operations and financial records.

- Priority: Medium

- Stimulus/Response Sequences.

- Admin selects report type → System generates report → Admin views or exports report.

- Staff logs in → Views department-specific reports.

- Patient logs in → Views personal appointment or payment reports.

- Functional Requirements.

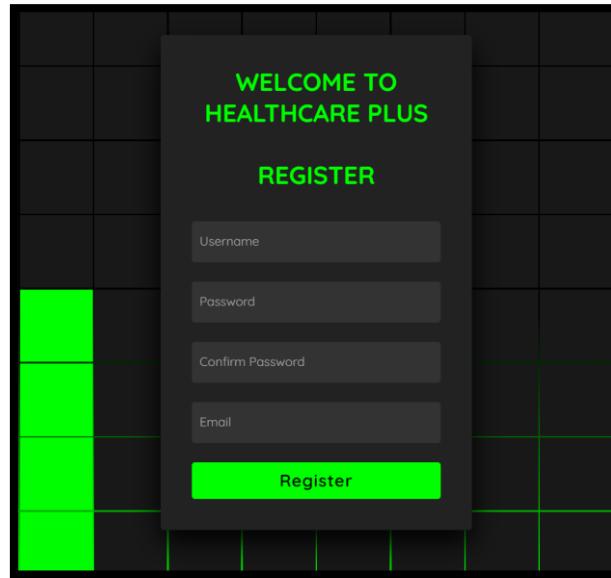
- REQ-21: The system will allow the admin to generate system-wide reports.

- REQ-22: The system will allow patients to view their booking/payment reports.

- REQ-23: The system will allow staff to view relevant operational reports.

External Interface Requirements.

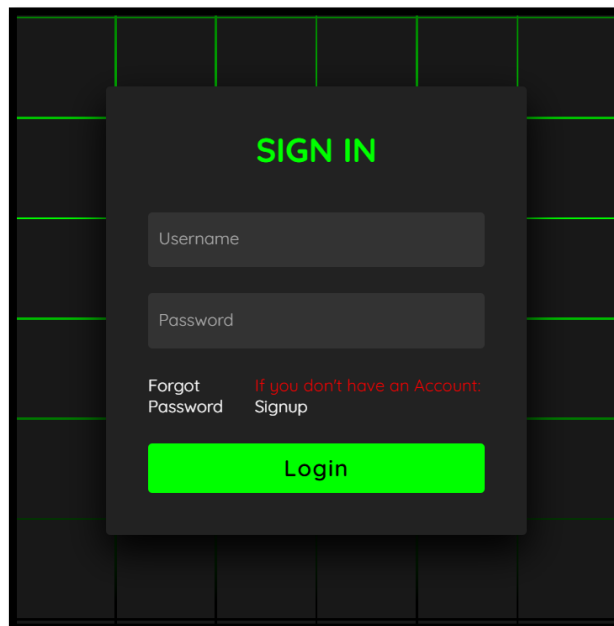
- **User Interfaces.**
- User Registration Page.



A dark-themed user registration form titled "WELCOME TO HEALTHCARE PLUS" and "REGISTER". The form includes input fields for Username, Password, Confirm Password, and Email, followed by a red "Register" button. The background features a dark grid with a vertical red bar on the left.

Figure 70: User Registration Page.

- Login Page.



A dark-themed login form titled "SIGN IN". It includes input fields for Username and Password, a "Forgot Password" link, a "If you don't have an Account: Signup" link, and a red "Login" button. The background features a dark grid.

Figure 71: Login Page.

This is the Login page of the system. There are four categories in this system to login. They are Admin, Patient, Doctor and Staff. These levels should enter username and password to login to the system. They would be wrong and if the any fields would be empty or incorrect when click the Login button will throw errors.

- Admin Dashboard.

If Admin login to the system successfully, Admin can show the Admin Dashboard. Admin can see the Total active staff, Total active consultants, Total categories, Total appointments and Total confirmed visits.

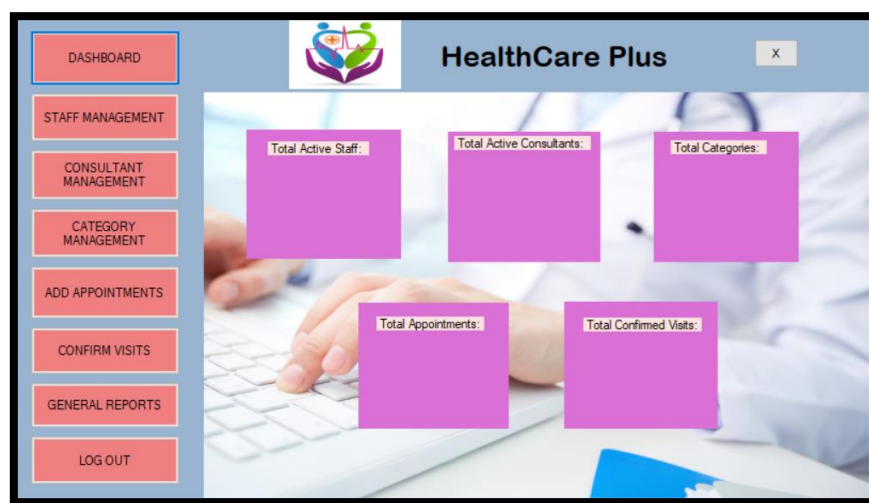


Figure 72: Admin Dashboard.

- Staff Management.

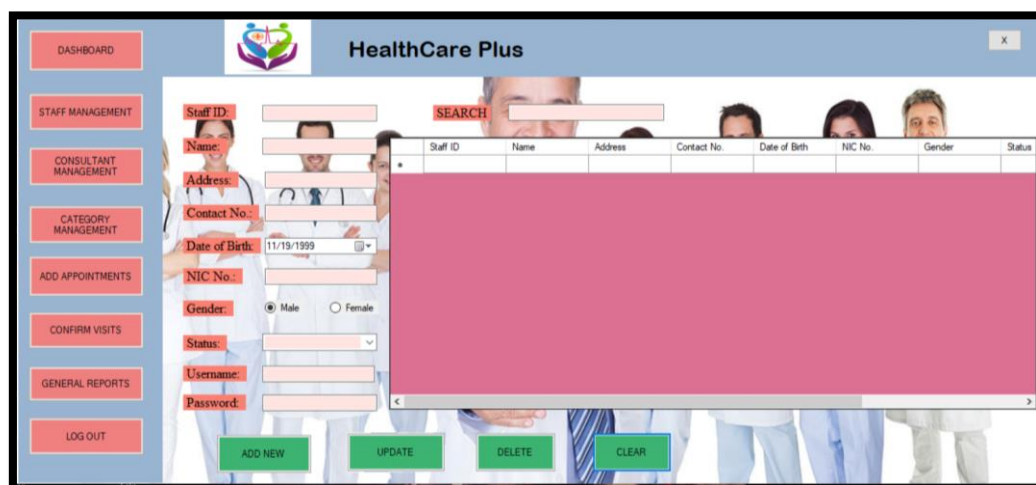


Figure 73: Staff Management.

In this page, Admin can add, update and delete staff. Admin should fill this to add, update or delete staff member. After add, update or delete the details, the results will be shown in the side of the web page. Then admin can search staff members in the search bar.

- Doctor Management.

Figure 74: Doctor Management.

In this page, Admin can add, update and delete doctor. Admin should fill doctor details to add, update or delete a doctor. After add, update or delete the details, the results will be shown in the side view of the page. Then admin can search a doctor in the search bar.

- Appointments Management.

Figure 75: Appointment Management.

In this page, Admin can add, update and delete appointments. Admin should fill the appointment details to add, update or delete. After add, delete or update the details, the results will be shown in the side view of the page. Then admin can search appointments in the search bar.

- Confirmed Appointments.

Figure 76: Confirmed Appointments.

In this page, Admin can add, update and delete confirmed appointments. Admin should fill confirmed appointment details to add, update or delete. After add, update or delete the details, the results will be shown in the side view of the page. Then admin can search confirmed appointments in the search bar.

- General Reports.

Figure 77: General Reports.

In this page, Admin can see the general reports. Admin can see Counts of the most appointments, least appointments, total consultant payments, consultant with the highest revenue and the least revenue and total profit of the consultants. The data will be shown in the web page. Admin can search data in the search bar.

- Patient Management.

Patient ID	Patient Name	Age	Contact No	Gender	Date	User
1	H.M. Herath	65	0719632587	Male	6/5/2023	herath
2	Lakmal	25	0773695524	Male	6/15/2023	lakma
3	Sachini	23	0741478523	Female	6/20/2023	sachin
4	Hirusha	32	0759632587	Male	7/10/2023	hirush
5	Nethmi	18	0765236958	Female	7/20/2023	nethmi
6	Shan	16	0732567410	Male	10/18/2023	shan
7	Chanakia	26	0783695521	Male	9/20/2023	chanak

Figure 78: Patient Management.

In this page, Admin can add a new patient, update or delete an existing patient that registered to the system. Admin should fill patient details to the system. After fill the details Admin should click the ADD button to enter the details to the system.

- Patient Form.

Figure 79: Patient Registration Form.

In this page, Patient can register to the system entering their details. After fill this form Patient should click the register button to register as a new patient. Patient can see their profiles after they registered.

- Payment Receipt Form.

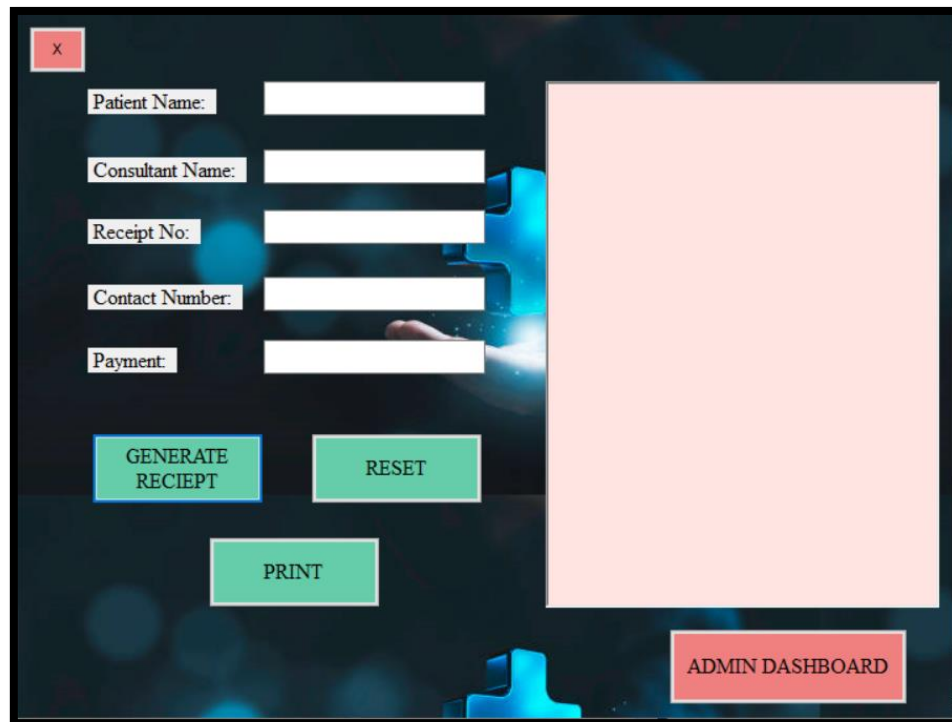
The screenshot shows a web application window titled 'Payment Receipt Form'. It features a dark blue background with a futuristic, glowing blue robot head on the right side. The form is located on the left and consists of several input fields: 'Patient Name:', 'Consultant Name:', 'Receipt No:', 'Contact Number:', and 'Payment:'. Below these fields are three buttons: 'GENERATE RECEIPT' (green), 'RESET' (green), and 'PRINT' (green). A large, empty light pink rectangular area is positioned to the right of the input fields. At the bottom right of the window, there is a red button labeled 'ADMIN DASHBOARD'. A small red 'X' button is in the top left corner of the window.

Figure 80: Payment Receipt Form.

In this page, Admin can issue a payment receipt to the patients. Admin should fill this page to issue the receipt. After fill the details Admin should click on the Generate Receipt button. Then Admin can print the payment receipt and give it to the patient. Patient can take that payment receipt and do cash payments to the hospital.

○ **Hardware Interfaces.**

This system needs a computer or a laptop. If the users use a computer, the users should use a properly working key board and a mouse as input devices. A minimum requirement for the processor should be 11th Gen Intel(R) Core (TM) 2Duo 3.00GHz. There can be used maximum 11th Gen Intel(R) Core (TM) i7-1165G7 @ 2.80GHz generation processor.

Further, there should be a minimum 1GB space from the hard disk to run this system. the primary memory (RAM) should have a minimum of 4GB. There can be used maximum 16GB RAM to run without interruption.

There should be a minimum 1024×680 display size of monitor to view this system. there can be use monitors with above this displays size like 1920×1080 . Moreover, the screen resolution should be at least 1024×768 . There can be use upper screen resolution monitor to use this system.

Component	Recommended
Device	Laptop
Processor	11th Gen Intel(R) Core (TM) i7-1165G7 @ 2.80GHz 2.80 GHz
Application Window Size	1920×1080
Screen Resolution	1920×1080
Hard Disk	512 GB
RAM	8 GB
System	64-bit operating system, x64-based processor

Table 18: Hardware Interfaces.

○ **Software Interfaces.**

Windows 8 operating system can be used as a minimum requirement in order to run this system. The latest version of windows can be used for this system as well. This system is not supported for the mobile devices.

XAMPP server is used as the web server as this system run in local host and is accepted in both Windows and Linux. MySQL is used as a database in order to store data. XAMPP – SQL version 2021 – v3.3.0 is used as the minimum requirement.

PHP is the server-side scripting language used in this system. HTML version 05, CSS version 03 and JavaScript are used as the client-side scripting languages.

Microsoft Visual Studio Code is used as the source code editor tool to develop the system. this system will support for Google Chrome and Microsoft Edge web browsers.

Operating System	Windows 10
Web Server	XAMPP – SQL Server 2021 – v3.3.0
Web Browser	Chrome and Microsoft Edge
Code Editor	Microsoft Visual Studio Code, Notepad++

Table 19: Software Interfaces.

○ **Communications Interfaces.**

This system is run in local host. IP address of the local host is 127.0.0.1. This system needs Internet to run. Users can use private or public network. The Internet connection should be in a proper bandwidth with a minimum 5Mbps. The router capacity should be 2.4GHz with 50Mbps sufficient to run the system. users can use any type of network like 4G or 5G.

Other Nonfunctional Requirements.

○ Performance Requirements.

- The system shall support at least 100 concurrent users without performance degradation.
- The response time for all user interactions (login, booking appointments, submitting payments) shall not exceed 3 seconds under normal operating conditions.
- The system shall handle up to 500 appointment booking per day and maintain optimal speed.
- The database should be optimized to retrieve and update patient records within 2 seconds for queries involving fewer than 1000 records.
- Scheduled tasks, such as daily backup or report generation, shall complete within 5 minutes during off-peak hours.

○ Safety Requirements.

- The system must ensure data is not lost due to unexpected power failure by implementing regular automatic database backups.
- The application shall include form validation and input sanitization to prevent erroneous data entry (invalid dates, empty mandatory fields).
- Only authorized personnel (admin role) shall be allowed to perform data-critical operations, such as staff deletion, database access, and system configuration.
- In case of system failure, the system shall provide a clear error message and automatic log generation for diagnostic purposes.
- There shall be limited access to sensitive areas (billing, reports, medical history) to prevent accidental misuse or data corruption

- **Security Requirements.**

- All users must authenticate using a username and password, and passwords must be stored securely using hashing techniques.
- The system shall enforce session timeouts after 15 minutes of inactivity to prevent unauthorized access.
- Admin will have the ability to assign and revoke user roles and permissions to control access levels.
- The application must prevent common web vulnerabilities, including:
 - SQL Injection
- All data transfers involving login, payments, or sensitive personal information must be secured via HTTPS encryption.
- Audit logs must be maintained for key user actions (login, logout, profile update, record deletion) for accountability.

- **Software Quality Attributes.**

- Usability:
 - The system shall provide a clean and intuitive user interface, even for non-technical users (patients and staff).
 - All primary actions (registration, login, booking) shall be completable in under 5 clicks.
- Reliability:
 - The system shall maintain 99.5% uptime during operational hours.
 - In case of failure, it must restore from the last backup with minimal data loss (maximum 12 hours' worth).

- Maintainability:
 - The code shall follow modular design principles, enabling updates to features (e.g., feedback, payments) without affecting unrelated modules.
 - Proper documentation and comments must be included in source code.
- Portability:
 - The application shall be deployable on both Windows and Linux servers with minimal reconfiguration.
- Scalability:
 - The system shall be designed to support future integration with APIs (e.g., payment gateways, medical records) and to scale to more hospitals if needed.
- Interoperability:
 - The system shall support integration with third-party tools such as email systems, SMS APIs, or external report generators if added later.

Other Requirements.

- Database:
 - The system will use MySQL as the database management system.
 - The database must follow a relational schema with primary and foreign keys properly defined.
 - All sensitive data fields (password, payment info) will be encrypted or hashed.
- Internationalization:
 - This system will be developed in English (default language).
- Legal and Regulatory:
 - This system must comply with local data protection regulations applicable to healthcare institutions (patient data confidentiality, access control).
 - This system will include user consent agreements before registration or data submission, in accordance with privacy laws.
- Reusability:
 - Using modular and component-based architecture including Login/authentication modules, Appointment scheduling logic and Payment handling.
- Accessibility:
 - This system interface follows web accessibility best practices where feasible.
 - Use of proper font sizes, keyboard navigation support and alternative text for key icons and images are included.
- Data Migration and Import:

- The system shall provide a data import utility (CSV upload).
 - There are validations in this system to ensure data integrity.
- Audit Trail and Logging:
 - This system will record critical user actions, including login/logout, appointment creation, data edits and deletion.
 - Audit logs are username, timestamp, action performed, affected entity (Patient ID, Appointment ID).

Appendix A: Glossary.

- Definitions.
 - MySQL – Relational Database Management System using in the system.
 - Web-based Application – An application program that is stored on a remote server and delivered over the internet through a browser interface.
 - Registered Patient – A person who registered to the system to book an appointment.
- Acronyms and Abbreviations.
 - SRS – Software Requirements Specification.
 - HTML – Hyper Text Markup Language.
 - CSS – Cascading Style Sheets.
 - PHP – Hypertext Preprocessor.
 - Mbps – Megabits per second.
 - EER – Enhanced Entity Relationship Diagram.
 - GHz – Gigahertz.
 - GB – Giga Byte.
 - IP – Internet Protocol.

Appendix B: Analysis Models.

- Enhanced Entity Relationship Diagram (EER).

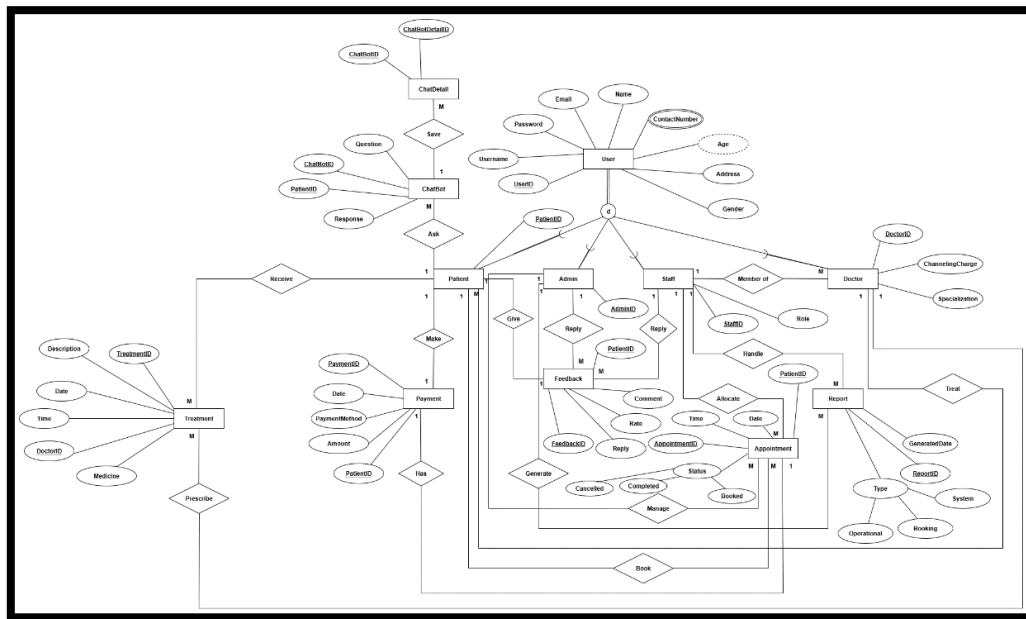


Figure 81: Enhanced Entity Relationship Diagram.

- Relational Schema for Online Hospital Management System based on the Enhanced Entity Relationship Diagram.
 - Patient (PatientID, Name, ContactNumber, Address, Age, AppointmentID*, Report ID*)
 - Appointment (AppointmentID, Date, Time, Status)
 - Doctor (DoctorID, Name, Category, ChannelingCharge, Specialization, Username, Password, Email, ContactNumber, Gender)
 - Admin (AdminID, ContactNumber, Username, Password, Address, Email, Age, Gender, Name)
 - Payment (Payment ID, Amount, Date, PaymentMethod, PatientID*)
 - Staff (Staff ID, Name, Address, ContactNumber, Username, Password, Email, Age)

- Report (ReportID, GeneratedDate, Type)
 - Treatment (TreatmentID, Medicine, Date, Description, Time, DoctorID*)
 - ChatBot (ChatBotID, Question, Response, PatientID*)
 - ChatDetail (ChatDetailID, ChatBotID*)
 - Feedback (FeedbackID, Comment, Rate, Reply, PatientID*)
- Class Diagram.

Each class is represented in this HealthCare Plus System class diagram by a box with eight sections stacked on top of one another, each component including the class name, properties and actions. Each class in this diagram contains a number of properties and operations with various goals. Every class displays the symbol before the attribute and method names to denote whether they are private (-) or public (+). The patient class indicate association relationship among payment, report and appointment. Here, generally patient has to take an appointment their doctor who can give consultation for them so the relationship is book. As a result, the staff can schedule appointments. Patient, Payment, Report, Appointment, Staff, Doctor, and Admin are the classes in this diagram.

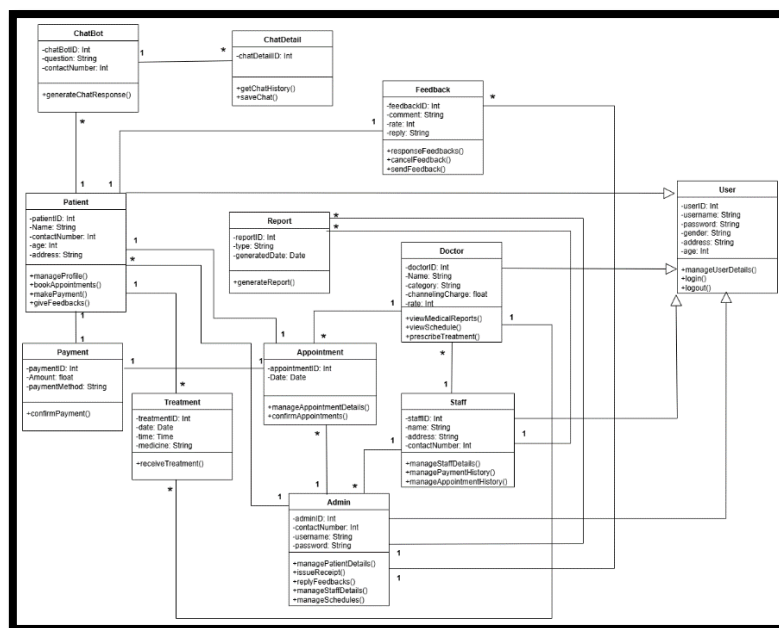


Figure 82: Class Diagram.

- Use Case Diagram.

The developer created the below use case diagram to show the actors and how they impact with the hospital system. There are four actors in this diagram. They are Patient, Staff, Admin and Doctor. In this case admin can add, update and delete patients, add, update and delete doctors, and confirm appointments.

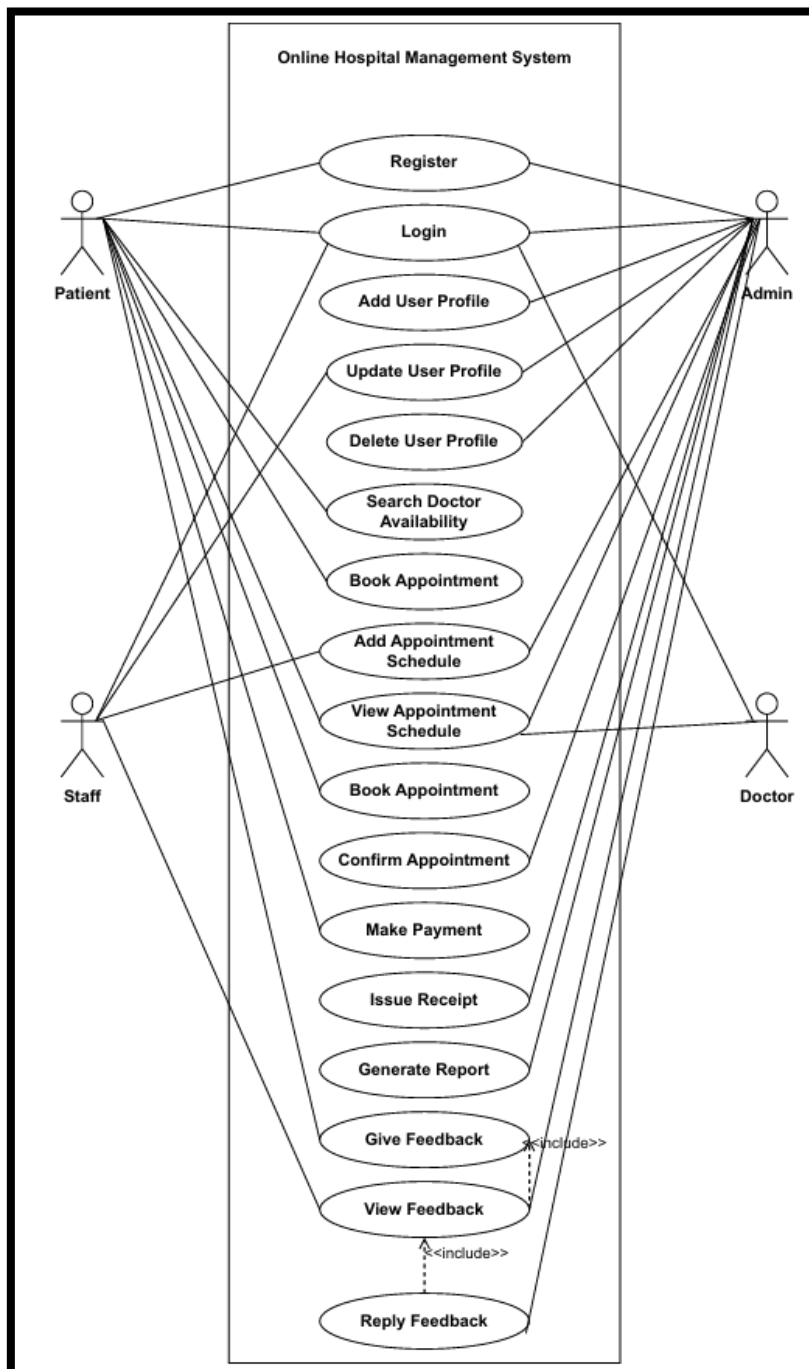


Figure 83: Use Case Diagram.

- Use case Diagram for Patient Registration.

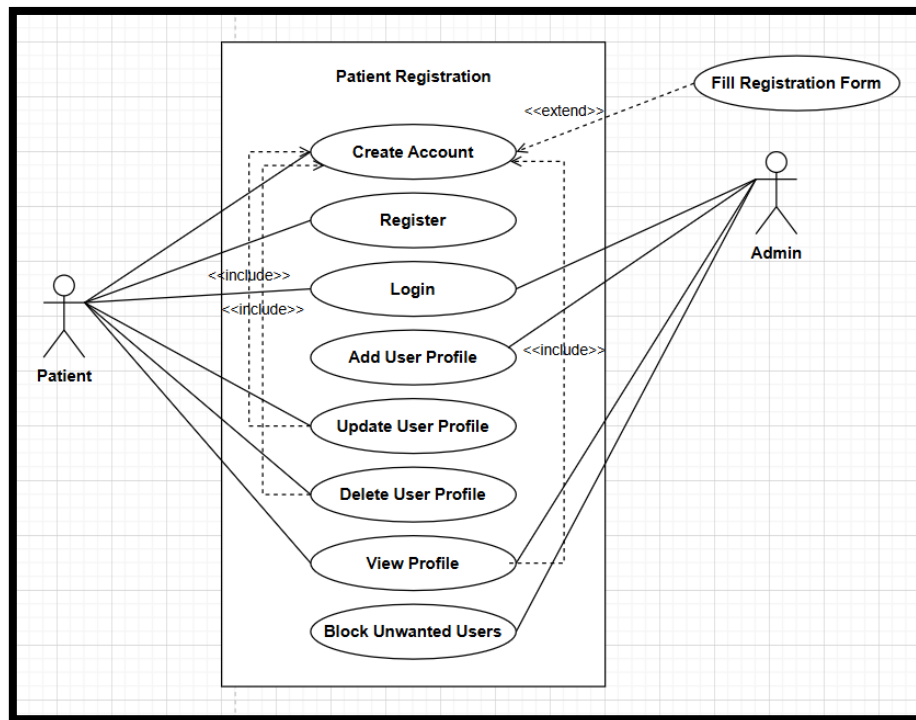


Figure 84: Use Case for Patient Registration.

- Use Case for Booking Appointment.

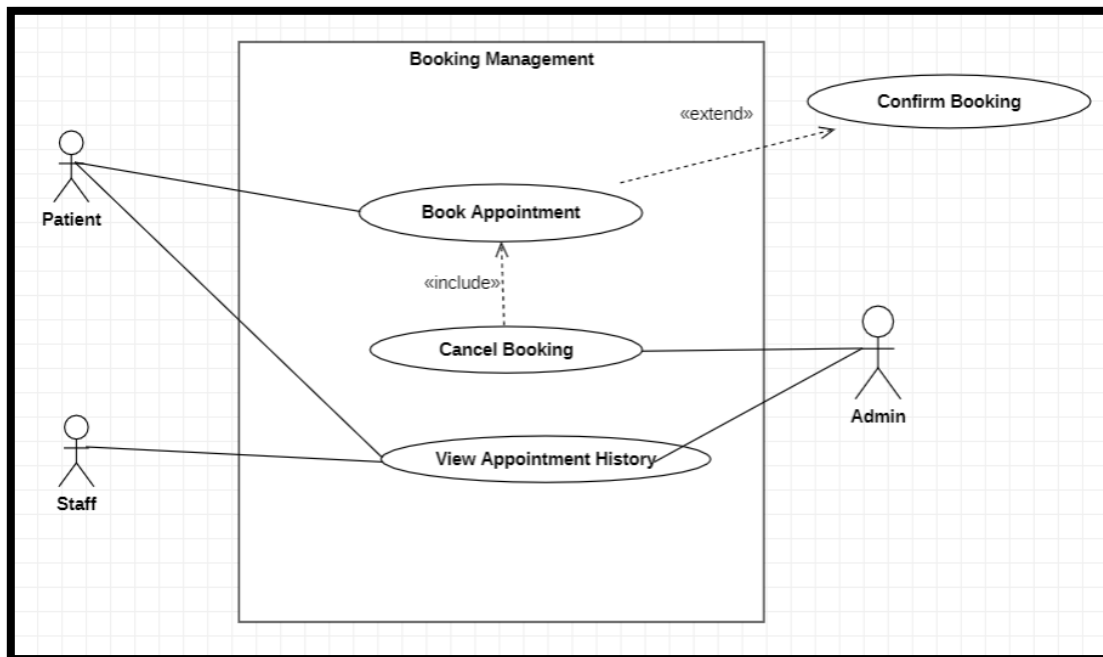


Figure 85: Use Case for Booking Appointment.

- Use Case for Payment.

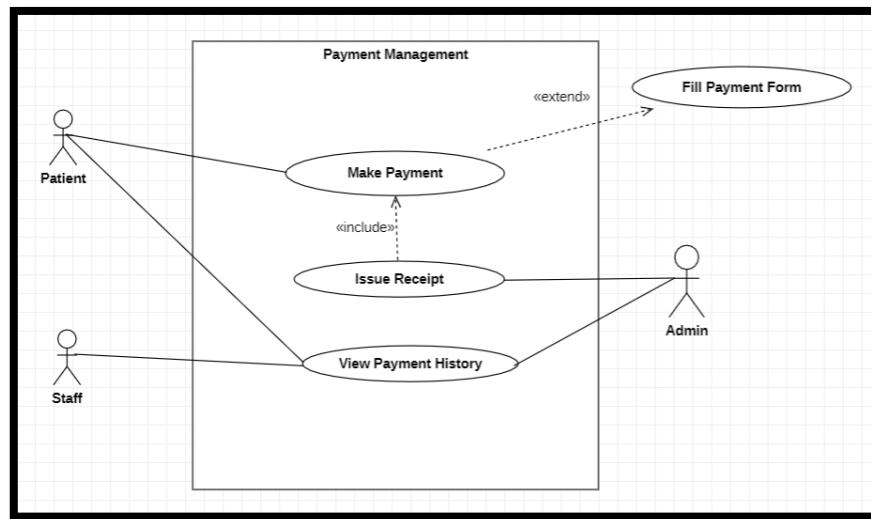


Figure 86: Use Case for Payment.

- Activity Diagram for Booking Appointment.

This is the Activity Diagram of this system. That visually represents the process in the system. it shows the sequence of activities that a user follows to successfully make an appointment. This activity diagram clarifies the steps required for successful booking and help developers and analysts understand what needs to be implemented in the system.

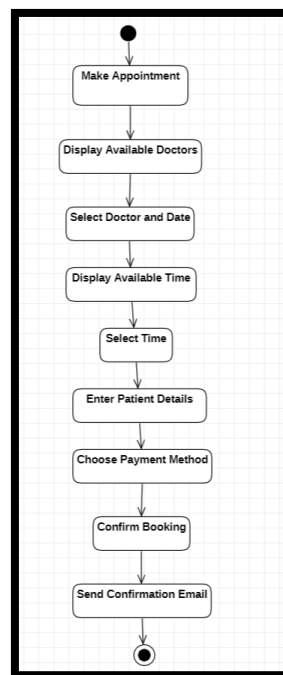


Figure 87: Activity Diagram for Booking Appointment.

- Activity Diagram for Payment.

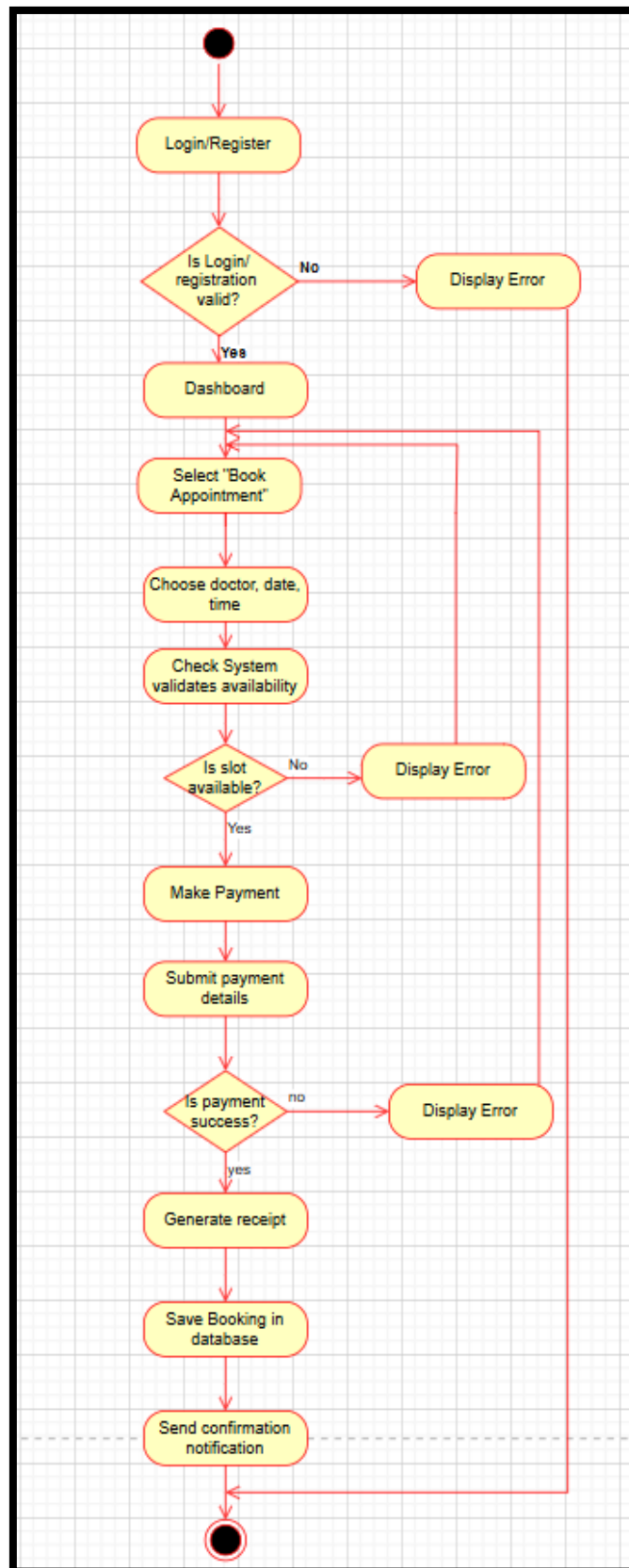


Figure 88: Activity Diagram for Payment.

- Sequence Diagram for Booking Appointment.

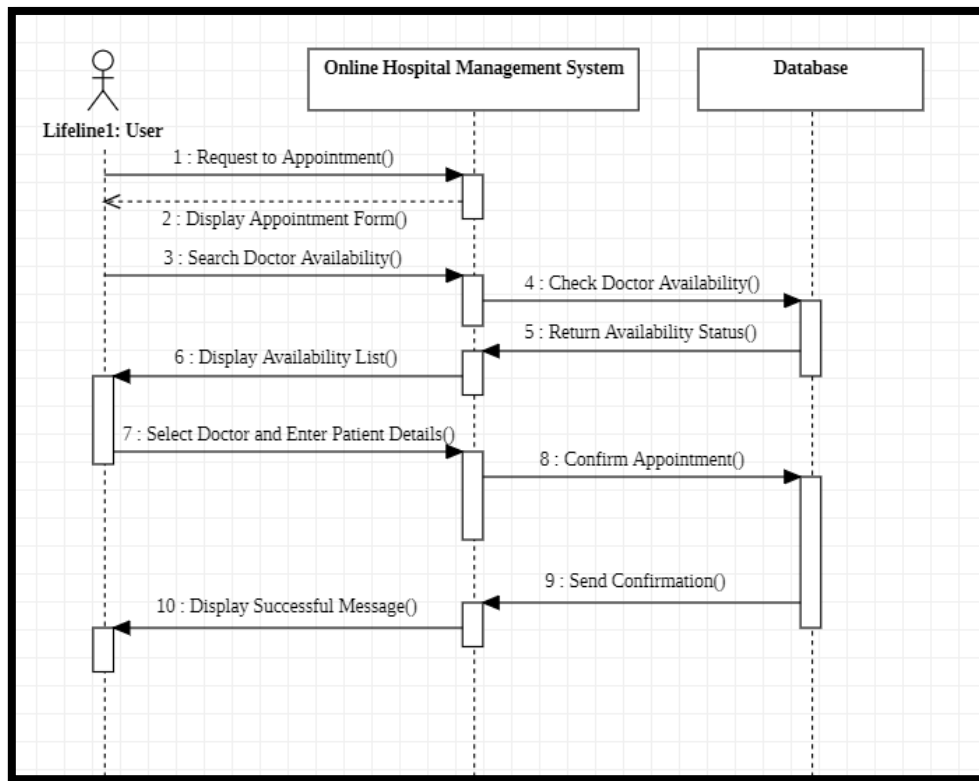


Figure 89: Sequence Diagram for Booking Appointment.

- Sequence Diagram for Payment.

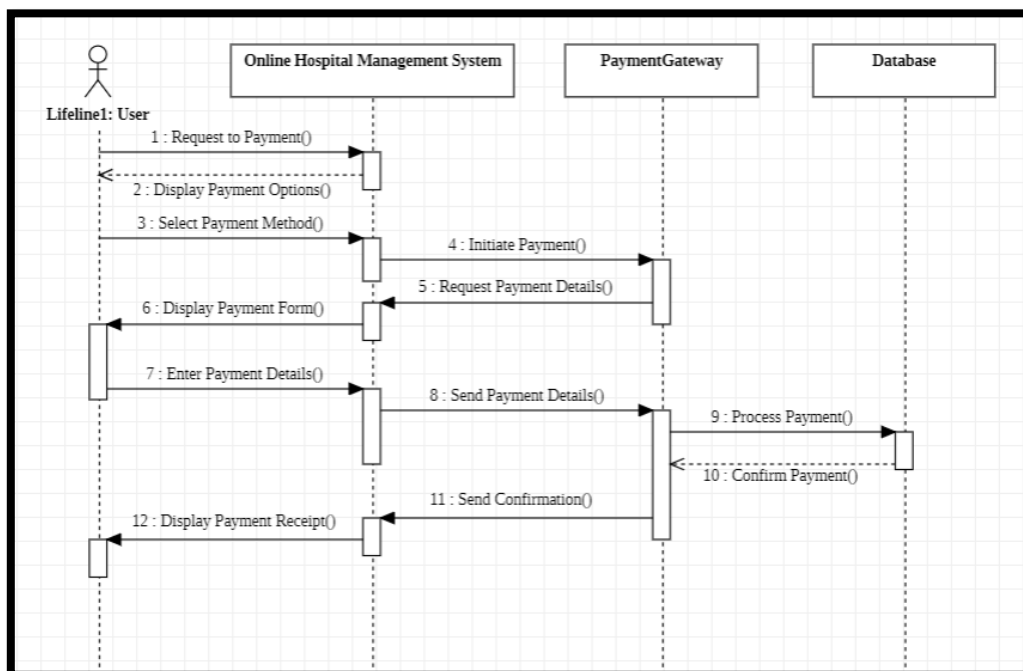


Figure 90: Sequence Diagram for Payment.

Appendix C: Issues List.

- Designing the database schema to accurately represent the hospital's data was challenging. HealthCare Plus required intricate relationships between reports, appointment, charges, doctors and more. Ensuring data consistency and referential integrity in MySQL was a significant challenge.
- Creating comprehensive documentation for the system was time-consuming but necessary. It involved documenting system architecture, database schema and user manuals.
- Errors occur when developing the system. So, the developer faced so many issues. The developer should have solved the errors one by one. It's a problem for time management.
- These issues, while demanding also provided valuable learning experiences for the developer.